



Departamento de
Computação - **UFSCar**



Departamento de Computação
Centro de Ciências Exatas e Tecnologia
Universidade Federal de São Carlos

Projeto e Análise de Algoritmos

Estratégias para análise, projeto de algoritmos e soluções aproximadas
para problemas NP-completos

Prof. Alexandre Luis Magalhães Levada
Email: alexandre.levada@ufscar.br

Sumário

Análise Assintótica.....	3
Relação com a Análise de Casos.....	9
Dividir para conquistar.....	16
Árvores de recursão.....	16
O Problema da Multiplicação de Inteiros.....	18
O algoritmo de Karatsuba.....	21
O algoritmo Mergesort.....	23
O Problema da Multiplicação de Matrizes.....	25
O algoritmo de Strassen.....	28
O problema do par de pontos mais próximos.....	30
A Transformada de Fourier Discreta.....	37
O algoritmo Fast Fourier Transform (FFT).....	41
O Teorema Mestre.....	44
Programação Dinâmica.....	49
A série de Fibonacci.....	49
O problema da sequência de cédulas.....	52
O problema do corte da haste (The rod cutting problem).....	55
O problema do robô coletor de moedas.....	61
O algoritmo Floyd-Warshall (All-pairs shortest path).....	65
Algoritmos Gulosos (Greedy algorithms).....	73
Árvores geradoras mínimas.....	74
O algoritmo de Kruskal.....	78
Algoritmo de Prim.....	83
O algoritmo de compactação de Huffman.....	88
Problemas NP-Completo.....	96
Grafos Hamiltonianos.....	98
O problema do caixeiro-viajante.....	99
Coloração de vértices.....	116
Apêndice A: Programação dinâmica e o algoritmo DTW.....	125
Apêndice B: Algoritmos gulosos e o problema do fluxo máximo.....	129
Bibliografia.....	141
Sobre o autor.....	142

The ultimate test of your knowledge is your capacity to convey it to another.
Richard Feynman

Análise Assintótica

Trata-se de uma ferramenta matemática para avaliar a performance de algoritmos em termos do tamanho da entrada n , contando o número de instruções a serem executadas.

Vantagem: não depende do hardware da máquina como o tempo de execução.

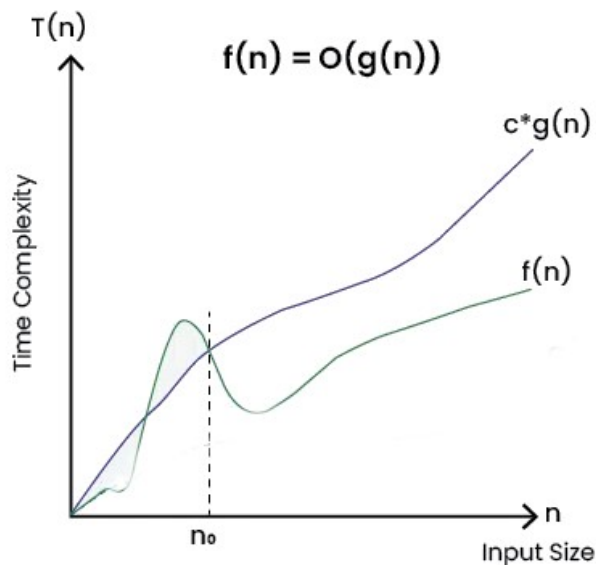
Na análise de algoritmos, o estudo do crescimento assintótico de funções deve ser empregado pois desejamos verificar o comportamento dos algoritmos para grandes valores de n (tamanho da entrada cresce arbitrariamente). Existem 3 notações básicas para análise assintótica de funções: Big-O, notação Ω e notação Θ . A seguir definiremos cada uma delas.

Notação Big-O

Representa o limite superior do tempo de execução de um algoritmo. Abordagem pessimista, especialmente adequada para estudar como algoritmo se comporta no pior cenário.

$$f(n) = O(g(n)) \text{ se há constantes positivas } c \text{ e } n_0 \text{ tais que } f(n) \leq c g(n) \quad \forall n \geq n_0$$

Em outras palavras, se para todo n maior que um limite arbitrário, $f(n)$ é limitada superiormente por $c g(n)$.



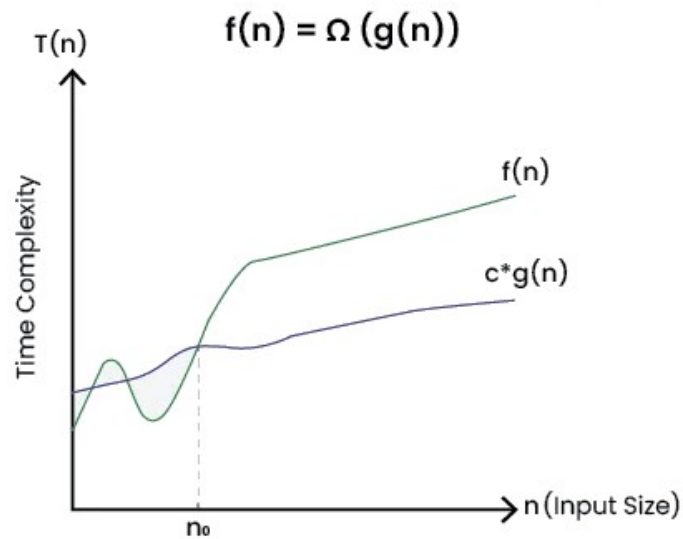
Um exemplo é o algoritmo de ordenação InsertionSort que é linear no melhor caso e quadrático no pior caso. Como ele nunca é pior que complexidade quadrática, podemos dizer que no geral esse algoritmo é $O(n^2)$.

Notação Ω

Representa o limite inferior do tempo de execução de um algoritmo. Abordagem otimista, empregada para estudar como um algoritmo se comporta no melhor cenário.

$$f(n) = \Omega(g(n)) \text{ se há constantes positivas } c \text{ e } n_0 \text{ tais que } f(n) \geq c g(n) \quad \forall n \geq n_0$$

Em outras palavras, se para todo n maior que um limite arbitrário, $f(n)$ é limitada inferiormente por $c g(n)$.



Por exemplo, o algoritmo InsertionSort possui complexidade sempre maior que linear, então podemos escrever que esse algoritmo é $\Omega(n)$.

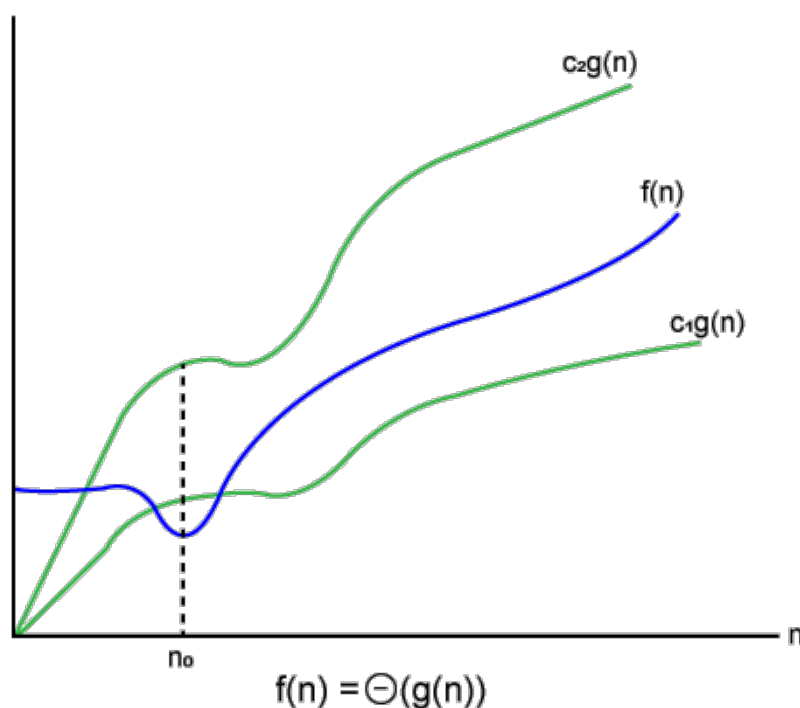
Notação Θ

Essa notação limita a função acima e abaixo simultaneamente. Representa o limite superior e inferior do algoritmo em questão, especialmente adequada para estudar o comportamento de algoritmos no caso médio.

$f(n) = \Theta(g(n))$ se há constantes positivas c_1, c_2 e n_0 tais que:

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \text{para todo } n \geq n_0$$

Na prática, é como se a função $f(n)$ ficasse prensada entre $c_1 g(n)$ e $c_2 g(n)$ (limitada acima e abaixo). A figura a seguir ilustra esse comportamento.



Propriedades da notação assintótica

1. Se $f(n) = O(g(n))$ então $af(n) = O(g(n))$ para uma constante a

$$\begin{aligned} f(n) &= 2n^2 + 5 \text{ é } O(n^2) \\ 7f(n) &= 14n^2 + 35 \text{ também é } O(n^2) \end{aligned}$$

O mesmo vale para notações Ω e Θ .

2. Transitividade: se $f(n) = O(g(n))$ e $g(n) = O(h(n))$, então $f(n) = O(h(n))$.

$$\begin{aligned} \text{Seja } f(n) &= n, \quad g(n) = n^2 \text{ e } h(n) = n^3. \\ n &= O(n^2) \text{ e } n^2 = O(n^3), \text{ então } n = O(n^3) \end{aligned}$$

O mesmo vale para notações Ω e Θ .

3. Se $f(n) = O(g(n))$, então $g(n) = \Omega(f(n))$

$$\begin{aligned} \text{Seja } f(n) &= n \text{ e } g(n) = n^2 \\ n &= O(n^2) \text{ e } n^2 = \Omega(n) \end{aligned}$$

Esta propriedade vale apenas para Big-O e Ω .

4. Se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$, então $f(n) = \Theta(g(n))$

5. Se $f(n) = O(g(n))$ e $d(n) = O(e(n))$, então:

$$f(n) + d(n) = O(\max\{g(n), e(n)\})$$

6. Se $f(n) = O(g(n))$ e $d(n) = O(e(n))$, então $f(n) * d(n) = O(g(n) * e(n))$

$$\text{Se } f(n) = n \text{ e } d(n) = n^2, \text{ então } f(n) * d(n) = O(n^3).$$

Obs: Matematicamente, dizemos que $O(n^2)$ representa um conjunto de funções. Alguns exemplos de funções que pertencem a esse conjunto são:

$$\begin{aligned} f(n) &= n^2 + n \\ f(n) &= n^2 + 1000n \\ f(n) &= 1000n^2 + 1000n \\ f(n) &= n \\ f(n) &= n^{1.999} \\ f(n) &= \frac{n^2}{\log n} \end{aligned}$$

O mesmo vale para $\Omega(n^2)$, que é um conjunto de funções. Exemplos de funções pertencentes a esse conjunto são:

$$\begin{aligned} f(n) &= n^2 - n \\ f(n) &= n^2 + n \end{aligned}$$

$$f(n) = 1000n^2 - 1000$$

$$f(n) = n^3$$

$$f(n) = n^{2.01}$$

$$f(n) = 2^n$$

A intuição é que podemos usar notações assintóticas como uma abstração de operadores relacionais:

$f(n)$ é $O(g(n))$ é aproximadamente o mesmo que $f(n) \leq g(n)$

$f(n)$ é $\Omega(g(n))$ é aproximadamente o mesmo que $f(n) \geq g(n)$

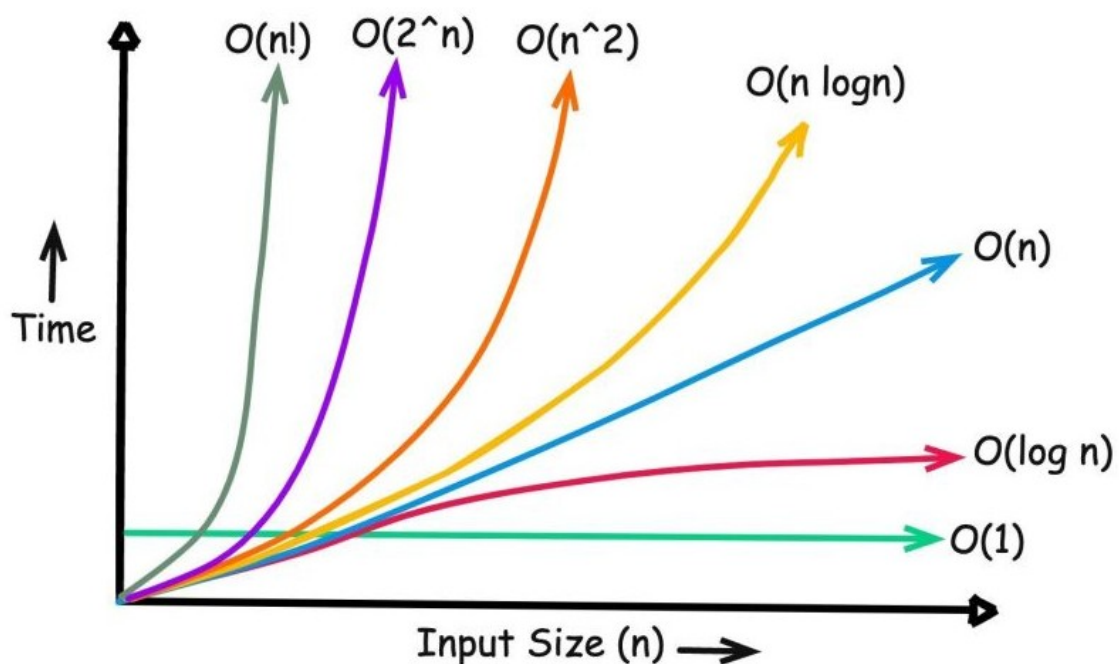
$f(n)$ é $\Theta(g(n))$ é aproximadamente o mesmo que $f(n) = g(n)$

Na prática, algumas regras que nos ajudam a simplificar funções omitindo termos dominados são:

1. Constantes multiplicadoras podem ser descartadas: $100n^2$ se torna n^2
2. A função n^a domina n^b , se $a > b$
3. Exponenciais dominam qualquer polinômio: 3^n domina n^8 , 3^n domina 2^n
4. Qualquer polinômio domina logaritmos: n domina $(\log n)^3$, n^2 domina $n \log n$

Principais classes de funções

Função	Classe
1	Constante
$\log n$	Logarítmica
n	Linear
$n \log n$	Log-Linear
n^k	Polinomial
k^n	Exponencial
$n!$	Fatorial



Analise de algoritmos

Objetivo: contar o número de atribuições no algoritmo A.

Hipóteses:

1. Cada comando de atribuição é $O(1)$
2. A função $T(n)$ deve contar o número de atribuições total a serem executadas quando entrada possui tamanho n

Considere o exemplo a seguir, com duas estruturas de repetição.

```
FuncA(n){  
    c = 0  
    for i = 1 to n  
        c = c + 1  
    for j = 1 to 2*n  
        c = c + 2  
    return c  
}
```

Note que temos uma atribuição inicial (1) e logo dois loops: o primeiro com n iterações e o segundo com $2n$ iterações. Assim, temos $T(n) = 3n + 1$, o que resulta em uma complexidade $O(n)$. Neste caso podemos escrever que: $T(n) = O(n)$ ou $T(n) = \Omega(n)$ ou ainda $T(n) = \Theta(n)$. Na verdade, poderíamos dizer que essa função é $\Omega(1)$ ou $O(n^2)$, mas busca-se os limites mais justos possíveis. De fato, qualquer função é $\Omega(1)$, mas isso não nos diz absolutamente nada sobre o comportamento assintótico dela. Considere o algoritmo a seguir:

```
Func_B(n) {  
    c = 0  
    for i = 1 to n {  
        for j = 1 to n  
            c = c + 1  
        }  
    return c  
}
```

Nesse caso, o loop interno tem n operações. Como o loop externo é executado n vezes, e temos uma inicialização, o total de operações é $T(n) = n^2 + 1$, o que resulta em $\Theta(n^2)$. Considere esse código em que o loop interno executa um número variável de vezes.

```
Func_C(n) {  
    count = 0  
    for i = 1 to n {  
        for j = 1 to i  
            c = c + 1  
        }  
    return c  
}
```

Note que quando $i = 0$, o loop interno executa uma vez, quando $n = 1$, o loop interno executa duas vezes, quando $n = 2$, o loop interno executa 3 vezes, e assim sucessivamente. Assim, o número de vezes que a variável c é incrementada é igual a: $1 + 2 + 3 + 4 + \dots + n$

Devemos resolver esse somatório para calcular a complexidade dessa função.

$$\sum_{k=1}^n k$$

Primeiramente, note que

$$(k+1)^2 = k^2 + 2k + 1$$

o que implica em

$$(k+1)^2 - k^2 = 2k + 1$$

Assim, $\sum_{k=1}^n [(k+1)^2 - k^2] = \sum_{k=1}^n [2k + 1]$. Porém, o lado esquerdo é uma soma telescópica e temos:

$$\sum_{k=1}^n [(k+1)^2 - k^2] = (n+1)^2 - 1$$

Dessa forma, podemos escrever:

$$(n+1)^2 - 1 = \sum_{k=1}^n [2k + 1]$$

Aplicando a propriedade associativa, temos:

$$(n+1)^2 - 1 = 2 \sum_{k=1}^n k + \sum_{k=1}^n 1$$

o que nos leva a:

$$2 \sum_{k=1}^n k = n^2 + 2n + 1 - 1 - n = n^2 + n$$

Finalmente, colocando n em evidência e dividindo por 2, finalmente temos:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

Portanto, T(n) é igual a:

$$T(n) = \frac{1}{2}(n^2 + n) + 1$$

o que resulta em $\Theta(n^2)$.

Considere a função em Python a seguir.

```

Func_D(n) {
    c = 0
    i = n
    while i > 1 {
        c = c + 1
        i = i // 2      # divisão inteira
    }
    return c
}

```

Essa função calcula quantas vezes o número pode ser dividido por 2. Por exemplo, considere a entrada $n = 16$. Em cada iteração esse valor será dividido por 2, até que atinja o zero.

$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

A variável c termina a função valendo 4, pois $2^4 = 16$.

Se $n = 25$, temos:

$25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 1$

A variável c termina a função valendo 5, pois $2^4 < 25 < 2^5$

Se $n = 40$, temos:

$40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2 \rightarrow 1$

A variável c termina a função valendo 5, pois $2^5 < 40 < 2^6$

Portanto, o número de iterações do loop é $\log_2 n$. Dentro do loop existem duas instruções, portanto neste caso teremos:

$T(n) = 2 + 2\lfloor \log_2 n \rfloor$, onde a função piso(x) retorna o maior inteiro menor que x .

o que resulta em $\Theta(\log_2 n)$.

Considere a função em Python a seguir.

```

Func_E(n) {
    c = 0
    for i = 1 to n
        c = c + Func_D(n)
    return c
}

```

Note que, como a função $\text{FuncD}(n)$ tem complexidade logarítmica, e o loop tem n iterações, temos que a complexidade da função em questão é $\Theta(n \log_2 n)$.

Relação com a Análise de Casos

As notações assintóticas Big-O, Big Omega (Ω) e Big Theta (Θ) são fundamentais para descrever a eficiência de algoritmos ao analisar seu comportamento em relação ao crescimento do tamanho da entrada. Elas estão diretamente relacionadas à análise de complexidade, mas possuem papéis

distintos, além de se conectarem com os conceitos de pior caso, melhor caso e caso médio. Vamos explorar essa relação:

A análise de complexidade de algoritmos considera três cenários principais:

Pior caso: Maior tempo de execução para qualquer entrada de tamanho n .

Melhor caso: Menor tempo de execução para qualquer entrada de tamanho n .

Caso médio: Tempo esperado para uma entrada aleatória de tamanho n .

As notações assintóticas são ferramentas matemáticas para descrever esses cenários, mas não são sinônimos diretos deles.

a) Big-O e o Pior Caso

Big-O é frequentemente usado para descrever o pior caso, pois define um limite superior.

Exemplo: O algoritmo de ordenação BubbleSort tem complexidade $O(n^2)$ no pior caso.

b) Big Omega e o Melhor Caso

Big Omega é usado para descrever o melhor caso, pois define um limite inferior.

Exemplo: O algoritmo Binary Search tem complexidade $\Omega(1)$ no melhor caso (encontra o elemento na primeira tentativa).

c) Big Theta e a Precisão

Big Theta é usado quando o melhor e o pior caso coincidem ou quando se deseja uma descrição exata.

Exemplo: O algoritmo Merge Sort tem complexidade $\Theta(n \log n)$ em todos os casos (pior, médio e melhor).

d) Caso Médio

O caso médio pode ser descrito com qualquer notação, dependendo do contexto.

Exemplo: O Quick Sort tem complexidade média $\Theta(n \log n)$, mas seu pior caso é $O(n^2)$.

Algumas observações importantes:

-> Notações $\neq$ Cenários: Big-O, Ω e Θ são ferramentas matemáticas para descrever limites de funções. Elas podem ser aplicadas a qualquer cenário (pior, melhor ou médio).

Exemplo: É possível dizer que o melhor caso de um algoritmo é $O(n)$ (limite superior do melhor desempenho) ou que o pior caso é $\Omega(n)$ (limite inferior do pior desempenho).

-> Uso Comum: Por convenção, Big-O é associado ao pior caso (para garantir segurança), e Big Omega ao melhor caso. Já Big Theta é usado quando o comportamento é "exato".

Outro exemplo é dado a seguir. Considere um algoritmo de busca em lista:

Melhor caso: Encontra o elemento na primeira posição $\rightarrow \Theta(1)$.

Pior caso: O elemento está na última posição ou não existe $\rightarrow \Theta(n)$.

Caso médio: Espera-se percorrer metade da lista $\rightarrow \Theta(n)$.

Neste caso, temos:

Big-O do pior caso: $O(n)$.

Big Omega do melhor caso: $\Omega(1)$.

Big Theta para pior/médio: $\Theta(n)$.

As notações assintóticas permitem quantificar matematicamente a eficiência de algoritmos em diferentes cenários. Enquanto Big-O, Ω e Θ definem limites de crescimento, a análise de pior, melhor e caso médio define quais cenários de entrada estão sendo considerados. A relação entre ambos é que as notações assintóticas são usadas para descrever formalmente a complexidade em cada um desses cenários.

Em resumo, a notação assintótica ajuda a caracterizar diferentes aspectos do desempenho do algoritmo, sendo usada de forma apropriada para descrever limites superiores, inferiores e exatos do tempo de execução.

A seguir, resolveremos alguns exercícios sobre análise assintótica.

Exercício: Mostre que, se c é um número real positivo, então:

$$g(n) = \sum_{k=0}^n c^k$$

é uma função:

a) $\Theta(1)$, se $c < 1$

Neste caso, temos que $c < 1$, o que nos leva a:

$$g(n) = 1 + c + c^2 + c^3 + \dots + c^n$$

Isso representa a soma de uma P.G, em que $a_1 = 1$ e a razão $q = c$. Em outras palavras, temos que $g(n) = S_{n+1}$, onde S_{n+1} denota a soma dos $n + 1$ primeiros termos da P.G. Sabemos que:

$$a_1 = 1, \quad a_2 = a_1 q, \quad a_3 = a_1 q^2, \quad \dots, \quad a_n = a_1 q^{n-1}, \quad a_{n+1} = a_1 q^n$$

o que nos leva a:

$$S_{n+1} = a_1 + a_1 q + a_1 q^2 + a_1 q^3 + \dots + a_1 q^n \quad (\text{I})$$

Multiplicando a equação (I) por q , chega-se em:

$$q S_{n+1} = a_1 q + a_1 q^2 + a_1 q^3 + a_1 q^4 + \dots + a_1 q^{n+1} \quad (\text{II})$$

Calculando (I) – (II), temos:

$$S_{n+1} - q S_{n+1} = a_1 - a_1 q^{n+1}$$

uma vez que os termos intermediários cancelam-se. Manipulando a equação anterior, temos:

$$S_{n+1}(1-q) = a_1(1-q^{n+1})$$

o que finalmente nos leva a:

$$S_{n+1} = \frac{a_1(1-q^{n+1})}{(1-q)}$$

Lembrando que $q = c$, temos:

$$g(n) = S_{n+1} = \frac{(1-c^{n+1})}{(1-c)}$$

Mas como $c < 1$, para $n \rightarrow \infty$ (n grande o suficiente), o numerador tende a 1 e o denominador é uma constante $1 - c$:

$$g(n) = \frac{1}{(1-c)} \quad \text{o que é } \Theta(1) .$$

b) $\Theta(n)$, se $c = 1$

Se $c = 1$, note que podemos escrever $g(n)$ como:

$$g(n) = 1 + 1^1 + 1^2 + 1^3 + \dots + 1^n = n + 1$$

o que é $\Theta(n)$.

c) $\Theta(c^n)$, se $c > 1$

Se $c > 1$, note que podemos escrever $g(n)$ como:

$$g(n) = 1 + c^1 + c^2 + c^3 + \dots + c^n$$

o que define um polinômio de grau n em c . Portanto, como o termo dominante é c^n , temos que $g(n)$ é $\Theta(c^n)$.

=> Moral da história: em termos de notação assintótica, a soma de uma P.G. é igual ao primeiro termo se a série é decrescente, é igual ao último termo se a série é crescente, ou é igual ao número de termos da série se a série é constante.

Exercício: Mostre que para quaisquer constantes reais $a, b > 0$ $(n+a)^b$ é $\Theta(n^b)$.

Para $(n+a)^b$ ser $\Theta(n^b)$ então deve existir constantes $c_1, c_2, n_0 > 0$ tal que:

$$c_1 n^b \leq (n+a)^b \leq c_2 n^b$$

para todo $n > n_0$.

Primeiramente, note que quando $a \leq n$, $n+a \leq 2n$ (I).

Note também que quando $a \leq \frac{n}{2}$ ou seja, $n \geq 2a$, temos que $n+a \geq n-a \geq \frac{n}{2}$ (II)

Então, para $n \geq n_0 = 2a$, vale a seguinte desigualdade:

$$\frac{n}{2} \leq n+a \leq 2n$$

o que implica em:

$$\left(\frac{n}{2}\right)^b \leq (n+a)^b \leq (2n)^b$$

pois $b > 0$. Portanto, podemos escrever:

$$\left(\frac{1}{2}\right)^b n^b \leq (n+a)^b \leq 2^b n^b$$

em que $c_1 = \left(\frac{1}{2}\right)^b$, $c_2 = 2^b$ e $n_0 = 2a$, o que mostra que $(n+a)^b$ é $\Theta(n^b)$.

Fatoriais

Sabemos que o fatorial de um número n é definido por:

$$n! = \begin{cases} 1, & n=0 \\ n(n-1)!, & n>0 \end{cases}$$

ou seja,
$$n! = \prod_{i=1}^n i$$

Note que

$$n! = n(n-1)(n-2)\dots 1 \leq n \times n \times n \times \dots \times n = n^n$$

o que implica em $n! \leq n^n$ e portanto a conclusão de que $n!$ é $O(n^n)$.

Aproximação de Stirling

Pode-se mostrar que o fatorial de um inteiro pode ser computado por:

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$$

Então, para $n > 10$, $\Theta\left(\frac{1}{n}\right)$ torna-se desprezível, o que nos leva a:

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

Dessa forma, note que:

$$\log n! = \log(2\pi n)^{1/2} + n \log\left(\frac{n}{e}\right) = \frac{1}{2} \log 2\pi + \frac{1}{2} \log n + n \log n + n \log e$$

Como o primeiro e o último termos da soma são constantes e $n \log n$ domina $\log n$, chegamos a conclusão de que $\log n!$ é $O(n \log n)$.

Limites para comparação assintótica

Podemos comparar o crescimento assintótico de duas funções calculando o limite da razão entre elas. Apenas 3 casos podem ocorrer:

1. $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$: significa que $f(n)$ cresce menos que $g(n)$, ou seja, $f(n) = O(g(n))$
2. $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$: significa que $f(n)$ cresce proporcional a $g(n)$, ou seja, $f(n) = \Theta(g(n))$
3. $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$: significa que $f(n)$ cresce mais rápido que $g(n)$, ou seja, $f(n) = \Omega(g(n))$

Regra de L'Hôpital

Pode-se mostrar que:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$$

onde $f'(n)$ denota a derivada da função f . Esse resultado é útil quando no cálculo de limites ocorrem indeterminações do tipo $0/0$, ∞/∞ , etc.

Compare o crescimento assintótico das funções $f(n) = \frac{1}{2}n(n-1)$ e $g(n) = n^2$.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \frac{1}{2} \lim_{n \rightarrow \infty} \frac{n^2 - n}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right) = \frac{1}{2}$$

Portanto, temos que $f(n) = \Theta(g(n))$.

Compare o crescimento assintótico das funções $f(n) = \log_2 n$ e $g(n) = \sqrt{n}$.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{\log_2 n}{\sqrt{n}} = \lim_{n \rightarrow \infty} \frac{(\log_2 n)'}{(\sqrt{n})'}$$

Para calcular a derivada de $f(n)$, devemos mudar a base para e ($\ln$):

$$\log_2 n = \frac{\ln n}{\ln 2}$$

Derivando em relação a n , temos:

$$\log_2 n = \left(\frac{\ln n}{\ln 2} \right)' = \frac{1}{\ln 2} \frac{1}{n}$$

Para o denominador temos:

$$(\sqrt{n})' = (n^{1/2})' = \frac{1}{2\sqrt{n}}$$

Assim, podemos escrever:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 2 \frac{1}{\ln 2} \frac{\lim_{n \rightarrow \infty} \sqrt{n}}{n} = 2 \frac{1}{\ln 2} \lim_{n \rightarrow \infty} \frac{1}{\sqrt{n}} = 0$$

Portanto, temos que $f(n) = \Omega(g(n))$, ou seja $f(n)$ cresce menos que $g(n)$. Isso significa que $\sqrt{n}$ cresce mais rápido que $\log n$, ou seja, $g(n) = O(f(n))$.

Exercício: Compare as ordens de crescimento das funções $f(n) = n!$ e $g(n) = 2^n$.

"Enquanto o conhecimento custa o seu tempo no presente, a ignorância custa todo seu futuro."
-- Autor desconhecido

Dividir para conquistar

Ao projetar algoritmos, podemos empregar diversas estratégias para torná-los mais eficientes. A estratégia dividir para conquistar (divide and conquer) é uma delas.

Trata-se de uma estratégia para projetar algoritmos assintoticamente eficientes.

Utiliza-se a recursão para simplificar um problema em instâncias menores e mais simples.

Na análise da complexidade, podemos utilizar um ferramental matemático para a resolução de recorrências.

Estratégia dividir para conquistar

Para aplicar a estratégia dividir para conquistar devemos seguir alguns passos:

1. Caso base: se o problema é pequeno o suficiente, resolva-o diretamente (para a recursão).

2. Caso recursivo: caso contrário, realize 3 passos

- a) Divida o problema em subproblemas que são instâncias menores do problema original.
- b) Conquiste os subproblemas, resolvendo-os recursivamente.
- c) Combine as soluções dos subproblemas para formar a solução do problema original.

Análise da complexidade: construir recorrências que descrevem o número de instruções de algoritmos que utilizam a estratégia dividir para conquistar. Nesse contexto, saber resolver recorrências é importante para podermos comparar a eficiência de diferentes algoritmos: por exemplo, suponha 2 algoritmos A e B com as seguintes recorrências:

$$T(n) = 8T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n^2)$$

Pergunta-se: qual deles é o mais eficiente assintoticamente? Veremos a seguir o método da árvore de recursão e como podemos utilizá-lo para responder perguntas como esta.

Árvores de recursão

Decompor problema original (raiz da árvore) em uma sequência de subproblemas menores. Somando os custos dentro de cada nível da árvore temos o custo de uma recursão. Ao somar todos os níveis, chega-se ao custo total do algoritmo. Vejamos um exemplo ilustrativo. Seja a seguinte recorrência:

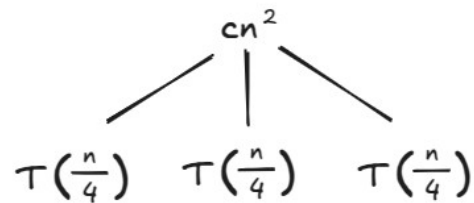
$$T(n) = 3T\left(\frac{n}{4}\right) + O(n^2)$$

Essa recorrência nos diz que cada problema é dividido em 3 subproblemas de tamanho $n/4$ e o custo computacional para combinar as soluções dos subproblemas é quadrática. Se temos uma

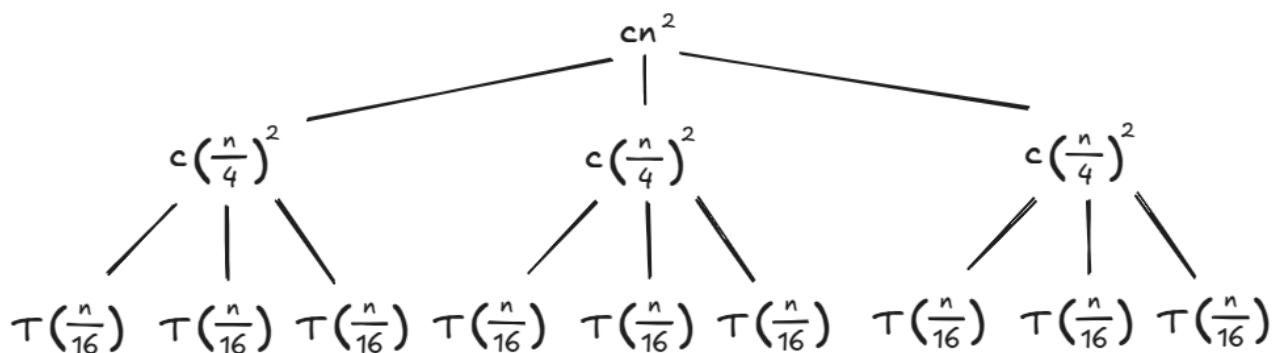
complexidade $O(n^2)$, significa que ela é limitada superiormente por cn^2 . Assim, a recorrência fica:

$$T(n) = 3T\left(\frac{n}{4}\right) + cn^2$$

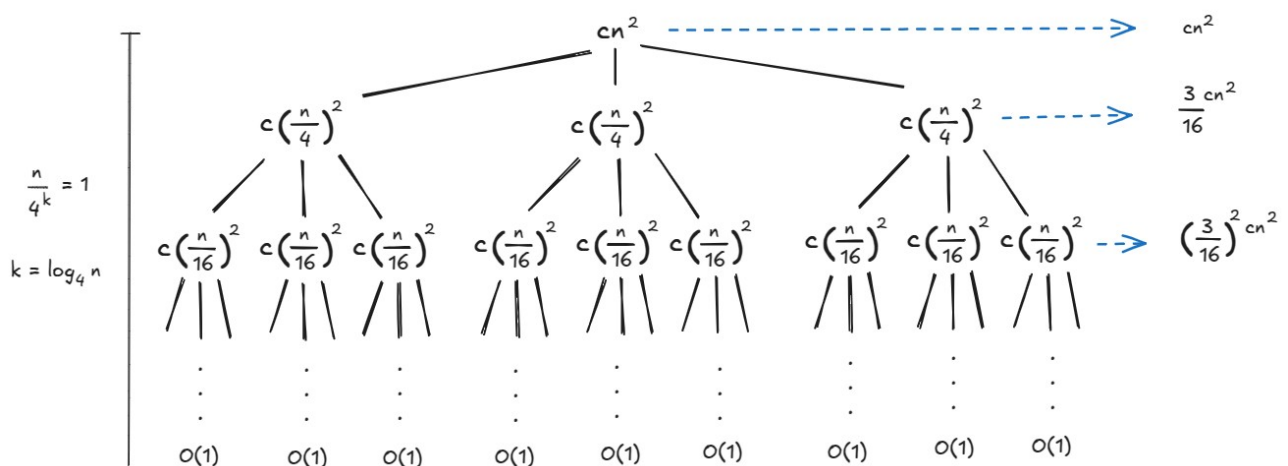
Podemos crescer uma árvore de recursão com raiz $T(n)$. Expandindo o primeiro nível da árvore, temos a seguinte construção:



A expansão do segundo nível, gera:



Continuando com o processo, chega-se a seguinte árvore de recursão:



Note que a condição de parada da recursão é:

$$\frac{n}{4^k} = 1$$

onde k é a altura da árvore. Isso implica que a altura da árvore é $k = \log_4 n$. Note que os números de nós em cada nível formam a seguinte sequência: 1, 3, 9, 27, 81, ... Assim, o número de nós folha na última camada será:

$$F = 3^k = 3^{\log_4 n}$$

Somando os custos obtidos em cada nível da árvore de recursão, chega-se a:

$$T(n) = cn^2 + \left(\frac{3}{16}\right)cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \dots + \left(\frac{3}{16}\right)^{\log_4 n} cn^2 = \sum_{i=0}^{\log_4 n} \left(\frac{3}{16}\right)^i cn^2$$

Note que a expressão define a soma de uma P.G. de razão menor que 1. Fazendo o número de termos da P.G. ser igual a infinito, temos um limite superior para $T(n)$:

$$T(n) \leq \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2$$

A soma de uma PG infinita é dada por:

$$S_{\infty} = \frac{1}{1-q}$$

onde $q < 1$ é a razão da PG.

Sendo assim, a soma infinita é dada por:

$$S_{\infty} = \frac{1}{1-3/16} = \frac{16}{13}$$

o que nos leva a :

$$T(n) \leq \frac{16}{13} cn^2$$

o que é $O(n^2)$.

O Problema da Multiplicação de Inteiros

Nesta seção, iremos projetar e analisar um algoritmo mais eficiente para a multiplicação de números inteiros. Primeiramente, lembre-se que a multiplicação de 2 inteiros de n dígitos requer n^2 multiplicações e 2 adições:

$\begin{array}{r} 384 \\ \times 156 \\ \hline 2304 \\ 1920+ \\ 384+ \\ \hline 59904 \end{array}$	3 dígitos 3 dígitos ($n = 3$) 9 multiplicações + 2 adições $O(n^2)$
--	--

A pergunta é: podemos fazer melhor que isso? Iremos tentar.

Sem perda de generalidade, é usual assumir que $n=2^m$ (potência de 2) para simplificar as coisas. Assim, sempre podemos particionar a e b em suas metades superiores e inferiores.

Por hora, vamos considerar os números a seguir:

$$\begin{aligned}a &= 12345678 \\ b &= 87654321\end{aligned}$$

Note que podemos escrever:

$$\begin{aligned}a &= 12340000 + 5678 \\ b &= 87650000 + 4321\end{aligned}$$

Logo, o produto entre a e b pode ser expresso como:

$$a \times b = (1234 \times 10^4 + 5678)(8765 \times 10^4 + 4321)$$

Aplicando a propriedade distributiva:

$$a \times b = 1234 \times 8765 \times 10^8 + (1234 \times 4321 + 5678 \times 8765) \times 10^4 + 5678 \times 4321$$

Usando essa notação, podemos escrever:

$$\begin{aligned}a &= a_1 10^{n/2} + a_2 \\ b &= b_1 10^{n/2} + b_2\end{aligned}$$

onde a_1 e a_2 são as partes superiores e inferiores de a e b_1 e b_2 são as partes superiores e inferiores de b. A multiplicação é dada:

$$a \times b = A \times 10^n + (B + C) \times 10^{n/2} + D$$

onde

$$\begin{aligned}A &= a_1 \times b_1 \\ B &= a_2 \times b_1 \\ C &= a_1 \times b_2 \\ D &= a_2 \times b_2\end{aligned}$$

A função a seguir utiliza uma estratégia recursiva para implementar uma solução alternativa para a multiplicação de inteiros.

```
def Multiply(a, b):  
    if len(a) <= 1:  
        return a*b  
    Particione a e b em  
        a = a1*10n/2 + a2 # essa partição requer um único loop - O(n)  
        b = b1*10n/2 + b2  
    A = Multiply(a1, b1)  
    B = Multiply(a1, b2)  
    C = Multiply(a2, b1)
```

```

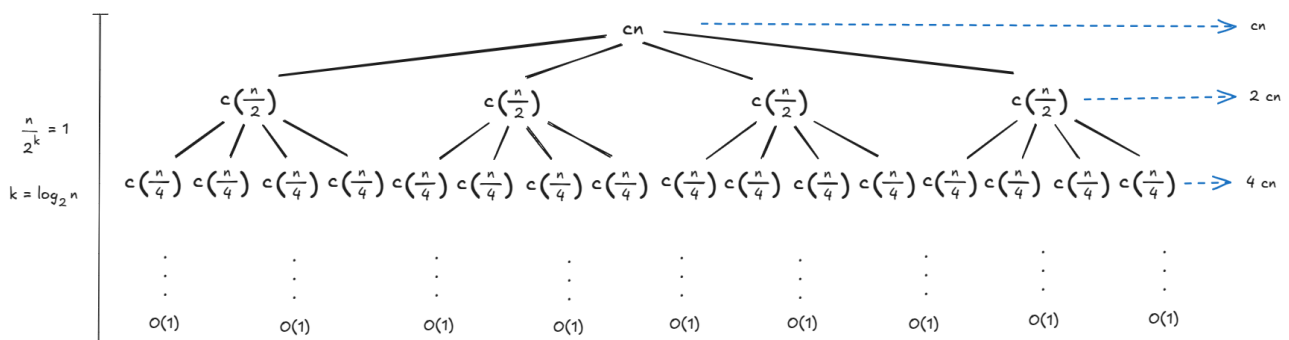
D = Multiply(a2, b2)
P = A × 10n + (B + C) × 10n/2 + D
return P
}

```

Note que um problema de tamanho n é dividido em 4 problemas de tamanho $n/2$ mais um particionamento que tem complexidade $O(n)$. Assim, a recorrência fica:

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

Podemos aplicar o método da árvore de recursão para gerar a seguinte estrutura.



Note que temos o seguinte padrão:

- * Nível 0: cn
- * Nível 1: $4 \cdot cn/2 = 2 \cdot cn$
- * Nível 2: $16cn/4 = 4 \cdot cn$
- * Nível 3: $64cn/8 = 8 \cdot cn$
- ...
- * Nível k : $4^k \frac{cn}{2^k} = 2^k \cdot cn$

O último nível da árvore ocorre quando $\frac{n}{2^k} = 1$, ou seja, a altura da árvore é $k = \log_2 n$.

Portanto, temos:

$$T(n) = \sum_{i=1}^{\log_2 n} 2^i \cdot cn = cn \sum_{i=1}^{\log_2 n} 2^i$$

Lembre-se que $2^{i+1} = 2 \cdot 2^i = 2^i + 2^i$, o que nos leva a $2^i = 2^{i+1} - 2^i$. Dessa forma, temos:

$$T(n) = cn \sum_{i=1}^{\log_2 n} 2^{i+1} - 2^i = cn(2^{\log_2 n + 1} - 2^0) = cn(2 \cdot 2^{\log_2 n} - 1) = cn(2n - 1) = 2cn^2 - cn$$

o que é $O(n^2)$. Conclusão: esse algoritmo não é mais eficiente que o usual. O que fazer então? Podemos melhorar diminuindo o número de subproblemas!

O algoritmo de Karatsuba

Note que no método anterior, o produto é dada por:

$$P = A \times 10^n + (B + C) \times 10^{n/2} + D$$

É possível reescrever a expressão $(B + C)$ como:

$$B + C = a_2 b_1 + a_1 b_2 = a_1 b_1 + a_2 b_1 + a_1 b_2 + a_2 b_2 - a_1 b_1 - a_2 b_2 = (a_1 + a_2)(b_1 + b_2) - a_1 b_1 - a_2 b_2$$

ou seja, utilizamos apenas 3 produtos e não mais 4 como antes.

A seguir apresentamos o algoritmo de Karatsuba para multiplicar inteiros.

```

Multiply_K(a, b) {
    if len(a) <= 1
        return a*b
    Particione a e b em
         $a = a_1 10^{n/2} + a_2$       (essa partição requer um único loop) -  $O(n)$ 
         $b = b_1 10^{n/2} + b_2$ 
    A = Multiply_K( $a_1$ ,  $b_1$ )
    B = Multiply_K( $a_2$ ,  $b_2$ )
    C = Multiply_K( $a_1 + a_2$ ,  $b_1 + b_2$ )

     $P = A \times 10^n + (C - A - B) \times 10^{n/2} + D$ 

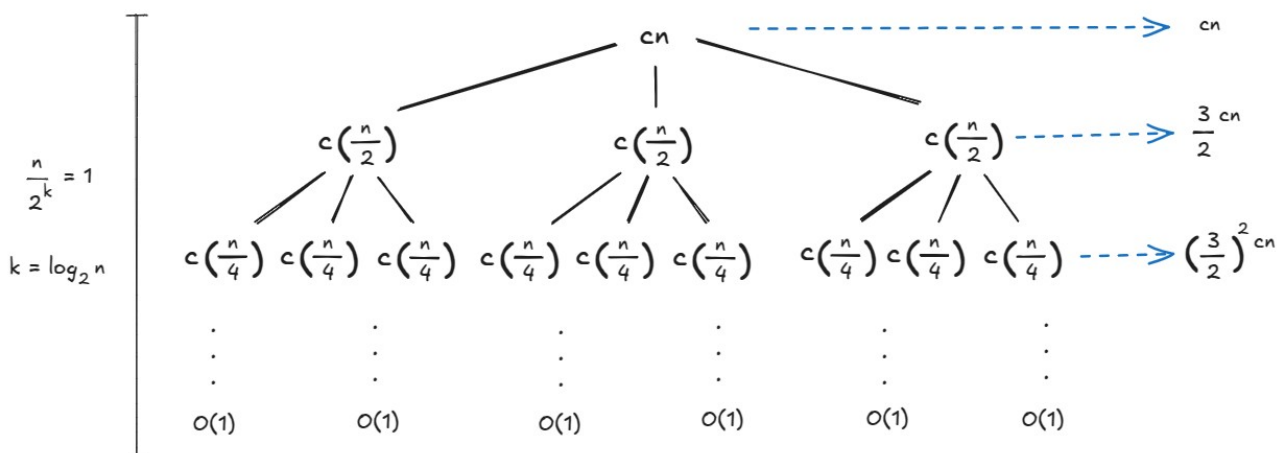
    return P
}

```

Sendo assim, qual seria a complexidade computacional agora? Note que a recorrência pode ser expressa como:

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

A árvore de recorrência pode ser construída como:



Note que temos o seguinte padrão:

- * Nível 0: cn
- * Nível 1: $3/2 \text{ } cn$
- * Nível 2: $3 (3/4) \text{ } cn = 9/4 \text{ } cn$
- * Nível 3: $9 (3/8) \text{ } cn = 27/8 \text{ } cn$
- ...
- * Nível k: $3^k \frac{cn}{2^k} = \left(\frac{3}{2}\right)^k cn$

O último nível da árvore ocorre quando $\frac{n}{2^k} = 1$, ou seja, a altura da árvore é $k = \log_2 n$.

Portanto, temos:

$$T(n) = \sum_{i=1}^{\log_2 n} \left(\frac{3}{2}\right)^i cn = cn \sum_{i=1}^{\log_2 n} \left(\frac{3}{2}\right)^i$$

Note que o somatório define a soma de uma P.G. em que o primeiro elemento é 1 e a razão $q = 3/2$.

$$\left(\frac{3}{2}\right)^0 + \left(\frac{3}{2}\right)^1 + \left(\frac{3}{2}\right)^2 + \left(\frac{3}{2}\right)^3 + \dots + \left(\frac{3}{2}\right)^{\log_2 n}$$

O número de termos n é igual a $\log_2 n + 1$. Lembrando que a soma da P.G. é dada por:

$$S_n = a_1 \frac{(q^n - 1)}{(q - 1)}$$

Sendo assim, nosso somatório é igual a:

$$S_{\log_2 n + 1} = 1 \frac{\left(\left(\frac{3}{2}\right)^{\log_2 n + 1} - 1\right)}{\left(\left(\frac{3}{2}\right) - 1\right)}$$

Somando 1 no numerador e 1 no denominador, temos:

$$S_{\log_2 n + 1} = \frac{\left(\frac{3}{2}\right)^{\log_2 n + 1}}{\left(\frac{3}{2}\right)} = \left(\frac{3}{2}\right)^{\log_2 n} = n^{\log_2 3/2}$$

Assim, a função $T(n)$ fica:

$$T(n) = cn n^{\log_2 3/2} = cn n^{\log_2 3 - \log_2 2} = cn n^{\log_2 3 - 1} = cn n^{\log_2 3} n^{-1} = c n^{\log_2 3}$$

Portanto, a complexidade do algoritmo de Karatsuba é $O(n^{\log_2 3})$, que é igual a $O(n^{1.585})$. Note com a estratégia dividir para conquistar conseguimos reduzir a complexidade da multiplicação de inteiros.

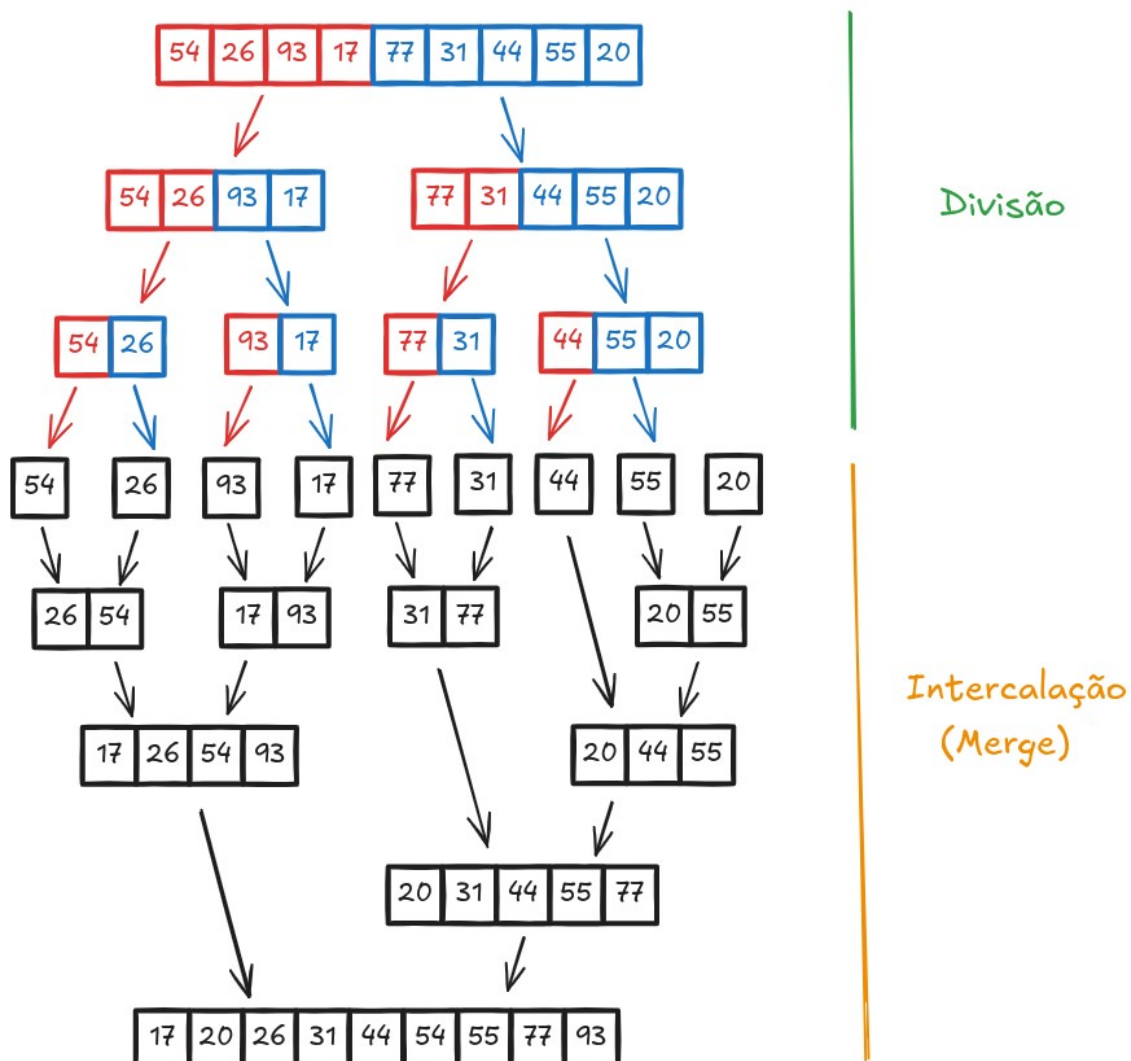
O algoritmo Mergesort

O algoritmo Mergesort utiliza a abordagem Dividir para Conquistar. A ideia básica consiste em dividir o problema em vários subproblemas e resolver esses subproblemas através da recursividade e depois conquistar, o que é feito após todos os subproblemas terem sido resolvidos através da união das resoluções dos subproblemas menores. Trata-se de um algoritmo recursivo que divide uma lista continuamente pela metade. Se a lista estiver vazia ou tiver um único elemento, ela está ordenada por definição (o caso base). Se a lista tiver mais de um elemento, dividimos a lista e invocamos recursivamente um Mergesort em ambas as metades. Assim que as metades estiverem ordenadas, a operação fundamental, chamada de **intercalação**, é realizada. Intercalar é o processo de pegar duas listas menores ordenadas e combiná-las de modo a formar uma lista nova, única e ordenada. A figura a seguir ilustra as duas fases principais do algoritmo Mergesort: a divisão e a intercalação.

Os três passos úteis dos algoritmos de dividir para conquistar que se aplicam ao MergeSort são:

1. **Dividir:** calcula o ponto médio do sub-arranjo, o que demora um tempo constante $O(1)$;
2. **Conquistar:** recursivamente resolve dois subproblemas, cada um de tamanho $n/2$, o que contribui com $T(n/2) + T(n/2)$ para o tempo de execução;
3. **Combinar:** unir os sub-arranjos em um único conjunto ordenado, que leva o tempo $O(n)$;

A seguir apresentamos o algoritmo Merge, que utiliza é responsável por realizar a intercalação de duas sublistas com custo computacional $O(n)$.



```

merge(A, low, q, high) {
    nL = q - low + 1      # q é o ponto médio
    nR = high - q         # número de elementos da sublista esq
                           # número de elementos da sublista dir
    Let L[0..nL - 1] be an array
    Let R[0..nR - 1] be an array
    for i = 0 to nL - 1
        L[i] = A[low + i]
    for j = 0 to nR - 1
        R[j] = A[(q+1)+j]
    i = 0
    j = 0
    k = 0
    while i < nL and j < nR {
        if L[i] ≤ R[j] {
            A[k] = L[i]
            i = i + 1
        }
        else {
            A[k] = R[j]
            j = j + 1
        }
        k = k + 1
    }
    while i < nL {        # se sobrar elementos em L
        A[k] = L[i]
        i = i + 1
        k = k + 1
    }
    while j < nR {        # se sobrar elementos em R
        A[k] = R[j]
        j = j + 1
        k = k + 1
    }
}

```

A seguir é apresentada a função mergesort, que realiza a ordenação de uma lista.

```

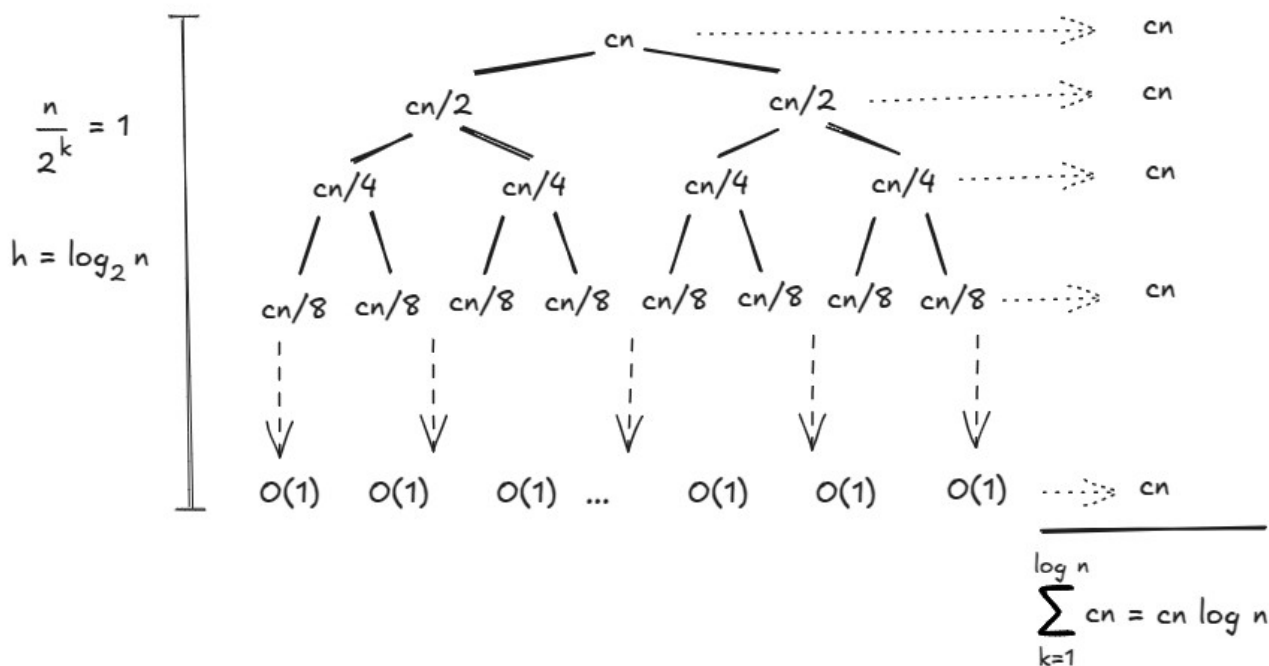
mergesort(A, low, high) {
    if low ≥ high          # lista tem zero ou um elemento
        return
    q = ⌊(low+high)/2⌋      # complexidade O(1)
    mergesort(A, low, q)
    mergesort(A, q+1, high)
    merge(A, low, q, high)
}

```

Assim, podemos escrever a relação de recorrência como:

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + O(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

A seguir empregamos o método da árvore de recursão para analisar a complexidade.



Podemos ver que em nós folhas devemos ter custo constante, ou seja, $\frac{n}{2^k} = 1$, o que implica em $k = \log_2 n$ como a altura da árvore de recursão. Note que ao somar os custos em cada nível desta árvore temos um valor constante igual a cn . Sendo assim, A função $T(n)$ fica:

$$T(n) = cn + cn + cn + \dots + cn = \sum_{i=0}^{\log_2 n} cn = cn \sum_{i=0}^{\log_2 n} 1 = cn \log_2 n$$

o que mostra que o algoritmo Mergesort possui complexidade $O(n \log_2 n)$.

O Problema da Multiplicação de Matrizes

Sabemos que multiplicar matrizes é uma operação recorrente em diversas aplicações da computação, matemática, estatística e áreas afins. Nesta seção, apresentaremos um algoritmo mais eficiente que a abordagem tradicional utilizando a estratégia dividir para conquistar. Primeiramente, lembre que o produto matricial é definido como:

$$\begin{matrix} \vec{a}_1 \\ \vec{a}_2 \end{matrix} \begin{bmatrix} 1 & 7 \\ 2 & 4 \end{bmatrix} \cdot \begin{matrix} \vec{b}_1 & \vec{b}_2 \\ \begin{bmatrix} 3 & 3 \\ 5 & 2 \end{bmatrix} \end{matrix} = \begin{bmatrix} \vec{a}_1 \vec{b}_1 & \vec{a}_1 \vec{b}_2 \\ \vec{a}_2 \vec{b}_1 & \vec{a}_2 \vec{b}_2 \end{bmatrix}$$

$A \quad B \quad C$

ou seja, os elementos da matriz C são os produtos escalares entre as linhas de A e as colunas de B .

Em outras palavras, temos:

$$c_{ij} = \vec{a}_i \cdot \vec{b}_j = \sum_{k=1}^n a_{ik} b_{kj}$$

O algoritmo tradicional para multiplicar duas matrizes quadradas A e B é ilustrado a seguir.

```
matrix_multiply(A, B, C, n) {
    for i = 1 to n {
        for j = 1 to n {
            for k = 1 to n
                c[i, j] = c[i, j] + a[i, k]*b[k, j]
        }
    }
}
```

Pode-se ver claramente, que como temos 3 estruturas de repetição FOR aninhadas, a complexidade dessa função é $O(n^3)$.

Uma simples estratégia dividir para conquistar consiste em assumir sem perda de generalidade que $n=2^m$ (potência de 2) e então subdividir as matrizes A e B em 4 submatrizes de tamanho $(n/2) \times (n/2)$. Dessa forma, as matrizes de entrada A e B podem ser decompostas como:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

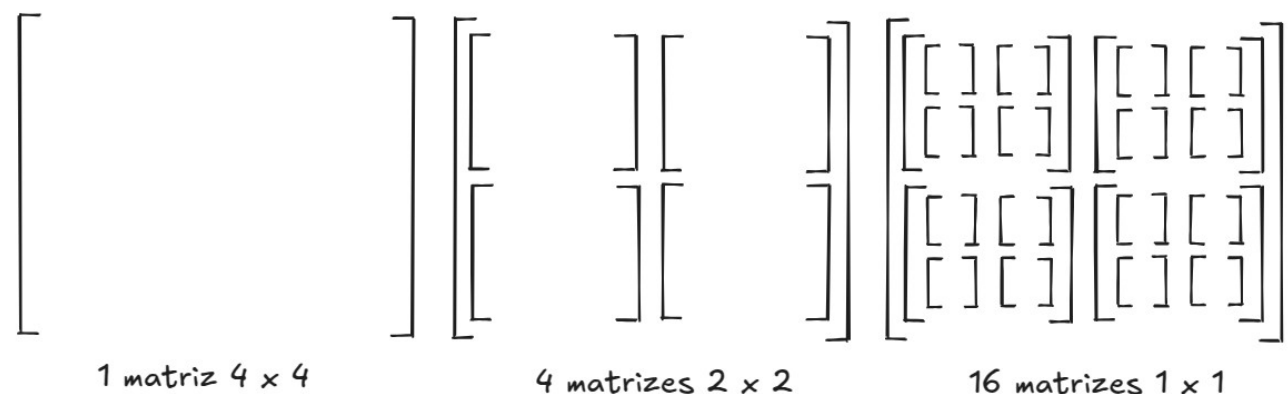
Dessa forma, o produto matricial fica dado por:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

Note que essa operação envolve:

8 multiplicações de matrizes $n/2 \times n/2$ + 4 adições de matrizes $n/2 \times n/2$

A ideia consiste em recursivamente subdividir a matriz até atingir o caso base em que temos matrizes de tamanho 1×1 .



O algoritmo a seguir implementa essa estratégia dividir para conquistar.

```

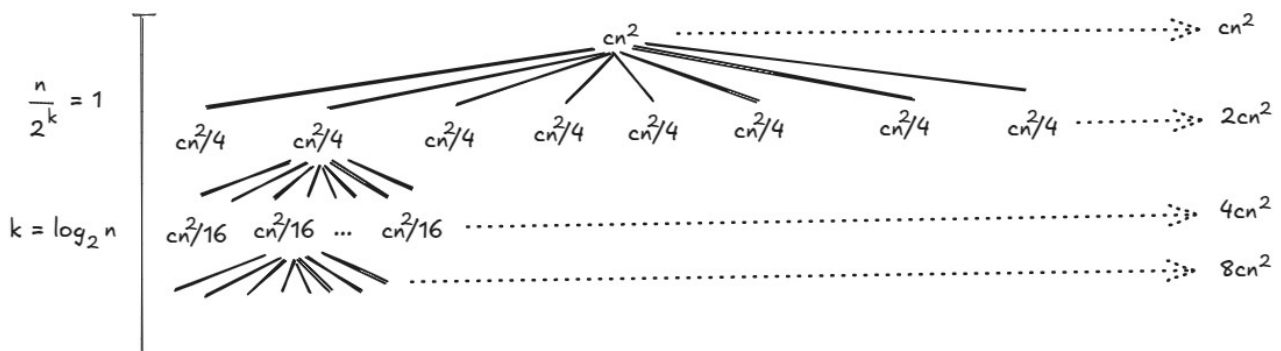
MMDC(A, B, n) {
    if n == 1
        return A*B
    A11, A12, A21, A22 = split(A)      # divide em 4 matrizes n/2 x n/2
    B11, B12, B21, B22 = split(B)      # divide em 4 matrizes n/2 x n/2
    P = MMDC(A11, B11)
    Q = MMDC(A12, B21)
    R = MMDC(A11, B12)
    S = MMDC(A12, B22)
    X = MMDC(A21, B11)
    Y = MMDC(A22, B21)
    W = MMDC(A21, B12)
    Z = MMDC(A22, B22)
    C11 = P + Q
    C12 = R + S
    C21 = X + Y
    C22 = W + Z
    C = merge(C11, C12, C21, C22)      # complexidade O(n2)
    return C
}

```

Note que o particionamento das matrizes A, B e C podem ser realizados com complexidade $O(n^2)$, o que nos permite escrever a seguinte recorrência:

$$T(n) = 8T\left(\frac{n}{2}\right) + O(n^2)$$

Utilizando o método da árvore de recursão temos a seguinte construção.



Podemos ver que em nós folhas devemos ter custo constante, ou seja, $\frac{n}{2^k} = 1$, o que implica em $k = \log_2 n$ como a altura da árvore de recursão. O custo de cada nível da árvore é dado por:

- * Nível 0: cn^2
- * Nível 1: $8 \frac{cn^2}{4} = 2cn^2$
- * Nível 2: $64 \frac{cn^2}{16} = 4cn^2$
- * Nível 3: $8cn^2$
- ...
- * Nível k: $2^k cn^2$

Dessa forma, temos:

$$T(n) = cn^2 + 2cn^2 + 4cn^2 + 8cn^2 + \dots = \sum_{i=0}^{\log_2 n} 2^i cn^2 = cn^2 \sum_{i=0}^{\log_2 n} 2^i$$

Lembre-se que $2^{i+1} = 2 \cdot 2^i = 2^i + 2^i$, o que nos leva a $2^i = 2^{i+1} - 2^i$. Dessa forma, temos:

$$T(n) = cn^2 \sum_{i=1}^{\log_2 n} 2^{i+1} - 2^i = cn^2 (2^{\log_2 n+1} - 2^0) = cn^2 (2 \cdot 2^{\log_2 n} - 1) = cn^2 (2n - 1) = 2cn^3 - cn^2$$

Em outras palavras, o método com a estratégia dividir para conquistar tem a mesma complexidade do que o método padrão. A seguir veremos como podemos melhorar essa estratégia através do algoritmo de Strassen.

O algoritmo de Strassen

A partir do particionamento das matrizes de entrada A e B, devemos aqui realizar o cálculo de 10 matrizes ($S_1, S_2, \dots, S_{10}$). Porém, para o cálculo dessas matrizes, utilizamos apenas soma e subtração de matrizes, conforme segue abaixo:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

As 10 matrizes são definidas como:

$$\begin{aligned} S_1 &= B_{12} - B_{22} & S_6 &= B_{11} + B_{22} \\ S_2 &= A_{11} - A_{12} & S_7 &= A_{12} - A_{22} \\ S_3 &= A_{21} + A_{22} & S_8 &= B_{21} + B_{22} \\ S_4 &= B_{21} - B_{11} & S_9 &= A_{11} - A_{21} \\ S_5 &= A_{11} + A_{22} & S_{10} &= B_{11} + B_{12} \end{aligned}$$

Note que o custo computacional de calcular todas essas 10 matrizes é $O(n^2)$. Em seguida, devemos calcular 7 matrizes (envolve produto matricial):

$$\begin{aligned} P_1 &= A_{11} S_1 = A_{11} B_{12} - A_{11} B_{22} \\ P_2 &= S_2 B_{22} = A_{11} B_{22} - A_{12} B_{22} \\ P_3 &= S_3 B_{11} = A_{21} B_{11} + A_{22} B_{11} \\ P_4 &= A_{22} S_4 = A_{22} B_{21} - A_{22} B_{11} \\ P_5 &= S_5 S_6 = A_{11} B_{11} + A_{11} B_{22} + A_{22} B_{11} + A_{22} B_{22} \\ P_6 &= S_7 S_8 = A_{12} B_{21} + A_{12} B_{22} - A_{22} B_{21} - A_{22} B_{22} \\ P_7 &= S_9 S_{10} = A_{11} B_{11} + A_{11} B_{12} - A_{21} B_{11} - A_{21} B_{12} \end{aligned}$$

Pode-se mostrar que as 4 submatrizes C_{11} , C_{12} , C_{21} e C_{22} podem ser expressas como:

$$\begin{aligned} C_{11} &= P_5 + P_4 - P_2 + P_6 = A_{11} B_{11} + A_{12} B_{21} \\ C_{12} &= P_1 + P_2 = A_{11} B_{12} + A_{12} B_{22} \\ C_{21} &= P_3 + P_4 = A_{21} B_{11} + A_{22} B_{21} \\ C_{22} &= P_5 + P_1 - P_3 - P_7 = A_{22} B_{22} + A_{21} B_{12} \end{aligned}$$

O algoritmo de Strassen é apresentado a seguir.

```

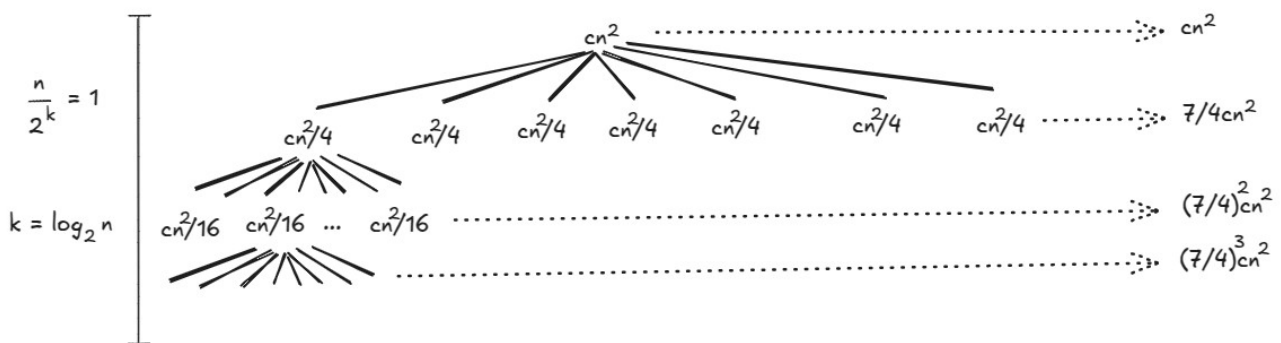
Strassen(A, B, n) {
    if n == 1
        return A*B
    A11, A12, A21, A22 = split(A)      # divide em 4 matrizes n/2 x n/2
    B11, B12, B21, B22 = split(B)      # divide em 4 matrizes n/2 x n/2
    P1 = Strassen(A11, B12 - B22)
    P2 = Strassen(A11 + A12, B22)
    P3 = Strassen(A21 + A22, B11)
    P4 = Strassen(A22, B21 - B11)
    P5 = Strassen(A11 + A22, B11 + B22)
    P6 = Strassen(A12 - A22, B21 + B22)
    P7 = Strassen(A11 - A21, B11 + B12)
    C11 = P5 + P4 - P2 + P6
    C12 = P1 + P2
    C21 = P3 + P4
    C22 = P5 + P1 - P6 - P7
    C = merge(C11, C12, C21, C22)      # complexidade O(n2)
    return C
}

```

Assim, a recorrência para o algoritmo de Strassen é dada por:

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$$

o que gera a seguinte árvore de recursão:



Podemos ver que em nós folhas devemos ter custo constante, ou seja, $\frac{n}{2^k} = 1$, o que implica em $k = \log_2 n$ como a altura da árvore de recursão. O custo de cada nível da árvore é dado por:

- * Nível 0: cn^2
- * Nível 1: $7cn^2/4$
- * Nível 2: $49cn^2/16 = (7/4)^2cn^2$
- * Nível 3: $(7/4)^3cn^2$
- ...
- * Nível 4: $(7/4)^kcn^2$

Sendo assim, podemos escrever $T(n)$ como:

$$T(n) = cn^2 + \left(\frac{7}{4}\right)cn^2 + \left(\frac{7}{4}\right)^2 cn^2 + \left(\frac{7}{4}\right)^3 cn^2 + \dots + \left(\frac{7}{4}\right)^{\log_2 n} cn^2$$

o que nos leva a:

$$T(n) = cn^2 \sum_{i=0}^{\log_2 n} \left(\frac{7}{4}\right)^i$$

Temos uma P.G. com $1 + \log_2 n$ termos em que o primeiro termo $a_1 = 1$ e $q = \frac{7}{4}$. Sendo assim, o somatório vale:

$$S = 1 \frac{\left(\left(\frac{7}{4}\right)^{1+\log_2 n} - 1\right)}{\left(\frac{7}{4} - 1\right)}$$

Somando 1 no numerador e no denominador, chega-se a:

$$S = \frac{\left(\frac{7}{4}\right)^{1+\log_2 n} - \frac{7}{4}}{\frac{7}{4} - 1} = \left(\frac{7}{4}\right)^{\log_2 n}$$

Pela propriedade $a^{\log_b c} = c^{\log_b a}$, podemos escrever:

$$S = n^{\log_2(7/4)} = n^{\log_2 7 - 2} = n^{\log_2 7} n^{-2}$$

Portanto, voltando a $T(n)$, chegamos em:

$$T(n) = cn^2 n^{\log_2 7} n^{-2} = c n^{\log_2 7}$$

o que mostra que o algoritmo de Strassen tem complexidade $O(n^{\log_2 7})$, o que equivale a aproximadamente $O(n^{2.8})$. Portanto, o algoritmo de Strassen é capaz de melhorar o desempenho assintótico da multiplicação de matrizes.

O problema do par de pontos mais próximos

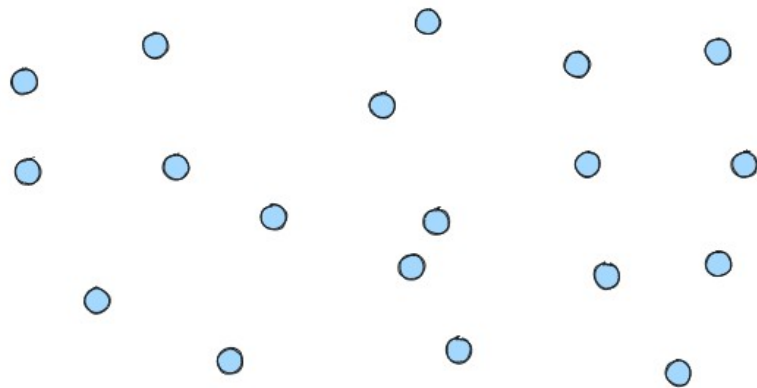
Suponha que sejam dados n pontos no plano, o objetivo consiste em encontrar o par composto pelos pontos mais próximos entre si. Esse problema é muito importante pois define uma primitiva muito comum em diversos problemas em geometria computacional e computação gráfica. Iniciaremos apresentando as variáveis e a notação envolvidas no problema.

Seja $P = \{p_1, p_2, \dots, p_n\}$ um conjunto de n pontos no plano em que cada p_i é definido em termos de suas coordenadas (x_i, y_i) . Podemos associar para cada par de pontos $p_i, p_j \in P$, a distância Euclidiana entre eles, denotada aqui por $d(p_i, p_j)$ e definida como:

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

O objetivo do problema consiste em encontrar os pontos p_i, p_j que minimizam a distância entre eles, ou seja:

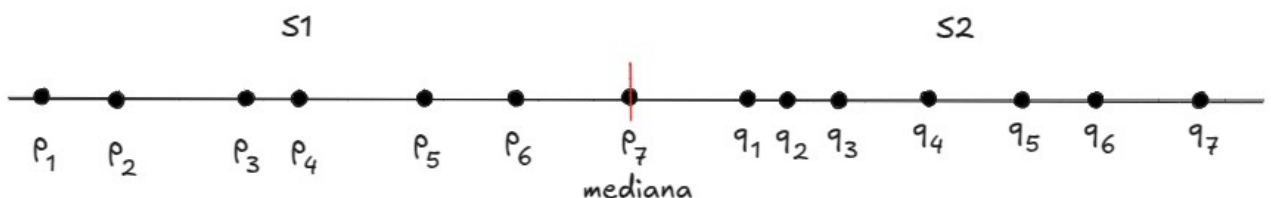
$$p_1^*, p_2^* = \operatorname{argmin} \{ d(p_i, p_j) \}$$



Por motivos didáticos, vamos iniciar com o caso 1D. Neste caso, os pontos estão localizados na reta real e possuem apenas uma coordenada x_i . A solução trivial para o problema envolve ordenar os pontos pelas coordenadas x_i e percorrer a lista calculando a distância de cada ponto para o seu sucessor na lista.

```
ClosestPair_1D(L) {
    # L é a lista das coordenadas dos pontos
    n = len(L)
    # P armazena índices dos pontos após ordenação
    P = argsort(L)
    # Lista de coordenadas ordenadas
    sorted(L)           # complexidade  $O(n \log n)$  ou mais
    min_d = ∞
    pto1 = -1
    pto2 = -1
    # Encontrar a menor diferença entre duas coordenadas vizinhas
    for i = 1 to n - 1 {
        d = abs(L[i] - L[i+1])  # diferença entre as coordenadas
        if d < min_d {
            min_d = d
            pto1 = P[i]
            pto2 = P[i+1]
        }
    }
    return (pto1, pto2)
}
```

Note que a complexidade desse algoritmo é $O(n \log n)$ que é o limite mínimo para a ordenação. É possível melhorar esse algoritmo? A resposta é não. Mesmo utilizando a estratégia dividir para conquistar para quebrar o problema em 2 subproblemas, a complexidade continua sendo $O(n \log n)$. Veremos a seguir o porque.



A estratégia **Dividir para Conquistar** pode ser aplicada ao problema do **par de pontos mais próximos** no caso **1D** (quando os pontos estão dispostos em uma reta) da seguinte maneira:

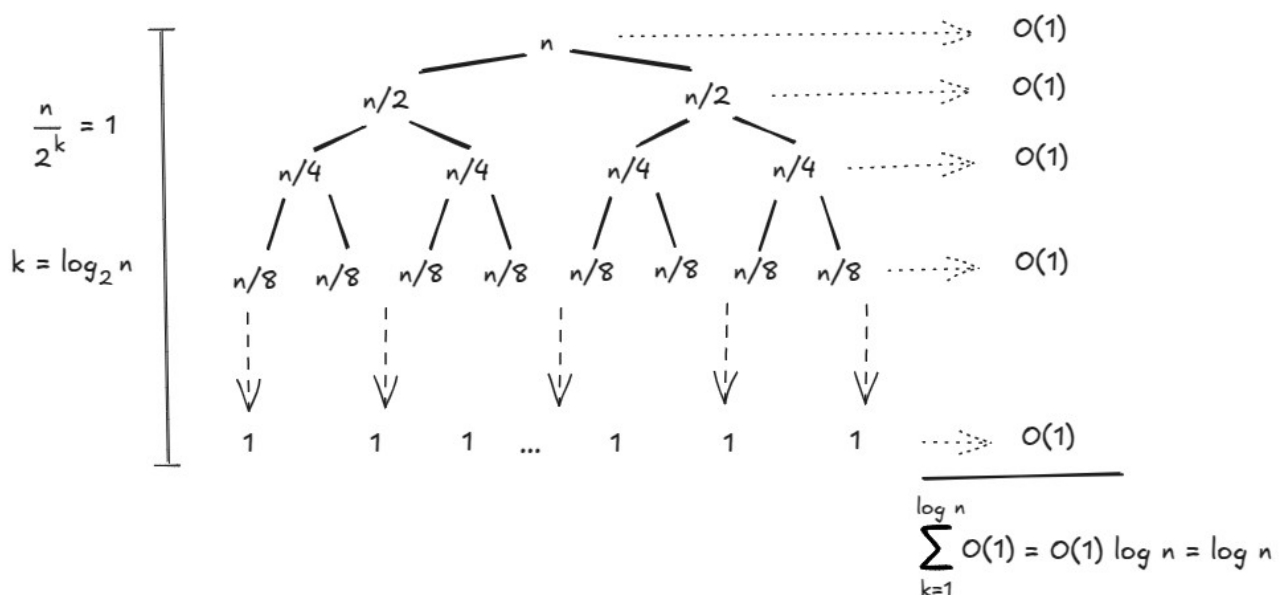
1. **Ordenação:** ordene os pontos em ordem crescente (se ainda não estiverem ordenados).
2. **Dividir:** Divida o conjunto de pontos em duas metades aproximadamente iguais (mediana).
3. **Conquistar:** resolva recursivamente o problema em cada metade. Cada subproblema retorna a menor distância entre dois pontos dentro da respectiva metade.
4. **Combinar:** após obter as menores distâncias d_L e d_R das metades esquerda e direita, a menor distância global d será:

$$d = \min \{ d_L, d_R \}$$

Como os pontos estão ordenados, a única possibilidade de encontrar um par mais próximo atravessando a fronteira entre as metades ocorre entre o maior ponto da metade esquerda e o menor ponto da metade direita. Assim, basta calcular a distância entre esses dois pontos e atualizar d se essa distância for menor. Note que a recorrência para essa estratégia recursiva é:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1)$$

A árvore de recursão para a recorrência acima é ilustrada na figura a seguir.



Apesar de concluirmos que a complexidade é $O(\log n)$, a ordenação dos pontos no início do processo é o gargalo, pois possui complexidade limitada a $O(n \log n)$. Em outras palavras, no caso 1D, a solução trivial é tão eficiente quanto a solução obtida pelo método dividir para conquistar.

No caso 2D, a solução trivial é a força bruta, ou seja, consiste em calcular as distâncias entre todos os possíveis pares de pontos e verificar qual delas é a menor.

```

ClosestPair_2D(L) {
    # L é a lista das coordenadas dos pontos
    n = len(L)
    min_d = ∞
    pto1 = -1
    pto2 = -1
    # Loop principal (obtem cada par de pontos)
    for i = 1 to n {
        for j = i to n {
            d = dist(L[i], L[j])    # calcula a distância
            if d < min_d {
                min_d = d
                pto1 = i
                pto2 = j
            }
        }
    }
    return (pto1, pto2)
}

```

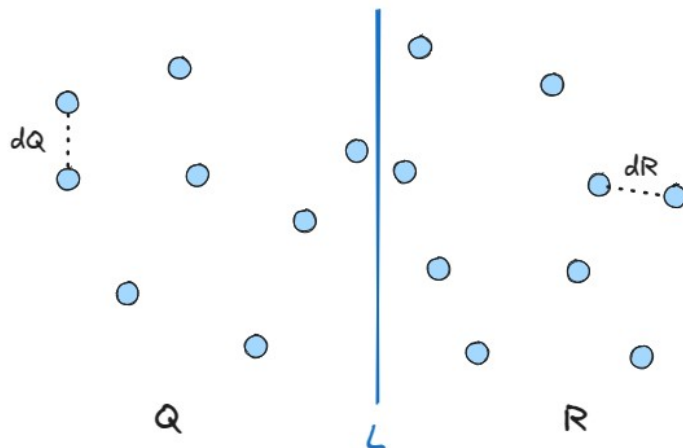
Note que devermos calcular o seguinte número de distâncias:

$$T(n) = 1 + 2 + 3 + \dots + n = \sum_{k=1}^n k = \frac{n(n+1)}{2}$$

Portanto, o algoritmo tem complexidade $O(n^2)$. A pergunta natural é: é possível melhorar esse algoritmo? A resposta é sim! No caso 2D podemos desenvolver um algoritmo log-linear. A seguir veremos como isso é possível. A ideia consiste em adotar a estratégia dividir para conquistar da seguinte maneira:

1. Encontrar o par mais próximo na metade a esquerda do plano
2. Encontrar o par mais próximo na metade a direita do plano
3. Combinar as soluções verificando se existe algum par formado por um ponto de Q e outro de R

A divisão, ou seja, a decomposição do problema em dois subproblemas menores pode ser realizada criando uma lista P_x dos pontos ordenados por ordem crescente de coordenada x. A mediana M desse conjunto define uma reta que particiona o plano em duas regiões: Q (left) e R (right). Recursivamente, podemos encontrar os pares mais próximos em cada região. A figura a seguir ilustra esse processo.

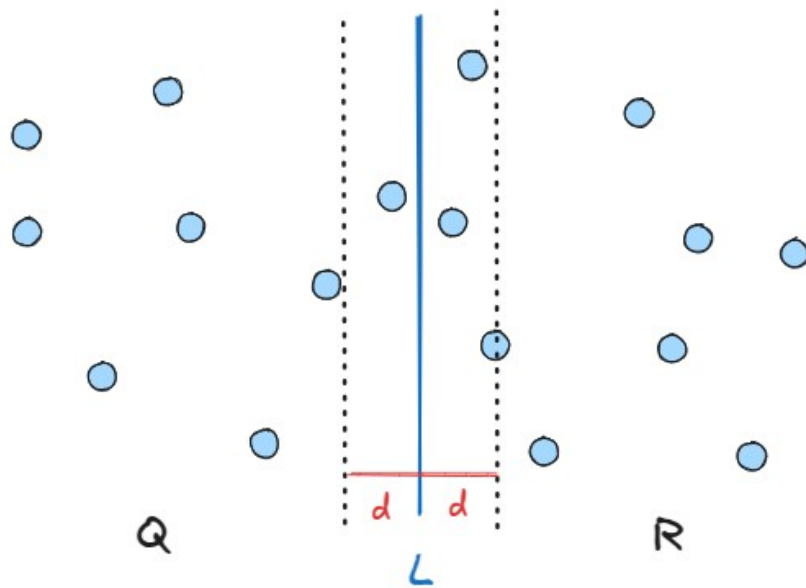


O passo da conquista consiste em combinar as soluções dos subproblemas menores. Seja:

$$d = \min\{d_Q, d_R\}$$

ou seja d é a menor das menores distâncias em Q e R. A princípio, d seria a solução do problema maior, mas note que pode existir um par em que um dos pontos está em Q e outro está em R, cuja distância entre eles é menor que d . Em outras palavras, o resultado a seguir mostra algo muito interessante.

Teorema: Se existe um par $\{q, r\}$ com $q \in Q$ e $r \in R$ (ou vice-versa) tal que $d(q, r) < d$ então ambos q e r devem estar no máximo a uma distância d de L (numa faixa ao redor de L).



Prova:

Podemos provar a afirmação acima notando que a reta L é definida pelo ponto x^* com coordenada x igual a mediana M das coordenadas x. Sabemos que as coordenadas dos pontos são: $q = (q_x, q_y)$ e $r = (r_x, r_y)$. Pela definição de x^* , temos:

$$q_x \leq x^* \leq r_x \quad (I)$$

Isso implica que $x^* - q_x \leq r_x - q_x$. Subtraindo q_x de ambos os lados, temos:

$$x^* - q_x \leq r_x - q_x \leq d(q, r) < d$$

De modo similar, de (I) temos que $x^* \geq q_x$. Isso implica que $r_x - x^* \leq r_x - q_x$. Somando r_x de ambos os lados, temos:

$$r_x - x^* \leq r_x - q_x \leq d(q, r) < d$$

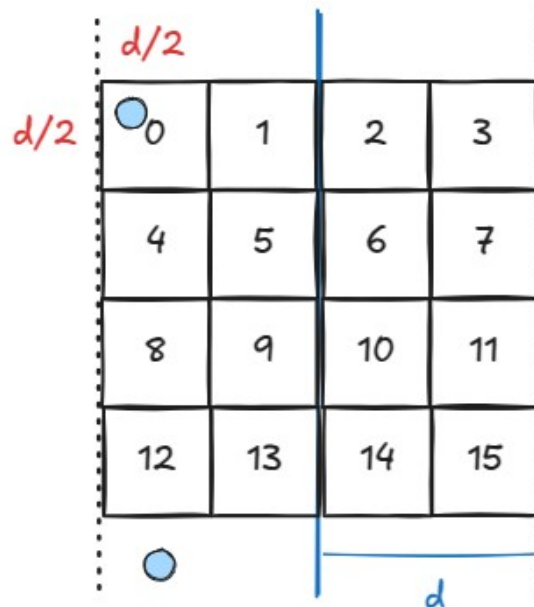
Portanto, os pontos de interesse estão dentro da margem definida por d .

A seguir veremos um outro resultado fundamental para o problema em questão.

Teorema: Seja S_y uma lista com as coordenadas y de todos os pontos a uma distância d de L (na faixa) em ordem crescente. Suponha que $S_y = \{p_1, p_2, \dots, p_m\}$ com $m \leq n$. Se $d(p_i, p_j) < d$, então $j - i \leq 15$. Em outras palavras, se dois pontos estão próximos na faixa, então eles também estão próximos na lista S_y .

Prova:

1. Divida a região dentro da faixa em quadrados de lado $d/2$



Então, quantos pontos podem existir em cada quadrado? Apenas 1, pois senão existiriam 2 pontos cuja distância entre eles seria menor que d .

2. Suponha agora que 2 pontos são separados por 15 índices. Então, existem 3 linhas separando os pontos. Como cada linha tem lado $d/2$, a distância entre os pontos é maior ou igual a $3d/2$. Ou seja, a distância entre os pontos é maior que d .

3. Portanto, basta procurar no máximo 15 posições a frente em S_y .

Obs: Note que o valor 15 pode até ser diminuído, mas o que importa aqui é que ele é uma constante!

O algoritmo a seguir mostra a abordagem dividir para conquistar para o problema do par mais próximo no $\mathbb{R}^2$.

```
ClosestPair_DC_2D(Px, Py) {
    # Px são as coordenadas x dos pontos
    # Py são as coordenadas y dos pontos
    # caso base (apenas 2 pontos)
    if |Px| == 2
        return dist(Px[1], Px[2], Py[1], Py[2])
    # Divisão (divide plano em Q e R)
    m = meio(Px)           # pode ser feito em O(1)
    d1 = ClosestPair_DC_2D(FirstHalf(Px, m), Py)
    d2 = ClosestPair_DC_2D(SecondHalf(Px, m), Py)
```

```

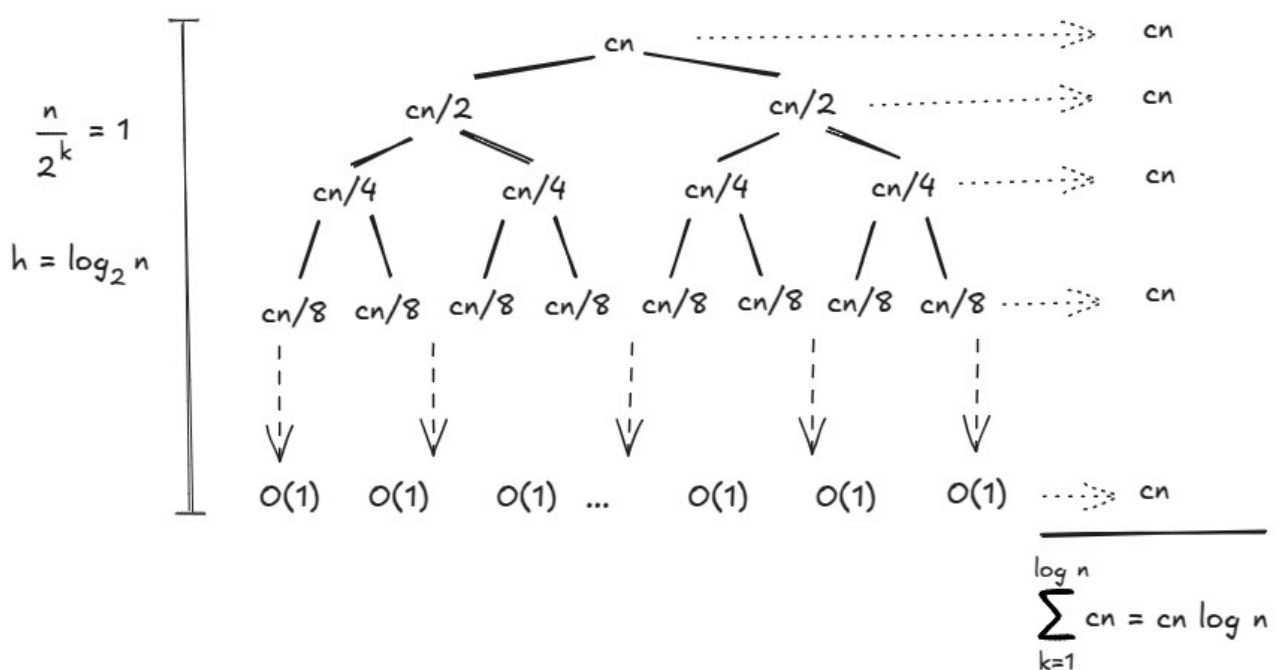
d = min(d1, d2)
# Conquista (merge)
Sx = coordenadas dos pontos em Px no intervalo [m - d, m + d]
Sy = coordenadas y dos pontos adicionados a Sx
for i = 1 to |Sy| {
    h = min(i+15, |Sy|)
    for j = i to h
        d = min(dist(Sx[i], Sx[j], Sy[i], Sy[j]), d)
    }
}
return d
}

```

Note que trata-se de um algoritmo recursivo que quebra o problema de tamanho n em 2 subproblemas menores de tamanho $n/2$ e depois combina as soluções (merge) com custo linear em n , ou seja, com complexidade $O(n)$, pois o loop mais interno é constante. Sendo assim, a recorrência é dada por:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Podemos expandir a árvore de recursão como ilustrado a seguir.



Sabemos que o nível máximo da árvore ocorre quando $\frac{n}{2^k} = 1$, ou seja o número de níveis da árvore é $k = \log_2 n$. Somando os custos em cada nível da árvore, temos o custo total do algoritmo, que é dado por:

$$T(n) = \sum_{i=1}^{\log_2 n} cn = cn \log_2 n$$

o que é igual a $O(n \log_2 n)$ e portanto melhor que o método trivial que é $O(n^2)$.

A Transformada de Fourier Discreta

O processamento digital de sinais lida com sinais no domínio do tempo. Sinais analógicos e digitais estão presentes no mundo ao nosso redor: desde a variação do preço do dólar nos últimos anos até a sinais de voz humana em gravações telefônicas, todos esses tipos de dados são sinais que variam no tempo. Durante a análise de tais sinais, é fundamental ter acesso as componentes de frequência dos mesmos, visto que ao modificar determinadas frequências dos sinais podemos por exemplo filtrar ruídos ou até mesmo reforçar agudos ou graves.

=> A Transformada de Fourier é a ferramenta matemática que nos permite analisar as componentes de frequência de sinais.

=> No caso de sinais discretos (processamento digital), utiliza-se a versão discreta, conhecida como DFT (Discrete Fourier Transform).

Ideia: mapear n pontos do domínio do tempo para n pontos no domínio da frequência.

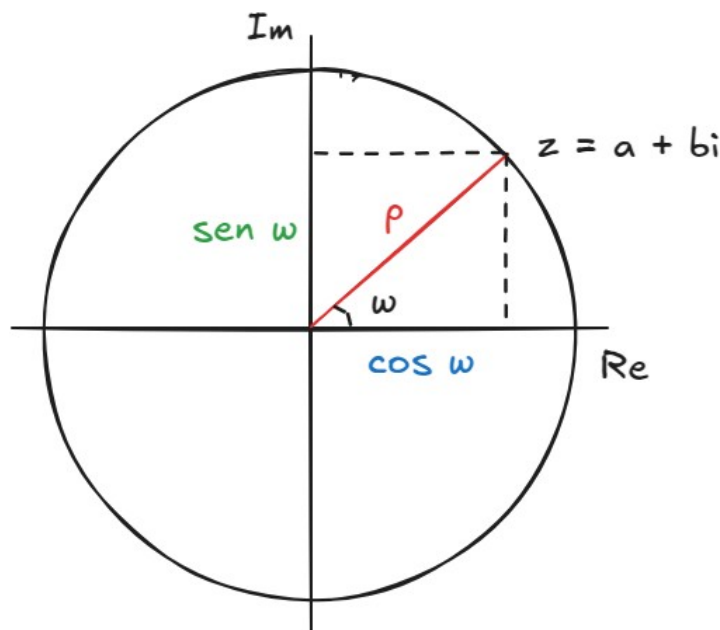
Seja $x[n]$ um sinal de entrada com n amostras. Então a DFT de $x[n]$, denotada por $X[k]$, é dada por:

$$X[k] = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi kn}{N}} \quad \text{para } k = 0, 1, 2, \dots, N-1$$

Analogamente, a transformada inversa, conhecida por IDFT (Inverse Discrete Fourier Transform), que reconstrói o sinal original a partir da sua representação no domínio da frequência, é dada por:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j \frac{2\pi kn}{N}} \quad \text{para } n = 0, 1, 2, \dots, N-1$$

A seguir, iremos ver que tanto a DFT quanto a IDFT são nada mais nada menos que produtos entre uma matriz quadrada de coeficientes e um vetor com as amostras do sinal. Uma pergunta relevante é: porque a DFT decompõe um sinal em componentes de frequência? Devemos nos lembrar da relação entre exponenciais complexas e as funções trigonométricas seno e cosseno. A figura a seguir ilustra a circunferência no plano complexo de Argand-Gauss.



Note que um número complexo, além de sua forma padrão, pode ser expresso pela notação polar, em termos de sua magnitude ρ e do ângulo ω (fase): ρ

$$z = \rho e^{i\omega}$$

onde $\rho = \sqrt{a^2 + b^2}$. Como estamos na circunferência trigonométrica, $a = \cos \omega$ e $b = \sin \omega$. Além disso, note que, neste caso, devido a identidade trigonométrica, temos que:

$$\rho = \sqrt{\sin^2(\omega) + \cos^2(\omega)} = 1$$

de modo que

$$e^{i\omega} = \cos(\omega) + i \sin(\omega)$$

Se conjugarmos o número complexo, ou seja, trocarmos o sinal da parte imaginária, temos:

$$e^{-i\omega} = \cos(\omega) - i \sin(\omega)$$

Na engenharia e na computação, por questões de nomenclatura, é comum usar a letra j ao invés de i para indicar a unidade complexa, ou seja, $j = \sqrt{-1}$. Portanto, note que:

$$e^{\frac{-j2\pi kn}{N}} = \cos\left(\frac{2\pi kn}{N}\right) - j \sin\left(\frac{2\pi kn}{N}\right)$$

Assim, o que a DFT faz é essencialmente escrever um sinal $x[n]$ do domínio do tempo como combinação linear de senos e cossenos (frequências do sinal). Porém, note que o resultado da DFT de um sinal é um vetor de mesmo tamanho composto por números complexos (e não reais)! Por essa razão, é comum analisarmos a magnitude dos coeficientes da DFT, uma vez que cada valor é composto de uma parte real e outra imaginária.

A Matriz DFT

A DFT é aplicada fazendo $\vec{X} = W \vec{x}$, onde W é a matriz dos coeficientes da DFT:

$$W_{k,n} = e^{\frac{-j2\pi kn}{N}} \quad \text{para } k = 0, 1, 2, \dots, N-1 \text{ e } n = 0, 1, 2, \dots, N-1$$

Note que se $k = 0$ ou $n = 0$, $W_{k,n} = e^0 = 1$. Na prática, isso significa que todos os elementos da primeira linha e da primeira coluna de W são iguais a 1. Em todas as demais posições da matriz, os elementos são potências de w , definido como:

$$w = e^{\frac{-j2\pi}{N}}$$

Por exemplo, o elemento da posição (2, 2) é dado por:

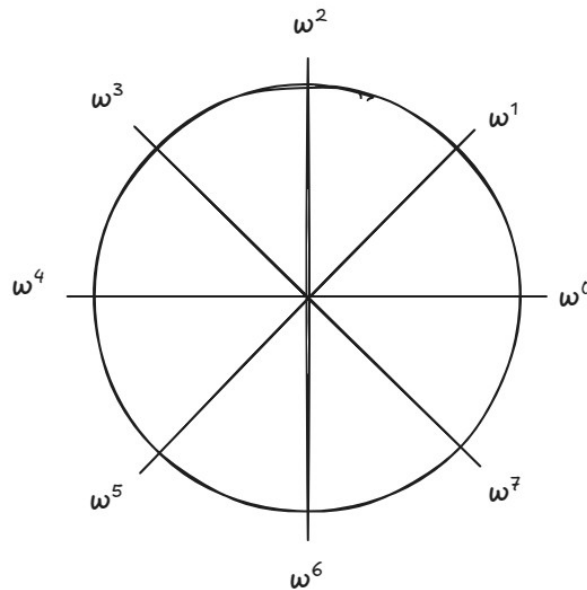
$$W_{2,2} = e^{\frac{-j2\pi 22}{N}} = e^{\frac{-j2\pi 4}{N}} = e^{\frac{-j2\pi}{N}} e^{\frac{-j2\pi}{N}} e^{\frac{-j2\pi}{N}} e^{\frac{-j2\pi}{N}} = w w w w = w^4$$

Dessa forma, pode-se mostrar que a matriz DFT W é uma matriz de Vandermonde, dada por:

$$W = \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & w & w^2 & w^3 & \dots & w^{N-1} \\ 1 & w^2 & w^4 & w^6 & \dots & w^{2(N-1)} \\ 1 & w^3 & w^6 & w^9 & \dots & w^{3(N-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & w^{N-1} & w^{2(N-1)} & w^{3(N-1)} & \dots & w^{(N-1)(N-1)} \end{bmatrix}$$

Exemplo: Calcule a DFT do sinal $x[] = [1, 2, 3, 4, 1, 2, 3, 4]$.

Primeiramente, como $N = 8$, devemos dividir a circunferência unitária no plano complexo em 8 intervalos iguais, conforme ilustra a figura a seguir.



onde $w = e^{-j2\frac{\pi}{8}} = \cos \frac{\pi}{4} - j \sin \frac{\pi}{4} = \frac{\sqrt{2}}{2} - j \frac{\sqrt{2}}{2}$

A matriz W para esse caso fica:

$$W = \frac{1}{\sqrt{8}} \begin{bmatrix} w^0 & w^0 & w^0 & w^0 & w^0 & w^0 & w^0 & w^0 \\ w^0 & w^1 & w^2 & w^3 & w^4 & w^5 & w^6 & w^7 \\ w^0 & w^2 & w^4 & w^6 & w^8 & w^{10} & w^{12} & w^{14} \\ w^0 & w^3 & w^6 & w^9 & w^{12} & w^{15} & w^{18} & w^{21} \\ w^0 & w^4 & w^8 & w^{12} & w^{16} & w^{20} & w^{24} & w^{28} \\ w^0 & w^5 & w^{10} & w^{15} & w^{20} & w^{25} & w^{30} & w^{35} \\ w^0 & w^6 & w^{12} & w^{18} & w^{24} & w^{30} & w^{36} & w^{42} \\ w^0 & w^7 & w^{14} & w^{21} & w^{28} & w^{35} & w^{42} & w^{49} \end{bmatrix}$$

O preenchimento da matriz W é feito linha a linha, efetuando deslocamentos variáveis na circunferência unitária. Para a primeira linha, o deslocamento é zero e temos todos os elementos iguais a w^0 . Na segunda linha o deslocamento é de um intervalo, na terceira linha o deslocamento é de 3 intervalos e assim sucessivamente. Em outras palavras, isso significa que $w^{N+n} = w^n$. Sendo assim, a matriz simplifica-se para:

$$W = \frac{1}{\sqrt{8}} \begin{bmatrix} w^0 & w^0 & w^0 & w^0 & w^0 & w^0 & w^0 & w^0 \\ w^0 & w^1 & w^2 & w^3 & w^4 & w^5 & w^6 & w^7 \\ w^0 & w^2 & w^4 & w^6 & w^0 & w^2 & w^4 & w^6 \\ w^0 & w^3 & w^6 & w^1 & w^4 & w^7 & w^2 & w^5 \\ w^0 & w^4 & w^0 & w^4 & w^0 & w^4 & w^0 & w^4 \\ w^0 & w^5 & w^2 & w^7 & w^4 & w^1 & w^6 & w^3 \\ w^0 & w^6 & w^4 & w^2 & w^0 & w^6 & w^4 & w^2 \\ w^0 & w^7 & w^6 & w^5 & w^4 & w^3 & w^2 & w^1 \end{bmatrix} = \frac{1}{\sqrt{8}} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & w & -j & -jw & -1 & -w & j & jw \\ 1 & -j & -1 & j & 1 & -j & -1 & j \\ 1 & -jw & j & w & -1 & jw & -j & -w \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -w & -j & jw & -1 & w & j & -jw \\ 1 & j & -1 & -j & 1 & j & -1 & -j \\ 1 & jw & j & -w & -1 & -jw & -j & w \end{bmatrix}$$

uma vez que calculando os valores de w^0 até w^7 temos:

$$\begin{aligned} w^0 &= 1 \\ w^1 &= w \\ w^2 &= \left(\frac{\sqrt{2}}{2} - j \frac{\sqrt{2}}{2} \right) \left(\frac{\sqrt{2}}{2} - j \frac{\sqrt{2}}{2} \right) = \frac{1}{2} - \frac{1}{2}j - \frac{1}{2}j + \frac{1}{2}j^2 = -j \\ w^3 &= -jw \\ w^4 &= (-j)(-j) = j^2 = -1 \\ w^5 &= -w \\ w^6 &= j \\ w^7 &= jw \end{aligned}$$

A função a seguir implementa a DFT de um sinal x de N amostras. Note que temos um produto entre a matriz W definida pelo algoritmo e o sinal de entrada x .

```
DFT(x, N) {
    W = DFT_matrix(N)
    for i = 0 to N - 1 {
        X[i] = 0
        for j = 0 to N - 1
            X[i] = X[i] + W[i, j]*x[j]
        }
    return X
}
```

A construção da matriz W tem complexidade $O(N^2)$ e os dois loops FOR aninhados também, logo é fácil perceber que a complexidade da DFT é quadrática. Para sinais com um grande número de amostras, o processo torna-se bastante lento. Para acelerar o método, foi proposto o algoritmo FFT (Fast Fourier Transform), utilizando uma estratégia dividir para conquistar. Note que o resultado da DFT é um vetor de números complexos. Para visualizar a DFT de um sinal, podemos plotar a magnitude dos elementos de X , ou seja:

$$||X[i]|| = \sqrt{\Re\{X[i]\}^2 + \Im\{X[i]\}^2} \quad \text{para } i = 1, 2, \dots, N$$

Também é possível visualizar a fase da DFT, que nada mais é que o ângulo que o número complexo forma com o eixo R no plano complexo:

$$\theta[i] = \arctan\left(\frac{\Re\{X[i]\}}{\Im\{X[i]\}}\right) \quad \text{para } i = 1, 2, \dots, N$$

O algoritmo Fast Fourier Transform (FFT)

=> Trata-se de um algoritmo eficiente para o cálculo da DFT

=> Utiliza a estratégia dividir para conquistar

A FFT explora as simetrias de $e^{\frac{-j2\pi kn}{N}}$. Seja $W_N = e^{\frac{-j2\pi}{N}}$, então temos $W_N^k = e^{\frac{-j2\pi k}{N}}$.

i) Simetria do conjugado complexo

$$W_N^{k(N-n)} = W_N^{kN} W_N^{-kn} = W_N^{-kn} = (W_N^{kn})^*$$

ii) Periodicidade (devido a circunferência trigonométrica)

$$W_N^{kn} = 1 W_N^{kn} = W_N^{kN} W_N^{kn} = W_N^{k(N+n)}$$

FFT por decimação no tempo

Objetivo: construir uma grande DFT a partir de várias DFT menores.

Sem perda de generalidade, iremos assumir que $N=2^m$, para $m > 1$ (se o número de amostras do sinal não é potência de 2, podemos completar com zeros até a próxima potência de 2).

Ideia: Separar as amostras de $x[n]$ em índices pares e índices.

$$X[k] = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x[n] e^{\frac{-j2\pi kn}{N}} = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x[n] W_N^{kn} = \frac{1}{\sqrt{N}} \sum_{n \text{ par}} x[n] W_N^{kn} + \frac{1}{\sqrt{N}} \sum_{n \text{ ímpar}} x[n] W_N^{kn}$$

Realizando uma simples substituição de variáveis:

Se n par: $n = 2r$

Se n ímpar: $n = 2r + 1$

para $r = 0, 1, 2, \dots, N/2 - 1$, podemos escrever:

$$X[k] = \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r] W_N^{k2r} + \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r+1] W_N^{k(2r+1)}$$

o que nos leva a:

$$X[k] = \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r] (W_N^2)^{kr} + \frac{1}{\sqrt{N}} W_N^k \sum_{r=0}^{N/2-1} x[2r+1] (W_N^2)^{kr}$$

Mas note que:

$$W_N^2 = e^{\frac{-j2\pi 2}{N}} = e^{\frac{-j2\pi}{N/2}} = W_{N/2}$$

Então, temos:

$$X[k] = \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{kr} + W_N^k \frac{1}{\sqrt{N}} \sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{kr}$$

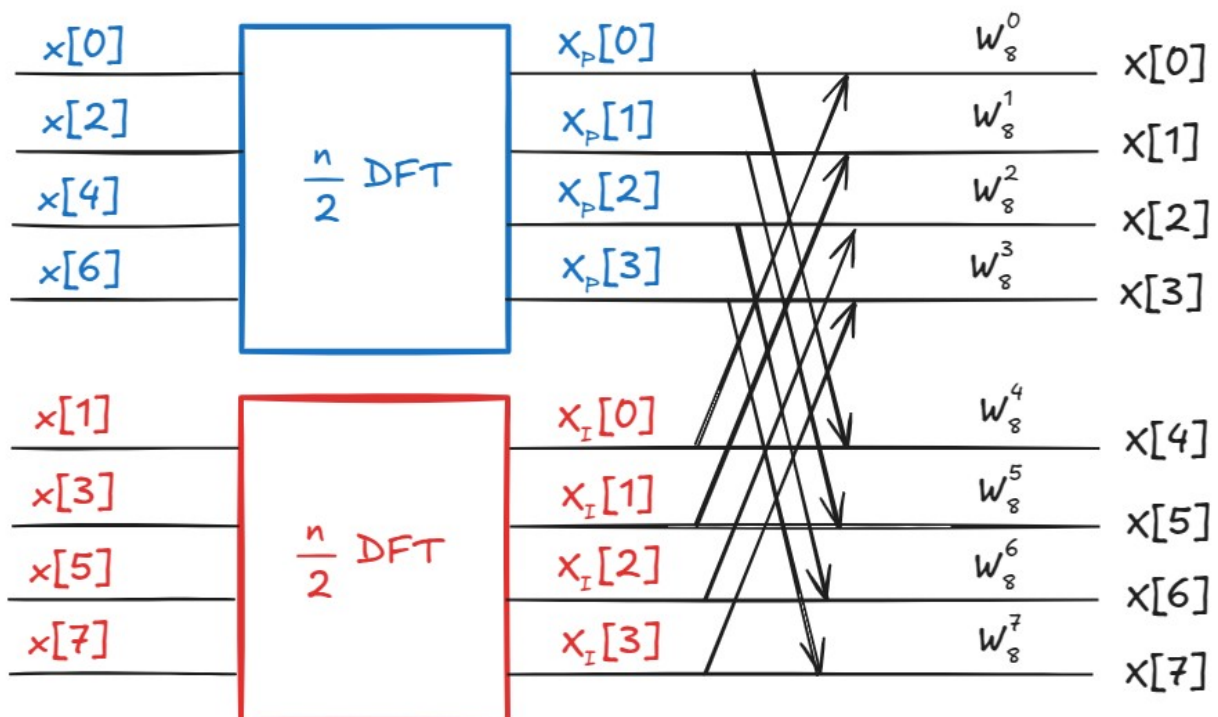
Note que o primeiro termo é justamente a DFT do sinal formado apenas pelas amostras localizadas nos índices pares do sinal, denotada por $X_p[k]$, e o segundo termo é justamente a DFT do sinal formado apenas pelas amostras localizadas nos índices ímpares do sinal, denotada por $X_I[k]$. Portanto, podemos escrever:

$$X[k] = X_p[k] + W_N^k X_I[k]$$

Na notação matemática temos:

$$\begin{aligned} X[0] &= X_p[0] + W_8^0 X_I[0] \\ X[1] &= X_p[1] + W_8^1 X_I[1] \\ X[2] &= X_p[2] + W_8^2 X_I[2] \\ X[3] &= X_p[3] + W_8^3 X_I[3] \\ X[4] &= X_p[0] + W_8^4 X_I[0] \\ X[5] &= X_p[1] + W_8^5 X_I[1] \\ X[6] &= X_p[2] + W_8^6 X_I[2] \\ X[7] &= X_p[3] + W_8^7 X_I[3] \end{aligned}$$

Podemos visualizar a FFT como um diagrama de blocos. Suponha um sinal $x[]$ com $N = 8$ amostras.



Algoritmo FFT

A seguir, apresentamos o algoritmo FFT, baseado na estratégia dividir para conquistar.

```

FFT(x, n) {
    if n == 1
        return x
     $w_n = \exp(j2\pi)/n$ 
     $X_P = (x_0, x_2, x_4, \dots, x_{n-2})$       # amostras pares
     $X_I = (x_1, x_3, x_5, \dots, x_{n-1})$     # amostras ímpares
     $X_P = \text{FFT}(X_P, n/2)$ 
     $X_I = \text{FFT}(X_I, n/2)$ 
    for k = 0 to n/2 - 1 {
         $X[k] = X_P[k] + w_n^k X_I[k]$ 
         $X[k+(n/2)] = X_P[k] + w_n^{k+n/2} X_I[k]$ 
    }
    return X
}

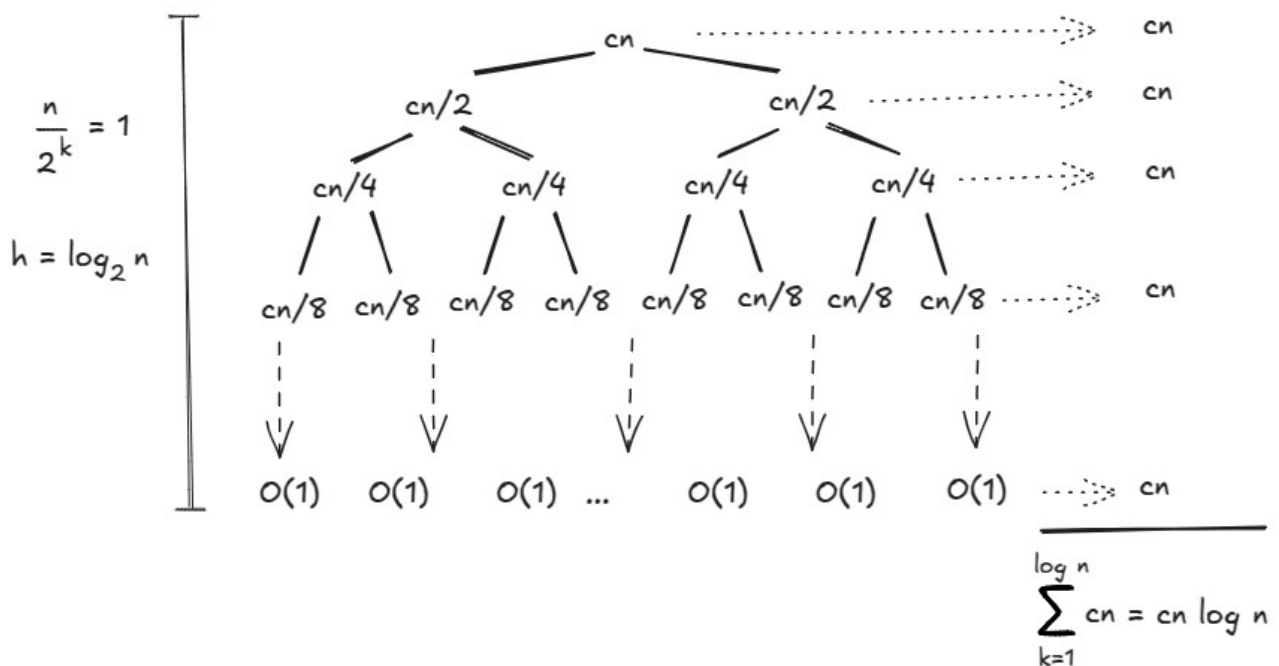
```

Análise da complexidade

Note que a recorrência para a FFT pode ser expressa como:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

pois cada FFT de N pontos é decomposta em duas FFT's de N/2 pontos além de mais N operações multiplicações. Podemos expandir a árvore de recursão.



Sabemos que o nível máximo da árvore ocorre quando $\frac{n}{2^k}=1$, ou seja o número de níveis da árvore é $k=\log_2 n$. Somando os custos em cada nível da árvore, temos o custo total do algoritmo FFT, que é dado por:

$$T(n) = \sum_{i=0}^{\log_2 n} 2^i \frac{cn}{2^i} = \sum_{i=0}^{\log_2 n} cn = cn \log_2 n$$

o que é igual a $O(n \log_2 n)$. Para se ter uma ideia de como a FFT é mais eficiente do que a DFT, apresentamos um pequeno quadro comparativo.

Número de instruções

N	10^6	10^9
N² (DFT)	10^{12}	10^{18}
N log N (FFT)	20×10^6	30×10^9

Considerando que cada instrução leva em torno de 1 nanosegundo (10^{-9} segundos), no caso do sinal ter 10^9 amostras, 10^{18} nanosegundos, que seria o tempo da DFT, equivale a aproximadamente 31 anos, enquanto que 30×10^9 nanosegundos, que é o tempo da FFT, equivale a apenas 30 segundos! Veja como temos uma diferença brutal entre a complexidade quadrática e log-linear do ponto de vista assintótico.

A seguir apresentaremos e demonstraremos o Teorema Mestre, que nos permite resolver recorrências do tipo dividir para conquistar de maneira direta. Uma recorrência do tipo dividir para conquistar tem a seguinte forma:

$$T(n) = aT\left(\frac{n}{b}\right) + g(n)$$

ou seja, 1 problema de tamanho n se divide em a problemas de tamanho n/b e gera um custo extra adicional de tamanho $g(n)$.

O Teorema Mestre

Seja uma recorrência dada por:

$$T(n) = aT\left(\frac{n}{b}\right) + cn^d$$

Então, se $n=b^k$ (potência de b , para dividir em um número exato de subproblemas) e para $k > 1$, $a \geq 1$, $b > 1$, $c > 0$ e $d \geq 0$, temos que:

1. $T(n) = O(n^d)$, se $a < b^d$
2. $T(n) = O(n^d \log n)$, se $a = b^d$
3. $T(n) = O(n^{\log_b a})$, se $a > b^d$

A seguir iremos demonstrar esse resultado teórico fundamental na análise de algoritmos. Temos que

$$T(n) = a T\left(\frac{n}{b}\right) + c n^d$$

Aplicando a recorrência novamente, temos:

$$T(n) = a \left[a T\left(\frac{n}{b^2}\right) + c \left(\frac{n}{b}\right)^d \right] + c n^d$$

o que nos leva a:

$$T(n) = a^2 T\left(\frac{n}{b^2}\right) + a c \left(\frac{n}{b}\right)^d + c n^d$$

Repetindo o processo mais uma vez, temos:

$$T(n) = a^2 \left[a T\left(\frac{n}{b^3}\right) + c \left(\frac{n}{b^2}\right)^d \right] + a c \left(\frac{n}{b}\right)^d + c n^d$$

o que nos leva a:

$$T(n) = a^3 T\left(\frac{n}{b^3}\right) + a^2 c \left(\frac{n}{b^2}\right)^d + a c \left(\frac{n}{b}\right)^d + c n^d$$

Seguindo com esse processo até o k-ésimo termo, chega-se na forma geral, dada por:

$$T(n) = a^k T\left(\frac{n}{b^k}\right) + a^{k-1} c \left(\frac{n}{b^{k-1}}\right)^d + a^{k-2} c \left(\frac{n}{b^{k-2}}\right)^d + \dots + a c \left(\frac{n}{b}\right)^d + c n^d$$

Mas lembre-se que $n = b^k$ (n é potência de b), então $\frac{n}{b^k} = 1$, o que nos leva a:

$$T(n) = a^k T(1) + \sum_{i=0}^{k-1} a^i c \left(\frac{n}{b^i}\right)^d$$

Podemos simplificar ainda mais a equação acima tirando as constantes fora do somatório:

$$T(n) = a^k T(1) + c n^d \sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i \quad (*)$$

Caso 2: Por motivos didáticos, iremos agora iniciar pelo caso 2, em que $a = b^d$. Note que neste caso o somatório é uma constante:

$$\sum_{i=0}^{k-1} \left(\frac{b^d}{b^d}\right)^i = \sum_{i=0}^{k-1} 1^i = k$$

o que nos leva a:

$$T(n) = a^k T(1) + c n^d k$$

Mas como $n = b^k$, então $k = \log_b n$. Assim, temos:

$$T(n) = a^k T(1) + c n^d \log_b n$$

Mas, neste caso, $a = b^d$:

$$T(n) = b^{dk} T(1) + c n^d \log_b n$$

Como $b^k = n$, chega-se em:

$$T(n) = n^d T(1) + c n^d \log_b n$$

e como o segundo termo é dominante, temos que $T(n)$ é $O(n^d \log_b n)$. Para analisar os demais casos (1 e 3), primeiramente note que o somatório em (*) é de fato uma soma de uma P.G.

$$S_k = \sum_{i=0}^{k-1} \left(\frac{a}{b^d} \right)^i$$

é a soma dos k primeiros termos de uma P.G. em que o primeiro termo $a_1 = 1$ e a razão $q = \left(\frac{a}{b^d} \right)$:

$$S_k = 1 \frac{\left[\left(\frac{a}{b^d} \right)^k - 1 \right]}{\left(\frac{a}{b^d} \right) - 1}$$

de modo que:

$$T(n) = a^k T(1) + c n^d \frac{\left[\left(\frac{a}{b^d} \right)^k - 1 \right]}{\left(\frac{a}{b^d} \right) - 1}$$

Multiplicando o numerador e o denominador por b^d :

$$T(n) = a^k T(1) + c n^d \left[\frac{\frac{a^k}{(b^d)^{k-1}} - b^d}{a - b^d} \right]$$

Aplicando a distributiva:

$$T(n) = a^k T(1) + c \left[\frac{n^d a^k}{a - b^d} \right] - c n^d \left(\frac{b^d}{a - b^d} \right)$$

Mas lembre que:

$$\frac{n^d}{(b^d)^{k-1}} = \frac{(b^k)^d}{(b^d)^{k-1}} = \frac{b^{kd}}{b^{kd-d}} = \frac{b^{kd}}{b^{kd} b^{-d}} = b^d$$

Isso nos permite escrever:

$$T(n) = a^k T(1) + \frac{c a^k b^d}{a - b^d} - \frac{c n^d b^d}{a - b^d}$$

Agrupando os termos iniciais em a^k (colocando em evidência):

$$T(n) = a^k \left(T(1) + \frac{c b^d}{a - b^d} \right) - n^d \left(\frac{c b^d}{a - b^d} \right)$$

Lembre-se que $n = b^k$, então $k = \log_b n$. Logo:

$$a^k = a^{\log_b n} = n^{\log_b a}$$

Assim, temos:

$$T(n) = n^{\log_b a} \left(T(1) + \frac{c b^d}{a - b^d} \right) - n^d \left(\frac{c b^d}{a - b^d} \right)$$

Podemos agora, multiplicar o numerador e o denominador do segundo termo por -1:

$$T(n) = n^{\log_b a} \left(T(1) + \frac{c b^d}{a - b^d} \right) + n^d \left(\frac{c b^d}{b^d - a} \right)$$

Iremos verificar a seguir que, na expressão anterior, se $a > b^d$ o primeiro termo domina, enquanto que se $a < b^d$ o segundo termo domina. Dessa forma, analisando o primeiro caso, temos:

Caso 1. $a < b^d$

$$\log_b a < \log_b b^d$$

$$\log_b a < d$$

$$n^{\log_b a} < n^d$$

=> Isso significa que o segundo termo de T(n) domina o primeiro!

Caso 3. $a > b^d$

$$\log_b a > \log_b b^d$$

$$\log_b a > d$$

$$n^{\log_b a} > n^d$$

=> Isso significa que o primeiro termo de $T(n)$ domina o segundo!

Portanto, temos que:

$$T(n) = O(n^d) \text{ , se } a < b^d \text{ e}$$

$$T(n) = O(n^{\log_b a}) \text{ , se } a > b^d$$

Vejam os a seguir um simples exemplo de aplicação do Teorema Mestre.

Ex: Seja A um algoritmo com a seguinte relação de recorrência:

$$T(n) = 2T\left(\frac{n}{3}\right) + 5$$

Qual é a complexidade de A? Aplicando o Teorema Mestre, temos que:

$$a = 2, b = 3, c = 1, d = 0$$

Como $b^d = 3^0 = 1$, temos que $a > b^d$. Portanto, $T(n) = O(n^{\log_3 2}) = O(n^{0.63})$.

Vimos a demonstração do Teorema Mestre na sua forma reduzida. Uma formal mais geral é apresentada a seguir. Seja uma recorrência dada por:

$$T(n) = aT\left(\frac{n}{b}\right) + cn^d \log^k n$$

Então, se $n = b^k$ (potência de b, para dividir em um número exato de subproblemas) e para $a \geq 1$, $b > 1$, $c > 0$, $d \geq 0$ e $k \geq 0$, temos que:

1. $T(n) = O(n^d \log^k n)$, se $a < b^d$
2. $T(n) = O(n^d \log^{k+1} n)$, se $a = b^d$
3. $T(n) = O(n^{\log_b a})$, se $a > b^d$

É interessante notar que existem casos inadmissíveis para o Teorema Mestre, dentre os quais podemos citar alguns exemplos:

- a) $T(n) = 2^n T\left(\frac{n}{2}\right) + n^2$: o valor de a não é constante
- b) $T(n) = 2^n T\left(\frac{n}{2}\right) + \frac{n}{\log n}$: note que $g(n) = n \log^{-1} n$, ou seja, $k < 0$ (não permitido)
- c) $T(n) = \frac{1}{2} T\left(\frac{n}{2}\right) + n$: $a < 1$ (não pode haver menos que um subproblema)
- d) $T(n) = 64 T\left(\frac{n}{8}\right) - n^2 \log n$: $g(n)$, o custo de combinação dos subproblemas, é negativo

"If you're always trying to be normal you will never know how amazing you can be."
-- Maya Angelou

Programação Dinâmica

A ideia da programação dinâmica consiste em resolver problemas combinando soluções de subproblemas. Porém, diferentemente da estratégia dividir para conquistar, onde os subproblemas são disjuntos, na programação dinâmica os problemas possuem sobreposições, ou seja, problemas maiores compartilham problemas menores.

Estratégia: resolver subproblema uma única vez (a primeira vez que aparece) armazenando resultado em uma tabela, de modo a evitar retrabalho e aumento do custo computacional.

Trata-se de uma estratégia muito útil para problemas de otimização.

Para desenvolver um algoritmo usando programação dinâmica, devemos:

1. Caracterizar a estrutura de uma solução ótima.
2. Recursivamente, definir o valor de uma solução ótima.
3. Computar o valor de uma solução ótima (tipicamente de maneira bottom-up).
4. Construir uma solução ótima a partir da informação computada.

Iniciaremos com o estudo de um problema clássico da computação: a sequência de Fibonacci.

A série de Fibonacci

A série de Fibonacci é um exemplo clássico de recursão, pois:

$F(0) = 0$ (caso base 1)

$F(1) = 1$ (caso base 2)

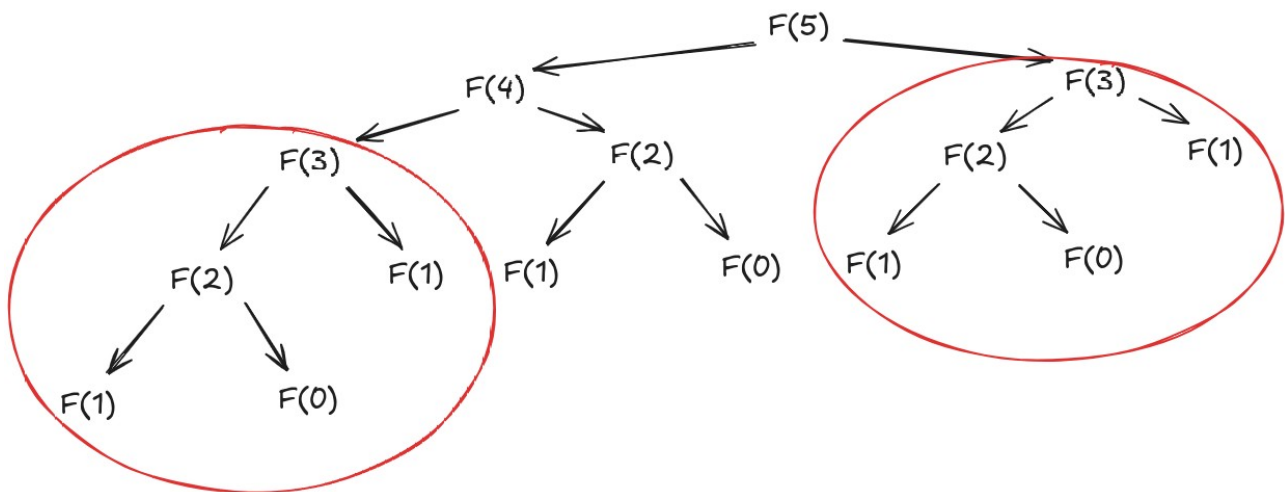
$F(n) = F(n - 1) + F(n - 2)$, para todo $n > 1$

Essa lei de formação gera a seguinte sequência: 0, 1, 1, 2, 3, 4, 8, 13, 21, 34, 55, ...

A função recursiva a seguir gera o n -ésimo termo da sequência.

```
Fib(n) {  
    if n == 0  
        return 0  
    else {  
        if n == 1 or n == 2  
            return 1  
        else  
            return Fib(n - 1) + Fib(n - 2)  
    }  
}
```

Problema com essa função recursiva: muito ineficiente. Suponha que deseja-se calcular o valor de $Fib(7)$. Não é muito eficiente de ponto de vista computacional. Isso ocorre pois durante o cálculo de $F(n)$, os valores de $F(n-2)$, $F(n-3)$, etc... são calculados várias vezes. O número de chamadas recursivas cresce exponencialmente. O padrão recursivo consiste na expansão de uma árvore binária.



Note que subproblemas possuem sobreposição: aplicar programação dinâmica!

Antes de prosseguir, vamos calcular a complexidade dessa função recursiva. Note que podemos definir a seguinte recorrência:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

Como para n grande, temos que $T(n-1) \approx T(n-2)$, iremos obter uma aproximação.

$$T(n) = 2T(n-1) + O(1)$$

$$T(n) = 2(2T(n-2) + O(1)) + O(1) = 2^2T(n-2) + 2O(1) + O(1)$$

$$T(n) = 2(2(2T(n-3) + O(1)) + O(1)) + O(1) = 2^3T(n-3) + O(1) \sum_{i=0}^2 2^i$$

Prosseguindo com essas substituições até o k -ésimo termo, chega-se em:

$$T(n) = 2^k T(n-k) + O(1) \sum_{i=0}^{k-1} 2^i = 2^k T(n-k) + (2^k - 1)O(1) = 2^k T(n-k) + 2^k O(1) = 2^k (T(n-k) + O(1))$$

A condição de parada da recursão é termos $T(0)$, ou seja, $k = n$. Logo, temos:

$$T(n) = 2^n (T(0) + O(1)) = 2^n O(1) = 2^n$$

Portanto, $T(n) = O(2^n)$, mostrando que a função possui complexidade exponencial (proibitiva).

Sendo assim, a solução para esse problema para um número elevado n é inviável. Por exemplo, suponha que tenhamos $n = 100$. Então, o número de operações necessárias para resolver o problema do Fibonacci recursivo é $2^{100} = 1.2676506 \times 10^{30}$. Considerando que cada instrução é executada em 1 nanosegundo, ou seja, 10^{-9} segundos, o tempo gasto seria 1.2676506×10^{21} segundos, o que equivale a aproximadamente 4.01×10^{13} anos! Para se ter uma ideia de quão grande isso é, a idade estimada do universo é de 26.7 bilhões de anos, o que seria 26.7×10^9 anos.

A programação dinâmica, em geral, pode ser implementada a partir de 2 estratégias distintas:

1. Top-Down: através de memorização (tabela auxiliar para evitar recálculo dos subproblemas)
2. Bottom-Up: através da reversão do problema (abordagem não recursiva)

Veremos agora como utilizar a memorização para melhorar a função Fibonacci.

```
Fib_M(n) {  
    m[0..n] = 0  
    if n == 1 or n == 2  
        m[n] = 1  
    else {  
        if m[n] == 0  
            m[n] = Fib(n - 1) + Fib(n - 2)  
    }  
    return m[n]  
}
```

Note que a função Fib_M calcula cada subproblema apenas uma única vez, ou seja, diminui a complexidade significativamente, pois emprega uma tabela auxiliar para armazenar o resultado dos subproblemas.

Na estratégia bottom-up, devemos reverter a direção que o algoritmo funciona, ou seja, iniciar do caso base e progredir em direção a solução. Essa é uma outra forma de implementar a programação dinâmica. Em geral, com essa estratégia, abordamos o problema de forma iterativa e não recursiva. A função a seguir ilustra essa ideia.

```
Fib_B(n) {  
    if n == 0  
        return 0  
    else {  
        previous = 0  
        current = 1  
        for i = 1 to n {  
            new = previous + current  
            previous = current  
            current = new  
        }  
    }  
    return current  
}
```

Note que a complexidade da função anterior é $O(n)$, o que é bem melhor que exponencial.

Não são todos os algoritmos recursivos que aceitam a programação dinâmica. Por exemplo, os algoritmos de ordenação MergeSort e QuickSort não podem utilizar programação dinâmica pois os subproblemas **não possuem sobreposição** (são totalmente disjuntos, portanto a estratégia adequada é a dividir para conquistar).

A seguir veremos outras aplicações da programação dinâmica como o problema da sequência de cédulas, o problema do robô coletor, o problema do corte da haste e o problemas de caminhos mínimos em grafos.

O problema da sequência de cédulas

Suponha uma sequência de n cédulas de dinheiro cujos valores são inteiros positivos $c_1, c_2, \dots, c_n$ não necessariamente distintos. O objetivo consiste em coletar a maior quantidade de dinheiro sujeito a restrição de que duas cédulas vizinhas não podem ser coletadas.

Seja $F(n)$ o valor máximo que pode ser coletado no arranjo de tamanho n . Para derivar uma recorrência utilizando programação dinâmica iremos separar em duas possibilidades:

1. a que inclui a cédula corrente;
2. a que não inclui a cédula corrente.

Note que o máximo valor que pode ser coletado na possibilidade 1 é: $c_n + F(n-2)$, que é o valor da cédula atual mais o máximo valor coletado das primeiras $n-2$ cédulas (aqui não consideramos a cédula imediatamente anterior pois seria uma violação das regras).

O máximo valor que pode ser coletado na possibilidade 2 é: $F(n-1)$, que é o máximo valor coletado das $n-1$ cédulas anteriores (aqui não considera a atual, o valor coletado até a posição anterior pode ser considerado). Sendo assim, podemos definir a seguinte recursão:

$$F(n) = \max\{c_n + F(n-2), F(n-1)\} \quad \text{se} \quad n > 1$$

com as seguintes condições iniciais:

$$F(0) = 0 \quad \text{e} \quad F(1) = c_1$$

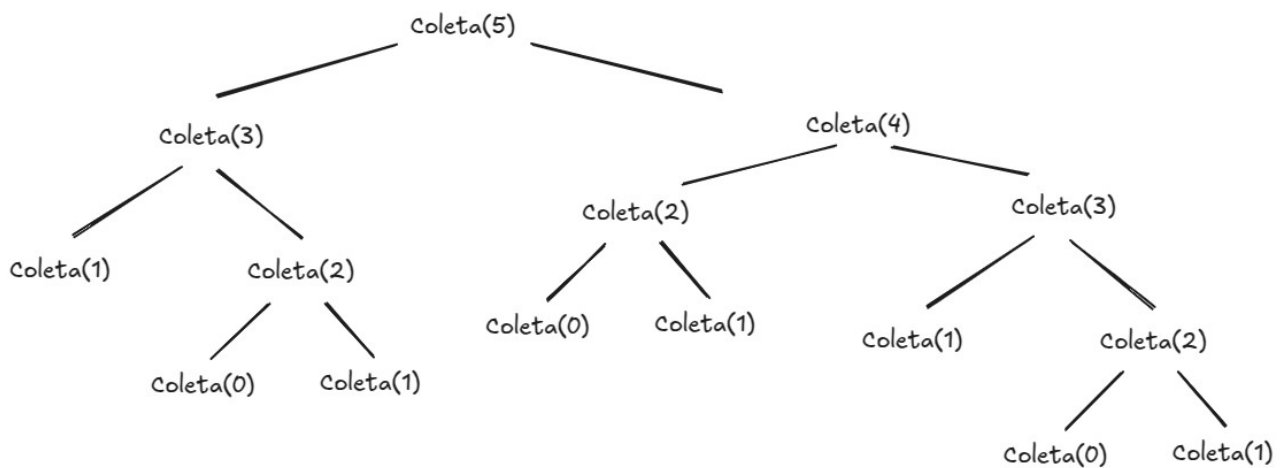
Podemos implementar a seguinte função recursiva para solucionar o problema.

Seja c o vetor com os valores das cédulas (iniciando no índice 1)
 $c = [x, y, \dots, z]$

```
Coleta(c, n) {  
    if  $n \leq 0$   
        return 0  
    else {  
        if  $n == 1$   
            return  $c[1]$   
        else  
            return  $\max(c[n] + \text{Coleta}(c, n - 2), \text{Coleta}(c, n - 1))$   
    }  
}
```

O algoritmo anterior possui um alto custo computacional, pois possui uma recursão dupla, ou seja, uma estrutura muito similar ao problema da sequência de Fibonacci recursiva.

Porém, qual é a complexidade dessa função? Vejamos a árvore de recursão. Suponha que $n = 5$, ou seja, temos apenas 5 cédulas.



Note que temos uma recorrência muito similar com a sequência de Fibonacci, pois ao invés de somar, tomamos o máximo entre dois valores, o que também é feito em $O(1)$. Podemos definir a seguinte recorrência:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

Como para n grande, temos que $T(n-1) \approx T(n-2)$, iremos obter uma aproximação.

$$T(n) = 2T(n-1) + O(1)$$

Vimos que a solução de tal recorrência nos leva a conclusão de que $T(n) = O(2^n)$, mostrando que a função possui complexidade exponencial (proibitiva). Como podemos melhorar? Utilizando uma abordagem bottom-up baseada em programação dinâmica.

Seja c é o vetor com os valores das cédulas (iniciando em 1)
 $c = [x, y, \dots, z]$

```

Coleta_PD(c, n) {
    F = zeros(n)      # cria um array de zeros de tamanho n
    F[0] = 0
    F[1] = c[1]
    # simples loop com complexidade O(n)
    for i = 2 to n
        F[i] = max(c[i]+F[i-2], F[i-1])
    return F[n]
}
  
```

É fácil perceber que a complexidade desse algoritmo é $O(n)$, o que é muito melhor que exponencial! Ou seja, com a programação dinâmica através de uma abordagem bottom-up baseada na reversão da recursão, podemos projetar um algoritmo muito mais eficiente.

Exemplo: Suponha que tenhamos as seguintes cédulas:

$c = [2, 5, 10, 20, 10, 2, 50, 100]$

As condições iniciais nos dizem que:

$F[0] = 0$
 $F[1] = c[1] = 2$

e, fazendo uso da programação dinâmica, temos a iteração:

$$F[i] = \max\{c[i] + F[i-2], F[i-1]\}$$

Devemos iterar de i igual 2 até n = 8:

i = 2: F = [0, 2, ___]

$$F[2] = \max\{5+0, 2\} = 5 \quad \text{como } 5 + 0 > 2, \text{ então } \pi[5] = 0$$

i = 3: F = [0, 2, 5, ___]

$$F[3] = \max\{10+2, 5\} = 12 \quad \text{como } 10 + 2 > 5, \text{ então } \pi[12] = 2$$

i = 4: F = [0, 2, 5, 12, ___]

$$F[4] = \max\{20+5, 12\} = 25 \quad \text{como } 20 + 5 > 12, \text{ então } \pi[25] = 5$$

i = 5: F = [0, 2, 5, 12, 25, ___]

$$F[5] = \max\{10+12, 25\} = 25 \quad \text{como } 10 + 12 < 25, \text{ então nada a fazer}$$

i = 6: F = [0, 2, 5, 12, 25, 25, ___]

$$F[6] = \max\{2+25, 25\} = 27 \quad \text{como } 2 + 25 > 25, \text{ então } \pi[27] = 25$$

i = 7: F = [0, 2, 5, 12, 25, 25, 27, ___]

$$F[7] = \max\{50+25, 27\} = 75 \quad \text{como } 50 + 25 > 27, \text{ então } \pi[75] = 25$$

i = 8: F = [0, 2, 5, 12, 25, 25, 27, 75, ___]

$$F[8] = \max\{100+27, 75\} = 127 \quad \text{como } 100 + 27 > 75, \text{ então } \pi[127] = 27$$

Resultado: 127

Como saber quais notas da sequência foram pegas: Backtracking! (usar mapa de predecessores)

c = [2, 5, 10, 20, 10, 2, 50, 100]

F = [0, 2, 5, 12, 25, 25, 27, 75, 127]

Quem me fez chegar em 127? **100** + 27 → Nota de **100**

Quem me fez chegar em 27? **2** + 25 → Nota de **2**

Quem me fez chegar em 25? **20** + 5 → Nota de **20**

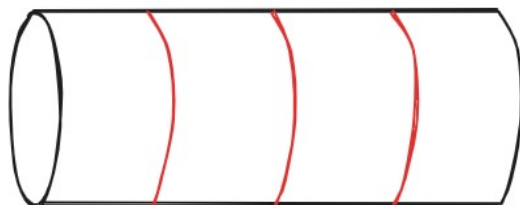
Quem me fez chegar em 5? **5** + 0 → Nota de **5**

Predecessores

F[i]	0	2	5	12	25	27	75	127
$\pi(F[i])$	-	0	0	2	5	25	25	27

O problema do corte da haste (The rod cutting problem)

Objetivo: dada uma haste de comprimento n e o preço associado a cada pedaço da haste, cortar e vender os pedaços de modo a maximizar o ganho.



Note que uma partição válida da haste deve satisfazer:

$$n = i_1 + i_2 + i_3 + \dots + i_k$$

onde i_k é o tamanho do k -ésimo pedaço. Supondo que $n = 4$, ou seja, a haste pode ser cortada em até 4 pedaços, todas as partições possíveis da haste são:

- a) 4
- b) 3, 1
- c) 2, 2
- d) 2, 1, 1
- e) 1, 3
- f) 1, 2, 1
- g) 1, 1, 2
- h) 1, 1, 1, 1

Isso resulta num total de $2^{n-1} = 2^3 = 8$ maneiras distintas. Iremos definir algumas variáveis:

p_i : preço de um pedaço da haste de tamanho i ($i = 1, 2, 3, \dots, n$)

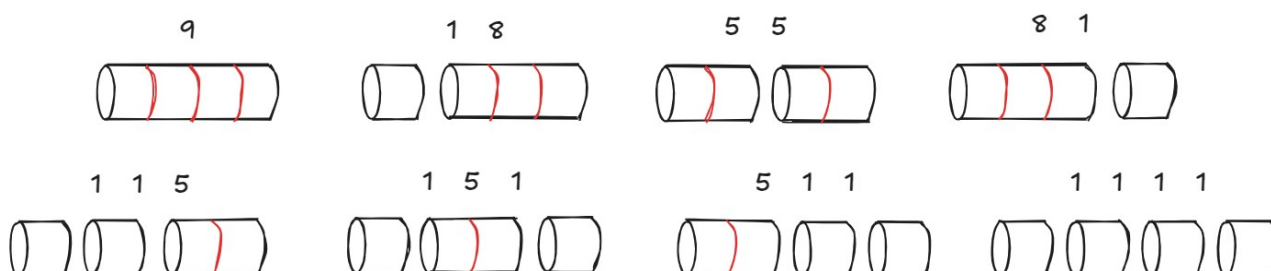
Suponha que para o nosso problema, o custo seja definido pela seguinte tabela:

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

Desejamos maximizar o valor de venda (ganho), definido como:

$$r_n = p_{i_1} + p_{i_2} + p_{i_3} + \dots + p_{i_k}$$

onde p_{i_k} é o preço do pedaço i_k . A figura a seguir ilustra os cortes e os preços dos pedaços.



Note que no nosso problema em particular ($n = 4$), temos os seguintes valores de venda:

$$4: r_n = 9$$

$$3, 1: r_n = 8 + 1 = 9$$

$$2, 2: r_n = 5 + 5 = 10$$

$$2, 1, 1: r_n = 5 + 1 + 1 = 7$$

$$1, 1, 1, 1: r_n = 1 + 1 + 1 + 1 = 4$$

Note que, na prática, mesmo que não precisemos avaliar todas as 2^{n-1} possibilidades, esse número, conhecido como função de partição, converge assintoticamente para:

$$\frac{e^{\pi\sqrt{2n/3}}}{4n\sqrt{3}}$$

o que ainda é exponencial em n , ou seja, o espaço de busca é muito grande para um algoritmo de força bruta. Para iniciar a formulação do problema, note que podemos expressar os valores r_n , para $n \geq 1$ em termos dos valores de venda ótimos das hastes menores:

$$r_n = \max \{ p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, r_3 + r_{n-3}, \dots, r_{n-1} + r_1 \}$$

onde p_n é o preço da haste sem cortes e os demais valores são referentes aos valores de venda de cortes em dois pedaços de tamanho i e $n - i$, para $i = 1, 2, \dots, n - 1$.

Note que para resolver o problema original de tamanho n , resolvemos problemas menores. Ao fazer o primeiro corte, os 2 pedaços resultantes formam instâncias menores do mesmo problema. A solução ótima final incorpora as soluções ótimas dos problemas menores. Dizemos que o problema do corte da haste exibe subestrutura ótima: soluções ótimas para um problema incorporam soluções ótimas aos subproblemas relacionados.

Podemos simplificar a formulação do problema pensada em toda decomposição de uma haste de tamanho n como o primeiro pedaço seguida da decomposição do restante:

$$r_n = \max \{ p_i + r_{n-i} : 1 \leq i \leq n \}$$

Nessa formulação, uma solução ótima depende da solução de um subproblema e não de dois! Por isso, nos leva a um algoritmo mais eficiente.

A seguir veremos uma simples abordagem top-down recursiva que recebe como parâmetro um vetor de preços $p[1:n]$ e um inteiro n que representa o tamanho da haste.

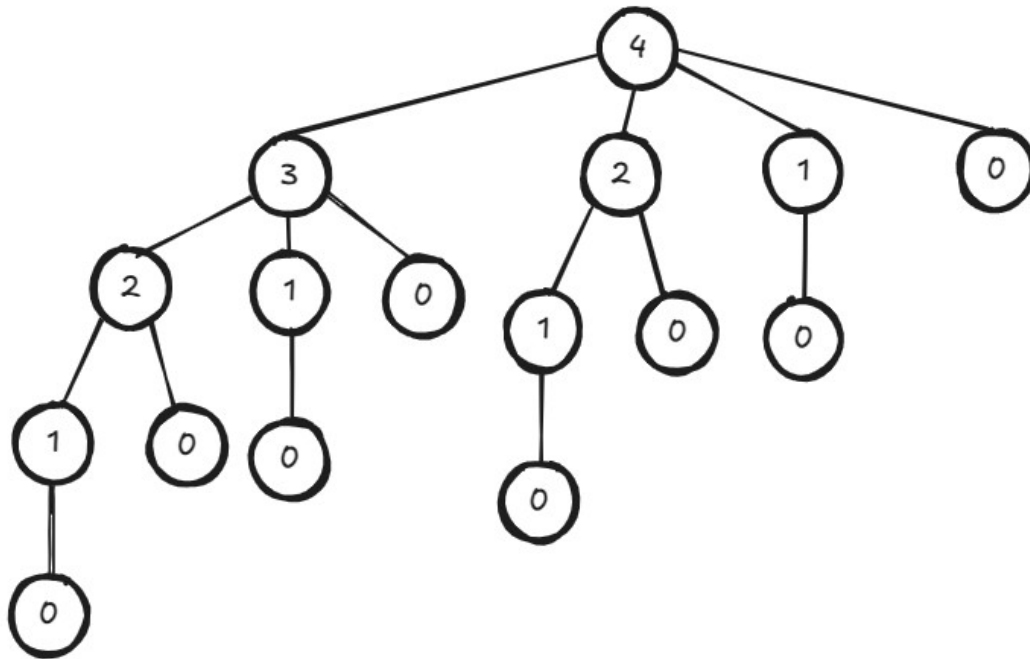
```

Cut_Rod(p, n) {
    if n == 0
        return 0
    q = -inf
    for i = 1 to n
        q = max{q, p[i] + CutRod(p, n-i)}
    return q
}

```

Porém, essa implementação não é eficiente. Vejamos o que ocorre quando $n = 4$. Na primeira chamada, note que o loop FOR vai de 1 até 4. Então, ao entrar no loop temos 4 chamadas

recursivas, para os valores de n iguais a 3, 2, 1 e 0. Dentro da chamada recursiva de n = 3, o loop FOR irá chamar a função recursivamente mais 3 vezes: 2, 1 e 0. Dessa forma, temos a seguinte árvore de recursão.



Seja $T(n)$ o número total de chamadas da função Cut_Rod (número de nós na árvore). Então, o custo total é dado por:

$$T(n) = T(n-1) + T(n-2) + \dots + T(0) = \sum_{j=0}^{n-1} T(j) = \sum_{j=0}^{n-1} (2^j - 1) = \sum_{j=0}^{n-1} 2^j - \sum_{j=0}^{n-1} 1$$

Note que $2^{j+1} = 2 \cdot 2^j = 2^j + 2^j$, o que nos leva a $2^j = 2^{j+1} - 2^j$. Usando a soma telescópica:

$$\sum_{j=0}^{n-1} 2^j = \sum_{j=0}^{n-1} (2^{j+1} - 2^j) = 2^n - 2^0 = 2^n - 1$$

Portanto, temos que:

$$T(n) = 2^n - 1 - (n-1)$$

ou seja, a complexidade da função é $O(2^n)$ (exponencial é proibitivo).

A seguir apresentaremos uma solução top-down com programação dinâmica a partir da técnica de memorização.

```

Cut_Rod_Mem(p, n) {
    r = zeros(n)      # cria um array de zeros de tamanho n
    for i = 0 to n
        r[i] = -inf
    return Cut_Rod_Mem_Aux(p, n, r)
}

```

```

Cut_Rod_Mem_Aux(p, n, r) {
    if r[n] >= 0

```

```

        return r[n]
    if n == 0
        q = 0
    else {
        q = -inf
        for i = 0 to n
            q = max{q, p[i] + Cut_Rod_Mem_Aux(p, n-i, r)}
    }
    r[n] = q
    return q
}

```

Note que nessa estratégia baseada em programação dinâmica a função resolve cada subproblema apenas uma única vez, ou seja, ela resolve os problemas para os tamanhos 0, 1, 2,..., n. Mas para resolver um problema de tamanho n, o loop FOR itera por n vezes, Por essa razão, essa função tem complexidade $O(n^2)$.

Veremos a seguir uma solução bottom-up baseada em programação dinâmica (reversão do processo, eliminando a recursão). A ideia aqui consiste em resolver cada subproblema de tamanho j em ordem crescente.

```

Cut_Rod_B(p, n) {
    r = zeros(n)           # cria um array de zeros de tamanho n
    r[0] = 0
    for j = 1 to n {
        q = -inf
        for i = 1 to j
            q = max{q, p[i] + r[j-i]}
        r[j] = q
    }
    return r[n]
}

```

A diferença aqui é que partimos de r[0] e vamos subindo em direção a r[n] no sentido contrário a recursão. O cálculo da complexidade do algoritmo bottom-up é dado pela solução do seguinte somatório:

$$T(n) = \sum_{i=1}^n i = 1+2+3+\dots+n$$

Note que $(i+1)^2 = i^2 + 2i + 1$, o que nos leva a:

$$(i+1)^2 - i^2 = 2i + 1$$

Aplicando somatório em ambos os lados:

$$\sum_{i=1}^n [(i+1)^2 - i^2] = 2 \sum_{i=1}^n i + \sum_{i=1}^n 1$$

Pela soma telescópica, temos:

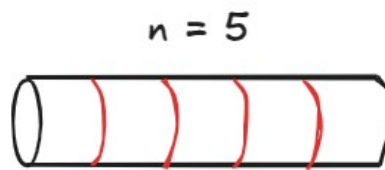
$$(n+1)^2 - 1 = 2 \sum_{i=1}^n i + n$$

Isolando o somatório finalmente nos fornece:

$$\sum_{i=1}^n i = \frac{n^2 + 2n + 1 - 1 - n}{2} = \frac{n(n+1)}{2}$$

Portanto, a complexidade dessa função é $O(n^2)$. Vimos que neste problema em particular, com o auxílio da programação dinâmica conseguimos reduzir a complexidade do algoritmo inicial de exponencial para quadrática!

Exemplo: Seja uma haste de tamanho $n = 5$ como a seguir



e a função de custo definida pela seguinte tabela:

i	1	2	3	4	5
p_i	1	3	5	6	7

Iniciamos fazendo $q = -\infty$

$r = [0, _, _, _, _, _]$ (r e c são arrays que iniciam em zero)
 $c = [-, _, _, _, _, _]$

j = 1:

$$i = 1: q = \max\{q, p_i + r[j-i]\} = \max\{-\infty, p_1 + r[0]\} = \max\{-\infty, 1+0\} = 1$$

$r = [0, 1, _, _, _, _]$
 $c = [-, 1, _, _, _, _]$ (adicionar em $c[j]$ o valor de i que fez aumentar o máximo)

j = 2:

$$i = 1: q = \max\{q, p_i + r[j-i]\} = \max\{-\infty, p_1 + r[1]\} = \max\{-\infty, 1+1\} = 2$$

$$i = 2: q = \max\{q, p_i + r[j-i]\} = \max\{2, p_2 + r[0]\} = \max\{2, 3+0\} = 3$$

$r = [0, 1, 3, _, _, _]$
 $c = [-, 1, 2, _, _, _]$ (adicionar em $c[j]$ o valor de i que fez aumentar o máximo)

j = 3:

$$i = 1: q = \max\{q, p_i + r[j-i]\} = \max\{-\infty, p_1 + r[2]\} = \max\{-\infty, 1+3\} = 4$$

$$i = 2: q = \max\{q, p_i + r[j-i]\} = \max\{4, p_2 + r[1]\} = \max\{4, 3+1\} = 4$$

$$i = 3: q = \max\{q, p_i + r[j-i]\} = \max\{4, p_3 + r[0]\} = \max\{4, 5+0\} = 5$$

$r = [0, 1, 3, 5, _, _]$
 $c = [-, 1, 2, 3, _, _]$ (adicionar em $c[j]$ o valor de i que fez aumentar o máximo)

j = 4:

$$\begin{aligned} i = 1: & \quad q = \max\{q, p_i + r[j-i]\} = \max\{-\infty, p_1 + r[3]\} = \max\{-\infty, 1+5\} = 6 \\ i = 2: & \quad q = \max\{q, p_i + r[j-i]\} = \max\{6, p_2 + r[2]\} = \max\{6, 3+3\} = 6 \\ i = 3: & \quad q = \max\{q, p_i + r[j-i]\} = \max\{6, p_3 + r[1]\} = \max\{6, 5+1\} = 6 \\ i = 4: & \quad q = \max\{q, p_i + r[j-i]\} = \max\{6, p_4 + r[0]\} = \max\{6, 6+0\} = 6 \end{aligned}$$

$r = [0, 1, 3, 5, 6, _]$
 $c = [-, 1, 2, 3, 1, _]$ (adicionar em $c[j]$ o valor de i que fez aumentar o máximo)

j = 5:

$$\begin{aligned} i = 1: & \quad q = \max\{q, p_i + r[j-i]\} = \max\{-\infty, p_1 + r[4]\} = \max\{-\infty, 1+6\} = 7 \\ i = 2: & \quad q = \max\{q, p_i + r[j-i]\} = \max\{6, p_2 + r[3]\} = \max\{7, 3+5\} = 8 \\ i = 3: & \quad q = \max\{q, p_i + r[j-i]\} = \max\{8, p_3 + r[2]\} = \max\{8, 5+3\} = 8 \\ i = 4: & \quad q = \max\{q, p_i + r[j-i]\} = \max\{8, p_4 + r[1]\} = \max\{8, 6+1\} = 8 \\ i = 5: & \quad q = \max\{q, p_i + r[j-i]\} = \max\{8, p_5 + r[0]\} = \max\{8, 7+0\} = 8 \end{aligned}$$

$r = [0, 1, 3, 5, 6, 8]$
 $c = [-, 1, 2, 3, 1, 2]$ (adicionar em $c[j]$ o valor de i que fez aumentar o máximo)

Solução: 8 (é o custo máximo que podemos obter vendendo os pedaços da haste)

Qual é a partição obtida? Backtracking!

Iniciar com $i = n = 5$ (haste completa)

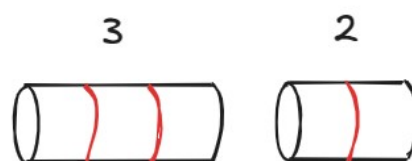
$c[5] = 2 \rightarrow$ Corte de 2 unidades

Restante = $5 - 2 = 3$

$c[3] = 3 \rightarrow$ Corte de 3 unidades

Restante = $3 - 3 = 0$

Portanto, a partição ótima da haste é dada por:



em que o pedaço de tamanho 3 custa 5 e o pedaço de tamanho 2 custa 3, gerando o custo total de 8.

O problema do robô coletor de moedas

Suponha que diversas moedas são posicionadas em células de um tabuleiro $n \times m$, sendo apenas 1 moeda por célula. Um robô localizado no canto superior esquerdo precisa coletar o maior número de moedas possível e trazê-las para o canto inferior direito.

A cada passo, o robô pode mover uma célula para a direita ou uma célula para baixo de sua posição atual. Ao visitar uma célula com uma moeda, o robô sempre coleta a moeda em questão.

Seja $F(i, j)$ o maior número de moedas que o robô pode coletar e trazer para a célula (i, j) .

Ele pode atingir essa célula tanto da célula de cima $(i-1, j)$ quanto da célula a esquerda $(i, j-1)$.

Os maiores números de moedas que podem ser trazidos até essas células são $F(i-1, j)$ e $F(i, j-1)$, respectivamente.

Obviamente, não há células vizinhas acima da primeira linha, nem a esquerda da primeira coluna (nas bordas). Para as células das bordas do tabuleiro, assumimos que $F(i-1, j)$ e $F(i, j-1)$ são nulos (condições iniciais).

Assim, o maior número de moedas que podem ser trazidas até a célula (i, j) é o máximo entre essas duas quantidades mais uma possível moeda na célula (i, j) . Em outras palavras:

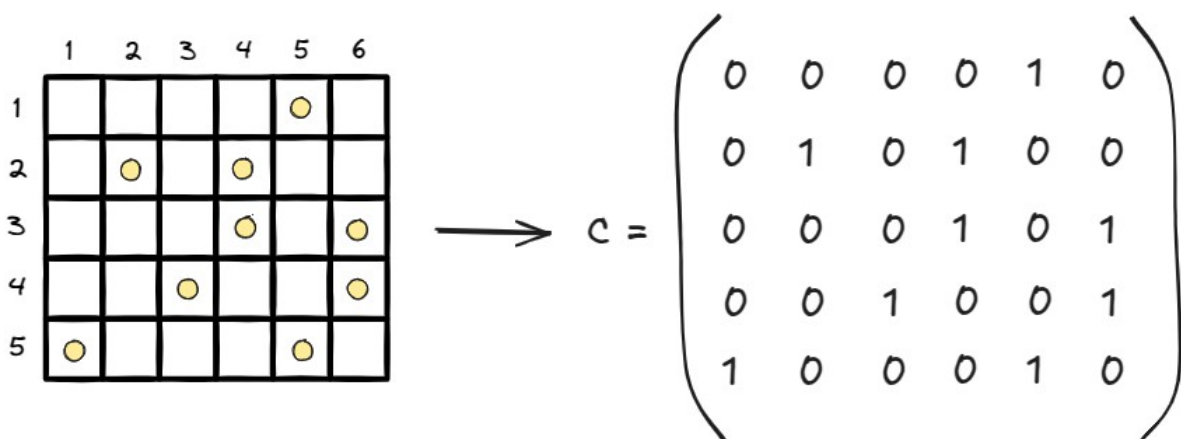
$$F(i, j) = \max\{F(i-1, j), F(i, j-1)\} + c_{ij} \quad \text{para } 1 \leq i \leq n, 1 \leq j \leq m$$

sujeito a $F(0, j) = 0$ para $1 \leq j \leq m$ e $F(i, 0) = 0$ para $1 \leq i \leq n$

As condições iniciais representam bordas na vertical da esquerda e também na horizontal superior. Da mesma forma que os problemas anteriores, somos tentados a pensar recursivamente. Porém, neste problema, a solução recursiva se mostra inviável, pelo mesmo motivo que os anteriores (a árvore de recursão gera um número de nós que cresce exponencialmente).

C é a matriz com as moedas dispostas (1 indica presença e 0 ausência)
 $C = \text{matriz}(n, m)$

Por exemplo, considere o seguinte arranjo de moedas e sua representação matricial.



Sendo assim, uma versão recursiva do algoritmo para resolver o problema é dada por:

```
Robo(C, i, j) {  
    if i == 0 or j == 0  
        return 0  
    else {  
        return max(Robo(C, i-1, j), Robo(C, i, j-1)) + C[i, j]  
    }  
}
```

Podemos observar um padrão de recursão que é bem similar aos problemas da sequência de Fibonacci e da sequência de cédulas, dada por:

$$T(n) = T(n-1) + T(n-1) + O(1)$$

o que nos permite escrever:

$$T(n) = 2T(n-1) + O(1)$$

Vimos anteriormente que a solução de tal recorrência nos leva a conclusão de que $T(n) = O(2^n)$, mostrando que a função possui complexidade exponencial (proibitiva). Como podemos melhorar? Trata-se de um cenário perfeito para a aplicação de técnicas de programação dinâmica.

A ideia consiste em construir uma matriz de custos acumulados F, em que cada elemento F[i, j] armazena o máximo número de moedas que podem ser coletadas até a célula (i, j).

Uma ilustração gráfica do problema pode ser encontrada na figura a seguir.

	1	2	3	4	5	6
1	0	0	0	0	1	1
2	0	1	1	2	2	2
3	0	1	1	3	3	4
4	0	1	2	3	3	5
5	1	1	2	3	4	5

Para preencher essa matriz de custos acumulados F, devemos utilizar um esquema de propagação de rótulos. Após o preenchimento total da matriz F, podemos utilizar um esquema de backtracking para encontrar o melhor caminho (partindo do destino e voltando para a origem). O algoritmo a seguir fornece uma solução eficiente baseada na programação dinâmica para o problema de preencher a matriz de custos acumulados F.

C é a matriz binária n x m em que um valor 1 indica presença de moeda na célula e um valor 0 indica ausência.

```
RoboColetor(C) {  
    F = zeros(n, m)          # cria matriz n x m de zeros  
    F[1, 1] = C[1, 1]
```

```

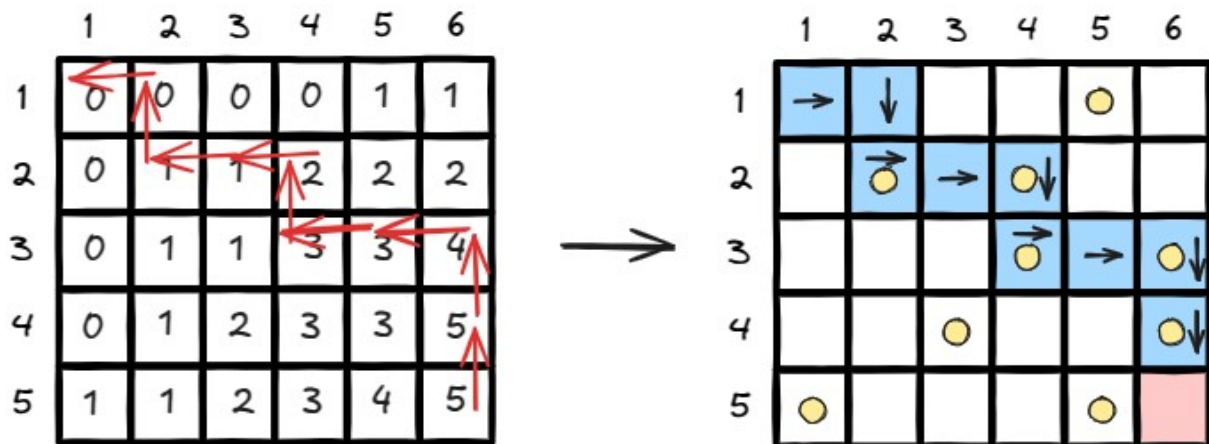
# Inicializa a primeira linha de F
for j = 2 to m
    F[1, j] = F[1, j-1] + C[1, j]
# Loop principal
for i = 2 to n {
    F[i, 1] = F[i-1, 1] + C[i, 1]    # primeiro elemento da coluna
    for j = 2 to n
        F[i, j] = max(F[i-1, j], F[i, j-1]) + C[i, j]
    }
return F[n, m]
}

```

Note que a função anterior retorna em $F[n, m]$ o número máximo de moedas que podem ser coletadas pelo robô. Podemos ver que a complexidade do algoritmo em questão é $O(nm)$, o que é eficiente se compararmos com uma complexidade exponencial da solução recursiva, mostrando mais uma vez que a programação dinâmica é uma estratégia muito interessante para tornar algoritmos recursivos mais eficientes.

Para construir o caminho que o robô deve percorrer, podemos utilizar backtracking na matriz F gerada pelo algoritmo. Iniciando do canto inferior direito, temos que:

- a) se $F[i-1, j] > F[i, j-1]$ o caminho ótimo deve acessar a posição (i, j) vindo da célula superior.
- b) se $F[i-1, j] < F[i, j-1]$ o caminho ótimo deve acessar a posição (i, j) vindo da célula a esquerda.
- c) se $F[i-1, j] = F[i, j-1]$ o caminho ótimo pode vir tanto da célula superior quanto da célula a esquerda (escolha qualquer um).



Com essa estratégia, note que qualquer caminho construído irá passar obrigatoriamente por $n + m$ células (pois devemos sair do canto inferior direito – destino – e voltar para o canto superior esquerdo – origem). Sendo assim, o custo computacional para a construção do caminho ótimo é $O(n+m)$.

Caminhos mínimos em grafos

Def: Seja $G = (V, E, w)$ um grafo ponderado com função de custo $w: E \rightarrow \mathbb{R}^+$. O peso do caminho $P = v_0 v_1 v_2 \dots v_n$ é dado por:

$$w(P) = \sum_{i=1}^n w(v_{i-1}, v_i)$$

Define-se o caminho ótimo P^* de v_0 a v_n como aquele que minimiza $w(P)$, ou seja:

$$P^* = \underset{P}{\operatorname{argmin}} w(P)$$

se existe um caminho de v_0 a v_n . Se não existe caminho P de v_0 a v_n , então o custo do caminho é infinito, ou seja, $w(P) = \infty$.

Veremos a seguir que caminhos ótimos possuem a propriedade de subestrutura ótima.

Teorema: Seja $G = (V, E, w)$ um grafo ponderado e $P = v_0 v_1 v_2 \dots v_n$ o caminho mínimo de v_0 a v_n . Seja ainda, para $0 \leq i < j \leq n$, $P_{ij} = v_i v_{i+1} \dots v_j$ o subcaminho de P de v_i a v_j . Então, P_{ij} é o caminho mínimo de v_i a v_j .

Prova:

1. Podemos decompor P em 3 subcaminhos:



2. Logo, o peso total do caminho P é dado por:

$$w(P) = w(P_{0i}) + w(P_{ij}) + w(P_{jn})$$

3. Suponha que exista um caminho P'_{ij} de v_i a v_j tal que $w(P'_{ij}) < w(P_{ij})$.

4. Então, o caminho $\bar{P}$, definido como:

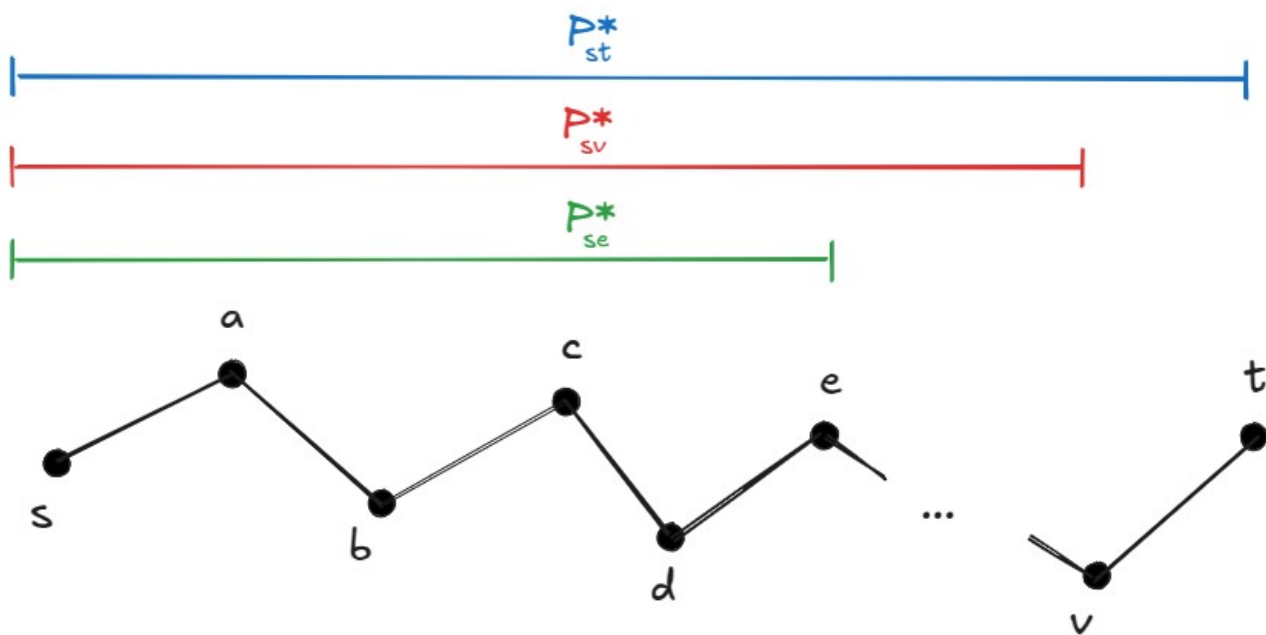


com peso:

$$w(\bar{P}) = w(P_{0i}) + w(P'_{ij}) + w(P_{jn})$$

possui $w(\bar{P}) < w(P)$, o que contradiz a hipótese de que P é o caminho mínimo. Portanto, não existe P'_{ij} diferente de P_{ij} com $w(P'_{ij}) < w(P_{ij})$.

Esse resultado é muito importante pois permite a aplicação da programação dinâmica!



Note que os caminhos mínimos P_{st}^* , P_{sv}^* e P_{se}^* compartilham arestas em comum, então temos um problema grande sendo dividido em vários subproblemas menores mas com sobreposição. A estratégia divisão e conquista não serve para esse problema, mas a programação dinâmica com certeza serve.

O algoritmo Floyd-Warshall (All-pairs shortest path)

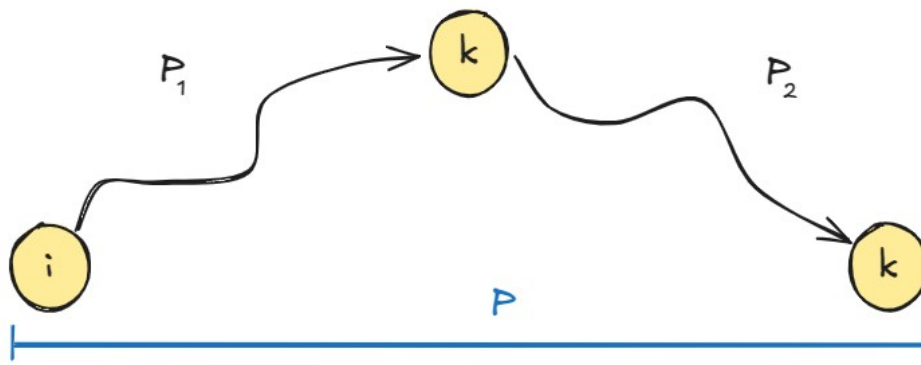
Todo caminho mínimo é composto pela origem, destino e vértices intermediários:

$$P = v_1 \ v_2 \ v_3 \ \dots \ v_{n-1} \ v_n$$

intermediários

Supondo que os vértices de G são denotados por $V = \{1, 2, 3, \dots, n\}$, podemos definir o conjunto $U_k = \{1, 2, \dots, k\}$, para $1 \leq k \leq n$. Para um par de vértices arbitrário $i, j \in V$ considere todos os caminhos de i até j cujos vértices intermediários pertencem a U_k e seja P_{ij}^* o menor deles (caminho mínimo). Podemos relacionar P_{ij}^* com caminhos mínimos P_{ij} de i até j que contém todos os vértices intermediários no subconjunto $U_{k-1} = \{1, 2, \dots, n-1\}$ da seguinte forma:

1. Se $k \notin P_{ij}^*$ então P_{ij} também é um caminho mínimo de i até j com todos os vértices intermediários em $U_k = \{1, 2, \dots, k\}$
2. Se $k \in P_{ij}^*$ então temos a seguinte figura ilustrativa:



P_1 : todos os vértices intermediários em $\{1, 2, \dots, K-1\}$

P_2 : todos os vértices intermediários em $\{1, 2, \dots, K-1\}$

P : todos os vértices intermediários em $\{1, 2, \dots, K\}$

Portanto, pela subestrutura ótima dos caminhos mínimos, p_1 é o caminho mínimo de i até k com vértices intermediários em U_{k-1} e p_2 é o caminho mínimo de k até j com vértices intermediários em U_{k-1} . Ambos os caminhos p_1 e p_2 possuem vértices intermediários em U_{k-1} .

Seja $d_{ij}^{(k)}$ o custo do caminho mínimo de i até j com todos os vértices intermediários em U_k . Quando $k = 0$, não há vértices intermediários e $d_{ij}^{(0)} = w_{ij}$, onde a matriz de adjacências W é:

$$w_{ij} = \begin{cases} 0, & \text{se } i=j \\ w(v_i, v_j), & \text{se } i \neq j \text{ e } (v_i, v_j) \in E \\ \infty, & \text{se } i \neq j \text{ e } (v_i, v_j) \notin E \end{cases}$$

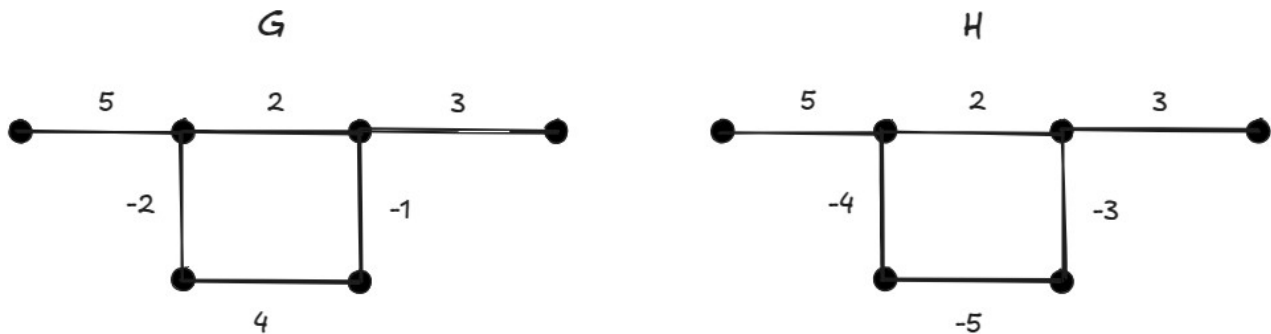
Então, com base na discussão anterior e utilizando programação dinâmica com uma abordagem bottom-up, podemos escrever:

$$d_{ij}^{(k)} = \begin{cases} w_{ij}, & \text{se } k=0 \\ \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}, & \text{se } k \geq 1 \end{cases}$$

Como para qualquer caminho P , todos os vértices intermediários pertencem ao conjunto $\{1, 2, \dots, n\}$, a matriz $D^{(n)} = d_{ij}^{(n)}$ fornece a resposta desejada.

```
Floyd_Warshall(W) {
    n = W.rows
    D(0) = W
    for k = 1 to n {
        Seja D(k) = (dij(k)) uma nova matriz
        for i = 1 to n {
            for j = 1 to n
                dij(k) = min{dij(k-1), dik(k-1) + dkj(k-1)}
        }
    }
    return D(n)
}
```

Uma vantagem do algoritmo de Floyd-Warshall é que ele funciona mesmo na presença de arestas com pesos negativos! Porém, ele não aceita ciclos negativos. A figura a seguir ilustra a diferença entre arestas com pesos negativos e ciclos negativos.



O grafo G possui arestas negativas enquanto que o grafo H possui um ciclo negativo. Note que no grafo H quanto mais voltas dermos no ciclo em questão, menor será o caminho. Nesse caso, nem o algoritmo de Floyd-Warshall funciona corretamente.

Construindo o menor caminho

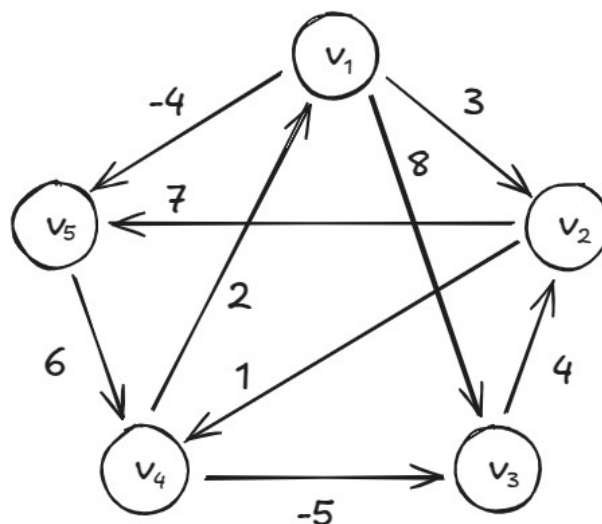
Definimos Π como sendo a matriz de predecessores. Se $k = 0$, um caminho mínimo entre i e j não possui vértices intermediários:

$$\pi_{ij}^{(0)} = \begin{cases} NIL, & \text{se } i = j \text{ ou } w_{ij} = \infty \\ i, & \text{se } i \neq j \text{ e } w_{ij} < \infty \end{cases}$$

Se $k \geq 1$, temos:

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)}, & \text{se } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)}, & \text{se } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$

Note que se $d_{ik}^{(k-1)} + d_{kj}^{(k-1)} < d_{ij}^{(k-1)}$, então significa que passar pelo vértice k diminui o caminho mínimo, então devemos alterar o predecessor. A seguir mostramos um exemplo ilustrativo. Considere o grafo direcionado a seguir:



Então, devemos iniciar as matrizes $D^{(0)}$ e $\Pi^{(0)}$ como:

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(0)} = \begin{pmatrix} \text{nil} & 1 & 1 & \text{nil} & 1 \\ \text{nil} & \text{nil} & \text{nil} & 2 & 2 \\ \text{nil} & 3 & \text{nil} & \text{nil} & \text{nil} \\ 4 & \text{nil} & 4 & \text{nil} & \text{nil} \\ \text{nil} & \text{nil} & \text{nil} & 5 & \text{nil} \end{pmatrix}$$

=> **Primeira iteração (k = 1):**

Vale a pena passar pelo v_1 para ir de v_1 a v_1 ? Não!

$$d_{11}^{(1)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{11}^{(0)}, d_{11}^{(0)} + d_{11}^{(0)} \} = \min \{ 0, 0 \} = 0$$

$$\pi_{11}^{(1)} = \pi_{11}^{(0)} = \text{nil}$$

Vale a pena passar pelo v_1 para ir de v_1 a v_2 ? Não!

$$d_{12}^{(1)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{12}^{(0)}, d_{11}^{(0)} + d_{12}^{(0)} \} = \min \{ 3, 0+3 \} = 3$$

$$\pi_{12}^{(1)} = \pi_{12}^{(0)} = 1$$

Vale a pena passar pelo v_1 para ir de v_1 a v_3 ? Não!

$$d_{13}^{(1)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{13}^{(0)}, d_{11}^{(0)} + d_{13}^{(0)} \} = \min \{ 8, 0+8 \} = 8$$

$$\pi_{13}^{(1)} = \pi_{13}^{(0)} = 1$$

Vale a pena passar pelo v_1 para ir de v_1 a v_4 ? Não!

$$d_{14}^{(1)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{14}^{(0)}, d_{11}^{(0)} + d_{14}^{(0)} \} = \min \{ \infty, 0+\infty \} = \infty$$

$$\pi_{14}^{(1)} = \pi_{14}^{(0)} = \text{nil}$$

Vale a pena passar pelo v_1 para ir de v_1 a v_5 ? Não!

$$d_{15}^{(1)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{15}^{(0)}, d_{11}^{(0)} + d_{15}^{(0)} \} = \min \{ -4, 0+(-4) \} = -4$$

$$\pi_{15}^{(1)} = \pi_{15}^{(0)} = 1$$

Vale a pena passar pelo v_1 para ir de v_2 a v_1 ? Não!

$$d_{21}^{(1)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{21}^{(0)}, d_{21}^{(0)} + d_{11}^{(0)} \} = \min \{ \infty, \infty+0 \} = \infty$$

$$\pi_{21}^{(1)} = \pi_{21}^{(0)} = \text{nil}$$

Não irá ocorrer modificações até atingirmos $d_{42}^{(1)}$, que será atualizado:

Vale a pena passar pelo v_1 para ir de v_4 a v_2 ? SIM!

$$d_{42}^{(1)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{42}^{(0)}, d_{41}^{(0)} + d_{12}^{(0)} \} = \min \{ \infty, 2+3 \} = 5$$

$$\pi_{42}^{(1)} = \pi_{kj}^{(0)} = \pi_{12}^{(0)} = 1$$

De agora em diante, mostraremos apenas as iterações em que a matriz D é atualizada.

Vale a pena passar pelo v_1 para ir de v_4 a v_5 ? SIM!

$$d_{45}^{(1)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{45}^{(0)}, d_{41}^{(0)} + d_{15}^{(0)} \} = \min \{ \infty, 2 + (-4) \} = -2$$

$$\pi_{45}^{(1)} = \pi_{kj}^{(0)} = \pi_{15}^{(0)} = 1$$

Ao final da 1ª iteração, temos as seguintes matrizes:

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \pi^{(1)} = \begin{pmatrix} \text{nil} & 1 & 1 & \text{nil} & 1 \\ \text{nil} & \text{nil} & \text{nil} & 2 & 2 \\ \text{nil} & 3 & \text{nil} & \text{nil} & \text{nil} \\ 4 & 1 & 4 & \text{nil} & 1 \\ \text{nil} & \text{nil} & \text{nil} & 5 & \text{nil} \end{pmatrix}$$

=> **Segunda iteração (k = 2):** apenas as atualizações serão mostradas.

Vale a pena passar pelo v_2 para ir de v_1 a v_4 ? SIM!

$$d_{14}^{(2)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{14}^{(1)}, d_{12}^{(1)} + d_{24}^{(1)} \} = \min \{ \infty, 3 + 1 \} = 4$$

$$\pi_{14}^{(2)} = \pi_{kj}^{(1)} = \pi_{25}^{(1)} = 2$$

Vale a pena passar pelo v_2 para ir de v_3 a v_4 ? SIM!

$$d_{34}^{(2)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{34}^{(1)}, d_{32}^{(1)} + d_{24}^{(1)} \} = \min \{ \infty, 4 + 1 \} = 5$$

$$\pi_{34}^{(2)} = \pi_{kj}^{(1)} = \pi_{24}^{(1)} = 2$$

Vale a pena passar pelo v_2 para ir de v_3 a v_5 ? SIM!

$$d_{35}^{(2)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{35}^{(1)}, d_{32}^{(1)} + d_{25}^{(1)} \} = \min \{ \infty, 4 + 7 \} = 11$$

$$\pi_{35}^{(2)} = \pi_{kj}^{(1)} = \pi_{25}^{(1)} = 2$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \pi^{(2)} = \begin{pmatrix} \text{nil} & 1 & 1 & 2 & 1 \\ \text{nil} & \text{nil} & \text{nil} & 2 & 2 \\ \text{nil} & 3 & \text{nil} & 2 & 2 \\ 4 & 1 & 4 & \text{nil} & 1 \\ \text{nil} & \text{nil} & \text{nil} & 5 & \text{nil} \end{pmatrix}$$

=> **Terceira iteração (k = 3):** apenas as atualizações serão mostradas.

Vale a pena passar pelo v_3 para ir de v_1 a v_4 ? SIM!

$$d_{42}^{(3)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{42}^{(2)}, d_{43}^{(2)} + d_{32}^{(2)} \} = \min \{ 5, -5 + 4 \} = -1$$

$$\pi_{42}^{(3)} = \pi_{kj}^{(2)} = \pi_{32}^{(2)} = 3$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(3)} = \begin{pmatrix} \text{nil} & 1 & 1 & 2 & 1 \\ \text{nil} & \text{nil} & \text{nil} & 2 & 2 \\ \text{nil} & 3 & \text{nil} & 2 & 2 \\ 4 & 3 & 4 & \text{nil} & 1 \\ \text{nil} & \text{nil} & \text{nil} & 5 & \text{nil} \end{pmatrix}$$

=> **Quarta iteração (k = 4):** apenas as atualizações serão mostradas.

Vale a pena passar pelo v_4 para ir de v_1 a v_3 ? SIM!

$$d_{13}^{(4)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{13}^{(3)}, d_{14}^{(3)} + d_{43}^{(3)} \} = \min \{ 8, 4 + (-5) \} = -1$$

$$\pi_{13}^{(4)} = \pi_{kj}^{(3)} = \pi_{43}^{(3)} = 4$$

Vale a pena passar pelo v_4 para ir de v_2 a v_1 ? SIM!

$$d_{21}^{(4)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{21}^{(3)}, d_{24}^{(3)} + d_{41}^{(3)} \} = \min \{ \infty, 1 + 2 \} = 3$$

$$\pi_{21}^{(4)} = \pi_{kj}^{(3)} = \pi_{41}^{(3)} = 4$$

Vale a pena passar pelo v_4 para ir de v_2 a v_3 ? SIM!

$$d_{23}^{(4)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{23}^{(3)}, d_{24}^{(3)} + d_{43}^{(3)} \} = \min \{ \infty, 1 + (-5) \} = -4$$

$$\pi_{23}^{(4)} = \pi_{kj}^{(3)} = \pi_{43}^{(3)} = 4$$

Vale a pena passar pelo v_4 para ir de v_2 a v_5 ? SIM!

$$d_{25}^{(4)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{25}^{(3)}, d_{24}^{(3)} + d_{45}^{(3)} \} = \min \{ 7, 1 + (-2) \} = -1$$

$$\pi_{25}^{(4)} = \pi_{kj}^{(3)} = \pi_{45}^{(3)} = 1$$

Vale a pena passar pelo v_4 para ir de v_3 a v_1 ? SIM!

$$d_{31}^{(4)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{31}^{(3)}, d_{34}^{(3)} + d_{41}^{(3)} \} = \min \{ \infty, 5 + 2 \} = 7$$

$$\pi_{31}^{(4)} = \pi_{kj}^{(3)} = \pi_{41}^{(3)} = 4$$

Vale a pena passar pelo v_4 para ir de v_3 a v_5 ? SIM!

$$d_{35}^{(4)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{35}^{(3)}, d_{34}^{(3)} + d_{45}^{(3)} \} = \min \{ 11, 5 + (-2) \} = 3$$

$$\pi_{35}^{(4)} = \pi_{kj}^{(3)} = \pi_{45}^{(3)} = 1$$

Vale a pena passar pelo v_4 para ir de v_5 a v_1 ? SIM!

$$d_{51}^{(4)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{51}^{(3)}, d_{54}^{(3)} + d_{41}^{(3)} \} = \min \{ \infty, 6+2 \} = 8$$

$$\pi_{51}^{(4)} = \pi_{kj}^{(3)} = \pi_{41}^{(3)} = 4$$

Vale a pena passar pelo v_4 para ir de v_5 a v_2 ? SIM!

$$d_{52}^{(4)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{52}^{(3)}, d_{54}^{(3)} + d_{42}^{(3)} \} = \min \{ \infty, 6+(-1) \} = 5$$

$$\pi_{52}^{(4)} = \pi_{kj}^{(3)} = \pi_{42}^{(3)} = 3$$

Vale a pena passar pelo v_4 para ir de v_5 a v_3 ? SIM!

$$d_{53}^{(4)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{53}^{(3)}, d_{54}^{(3)} + d_{43}^{(3)} \} = \min \{ \infty, 6+(-5) \} = 1$$

$$\pi_{53}^{(4)} = \pi_{kj}^{(3)} = \pi_{43}^{(3)} = 4$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(4)} = \begin{pmatrix} \text{nil} & 1 & 4 & 2 & 1 \\ 4 & \text{nil} & 4 & 2 & 1 \\ 4 & 3 & \text{nil} & 2 & 1 \\ 4 & 3 & 4 & \text{nil} & 1 \\ 4 & 3 & 4 & 5 & \text{nil} \end{pmatrix}$$

=> **Quinta iteração (k = 5):** apenas as atualizações serão mostradas.

Vale a pena passar pelo v_5 para ir de v_1 a v_2 ? SIM!

$$d_{12}^{(5)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{12}^{(4)}, d_{15}^{(4)} + d_{52}^{(4)} \} = \min \{ 3, -4+5 \} = 1$$

$$\pi_{12}^{(5)} = \pi_{kj}^{(4)} = \pi_{52}^{(4)} = 3$$

Vale a pena passar pelo v_5 para ir de v_1 a v_3 ? SIM!

$$d_{13}^{(5)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{13}^{(4)}, d_{15}^{(4)} + d_{53}^{(4)} \} = \min \{ -1, -4+1 \} = -3$$

$$\pi_{13}^{(5)} = \pi_{kj}^{(4)} = \pi_{53}^{(4)} = 4$$

Vale a pena passar pelo v_5 para ir de v_1 a v_4 ? SIM!

$$d_{14}^{(5)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} = \min \{ d_{14}^{(4)}, d_{15}^{(4)} + d_{54}^{(4)} \} = \min \{ 4, -4+6 \} = 2$$

$$\pi_{14}^{(5)} = \pi_{kj}^{(4)} = \pi_{54}^{(4)} = 5$$

Portanto, após as 5 iterações finalizadas, temos o resultado final, dado pelas matrizes abaixo.

$$^{(5)} \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(5)} = \begin{pmatrix} \text{nil} & 3 & 5 & 4 & 1 \\ 4 & \text{nil} & 4 & 2 & 1 \\ 4 & 3 & \text{nil} & 2 & 1 \\ 4 & 3 & 4 & \text{nil} & 1 \\ 4 & 3 & 4 & 5 & \text{nil} \end{pmatrix}$$

Ao final do algoritmo, suponha que desejamos saber o menor caminho de v_2 até v_5 . Como fazer?

Como $\pi_{25}^{(5)} = 1$, sabemos que o 1 é o predecessor do 5 neste caminho.

Depois, como $\pi_{21}^{(5)} = 4$, sabemos que o 4 é o predecessor do 1 neste caminho.

Por fim, como $\pi_{24}^{(5)} = 2$, sabemos que o 2 é o predecessor do 4 neste caminho.

Portanto, o caminho mínimo entre 2 e 5 é $P = v_2 v_4 v_1 v_5$

Outros exemplos de problemas interessantes na computação que envolvem programação dinâmica são a multiplicação de matrizes em cadeia (Matrix-chain multiplication) e o problema da subsequência em comum mais longa (Longest common subsequence).

“The ultimate test of your knowledge is your capacity to convey it to another.”
— Professor Richard Feynman

Algoritmos Gulosos (Greedy algorithms)

Um algoritmo guloso/ganancioso sempre faz a escolha que parece ser a melhor naquele momento.

A estratégia consiste em fazer escolhas localmente ótimas na expectativa de que elas nos levem à solução ótima global.

Nem sempre algoritmos gulosos obtêm soluções ótimas, mas se o problema em questão exibe subestrutura ótima, em geral, a abordagem gulosa tende a fornecer soluções ótimas.

É uma estratégia simples, mas bastante eficiente em uma variedade de problemas, como:

- Árvore geradoras mínimas (MST's)
- Código de Huffman
- Fluxo em redes (algoritmo de Ford-Fulkerson)
- Emparelhamentos estáveis (algoritmo de Gale-Shapley)
- Programação linear (algoritmo Simplex)

Algoritmos gulosos x Programação dinâmica

Apesar de parecerem similares, estratégias gulosas não são equivalentes à programação dinâmica.

Estratégia gulosa	Programação dinâmica
Não garante a obtenção da solução ótima	Pela subestrutura dos subproblemas, é garantido que teremos a solução ótima
Usa a melhor escolha local	Usa uma fórmula de recorrência para calcular os subproblemas
Não requer tabela para memorização	Memoriza os resultados dos subproblemas anteriores
Em geral, algoritmos gulosos são mais eficientes	Em geral, algoritmos com programação dinâmica são mais lentos que algoritmos gulosos
Algoritmos gulosos calculam a solução de uma maneira progressiva, nunca olhando para trás ou revistando escolhas anteriores	Algoritmos de programação dinâmica podem ser tanto top-down quanto bottom-up
Exemplo: Problema da mochila fracionário	Exemplo: Problema da mochila binário

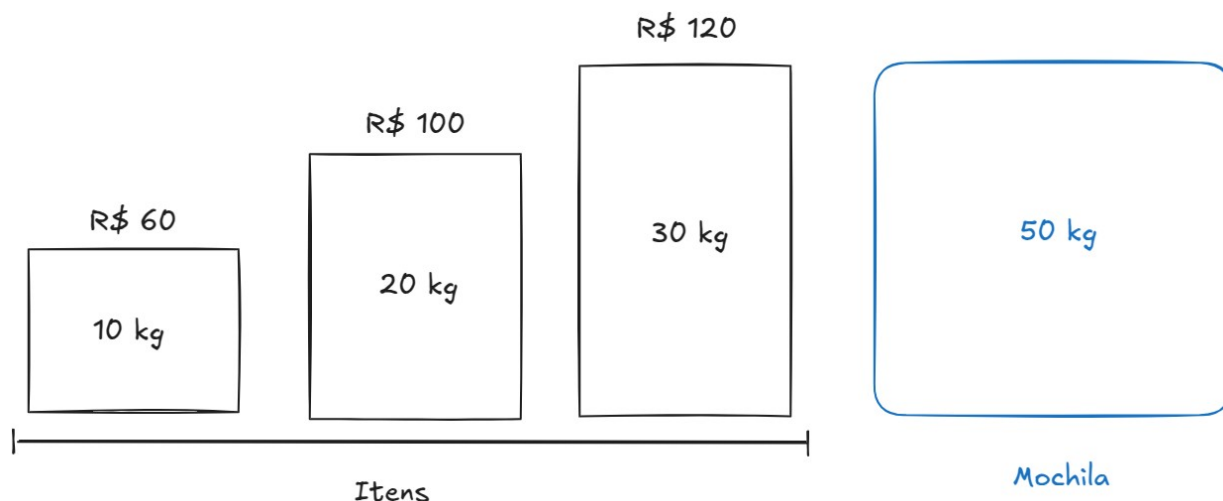
Para ilustrar a diferença, veremos duas variações do problema da mochila: a versão binária e a versão fracionária.

Suponha uma mochila com capacidade de W kg's e uma lista com n itens: i -ésimo item vale v_i reais e pesa w_i kg's

Deseja-se maximizar o valor a carregar, sujeito a restrição de peso total W da mochila.

Na versão fracionária, é possível carregar uma fração dos itens ($1/2$, $1/3$, $2/3$,...), já na binária ou carrega todo objeto ou não carrega nada.

Estratégia gulosa: escolher itens com maior valor por kg, ou seja, itens com maior razão $\frac{v_i}{w_i}$



Note que temos os seguintes valores para o peso por quilo:

$$\frac{v_A}{w_A} = 6 \quad \frac{v_B}{w_B} = 5 \quad \frac{v_C}{w_C} = 4$$

=> **Binário:** de acordo com a estratégia gulosa, devemos escolher A + B

Essa escolha resulta em $60 + 100 = 160$

Mas note que não é a escolha ótima, pois se escolhermos B + C, resultaria em $100 + 120 = 220$

=> **Fracionário:** aqui podemos escolher A + B + $20/30$ C

Essa escolha resulta em $60 + 100 + 2/3 \cdot 120 = 160 + 80 = 240$

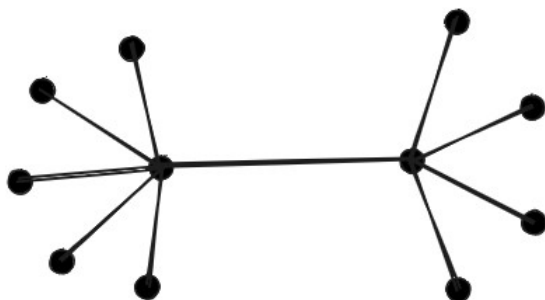
Solução ótima!

Portanto, a estratégia gulosa fornece a solução ótima apenas no caso fracionário.

Árvores geradoras mínimas

Árvores são grafos especiais com diversas propriedades únicas. Devido a essas propriedades são extremamente importantes na resolução de vários tipos de problemas práticos. Veremos ao longo do curso que vários problemas que estudaremos se resumem a: dado um grafo G , extrair uma árvore T a partir de G , de modo que T satisfaça uma certa propriedade (como por exemplo, mínima profundidade, máxima profundidade, mínimo peso, mínimos caminhos, etc).

Def: Um grafo $G = (V, E)$ é uma árvore se G é acíclico e conexo.



Teoremas e propriedades

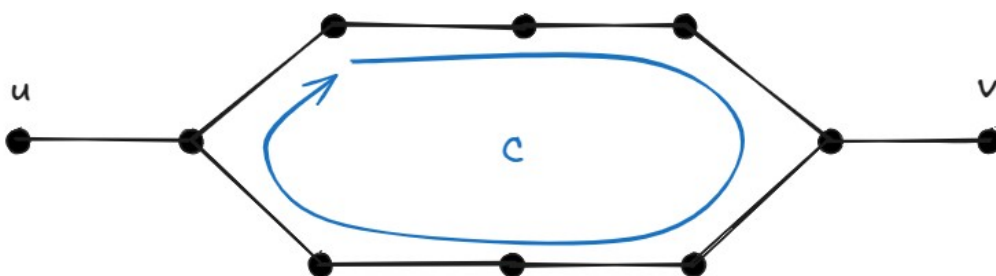
Teorema: G é uma árvore $\Leftrightarrow \exists$ um único caminho entre quaisquer 2 vértices $u, v \in V$

$\Rightarrow$ **Parte 1:** (ida) $p \rightarrow q = \neg q \rightarrow \neg p$

$\nexists$ um único caminho entre quaisquer $u, v \rightarrow G$ não é uma árvore

a) Pode existir um par u, v tal que $\nexists$ caminho (zero caminhos). Isso implica em G desconexo, o que implica que G não é uma árvore

b) Pode existir um par u, v tal que $\exists$ mais de um caminho.



Porém, neste caso temos a formação de um ciclo e portanto G não pode ser árvore.

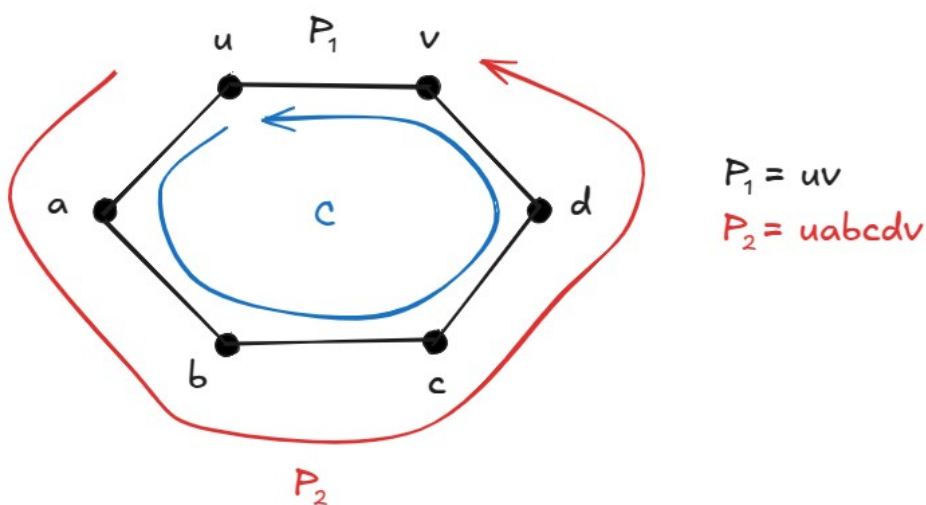
$\Rightarrow$ **Parte 2:** (volta) $q \rightarrow p = \neg p \rightarrow \neg q$

G não é árvore $\rightarrow \nexists$ único caminho entre quaisquer u, v

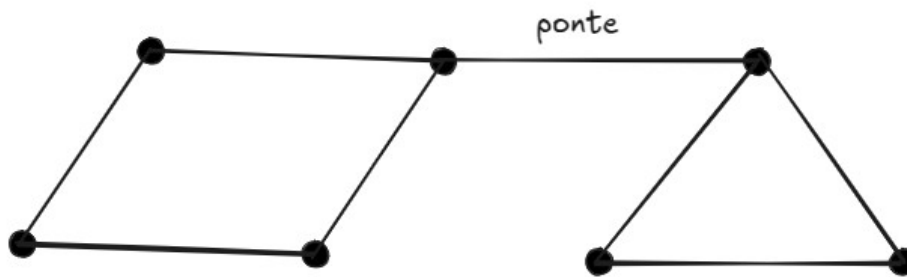
Para G não ser árvore, G deve ser desconexo ou conter um ciclo.

a) Note que no primeiro caso existe um par u, v tal que não há caminho entre eles.

b) Note também que no segundo caso existem 2 caminhos entre u e v , conforme ilustra a figura



Def: Uma aresta $e \in E$ é ponte se $G - e$ é desconexo. Ou seja, a remoção de uma aresta ponte desconecta o grafo

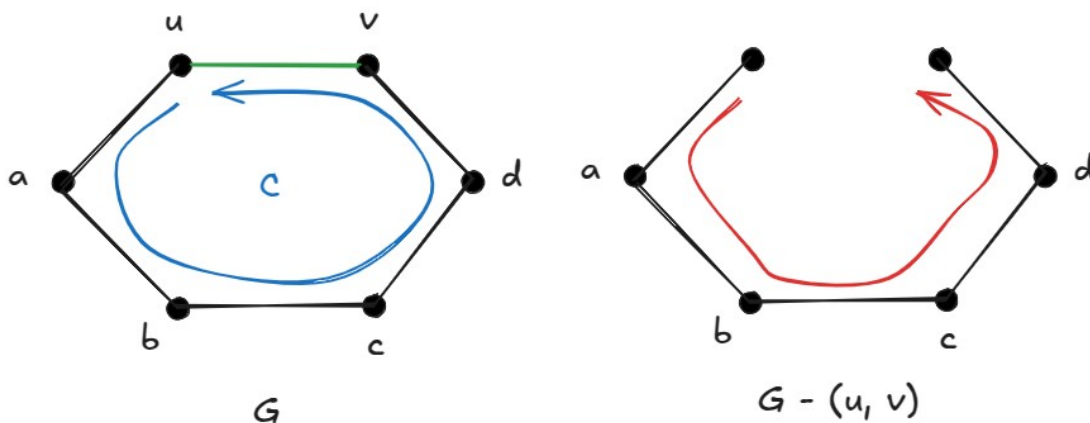


Como identificar arestas ponte?

Teorema: Uma aresta $e \in E$ é ponte $\Leftrightarrow$ aresta não pertence a um ciclo C

=> **Parte 1:** (ida) $p \rightarrow q = \neg q \rightarrow \neg p$

$e \in C \rightarrow$ aresta não é ponte



Como aresta pertence a um ciclo C , há 2 caminhos entre u e v . Logo a remoção da aresta $e = (u, v)$ não impede que o grafo seja conexo, ou seja, $G - e$ ainda é conexo. Portanto, e não é ponte

=> **Parte 2:** (volta) $q \rightarrow p = \neg p \rightarrow \neg q$

aresta não é ponte $\rightarrow e \in C$

Se aresta não é ponte então $G - e$ ainda é conexo. Se isso ocorre, deve-se ao fato de que em $G - e$ ainda existe um caminho entre u e v que não passa por e . Logo, em G existem 2 caminhos, o que nos leva a conclusão de que a união entre os 2 caminhos gera um ciclo C .

Teorema: G é uma árvore $\Leftrightarrow$ Toda aresta é ponte

=> **Parte 1:** (ida) $p \rightarrow q = \neg q \rightarrow \neg p$

$\exists$ aresta não ponte $\rightarrow G$ não é árvore

A existência de uma aresta não ponte implica na existência de ciclo. A presença de um ciclo C faz com que G não seja um árvore

=> **Parte 2:** (volta) $q \rightarrow p = \neg p \rightarrow \neg q$

G não é árvore $\rightarrow \exists$ aresta não ponte

Para G não ser uma árvore, deve existir um ciclo em G . Logo, todas as arestas pertencentes ao ciclo não são pontes.

Teorema: Se $G = (V, E)$ é uma árvore com $|V| = n$ então $|E| = n - 1$

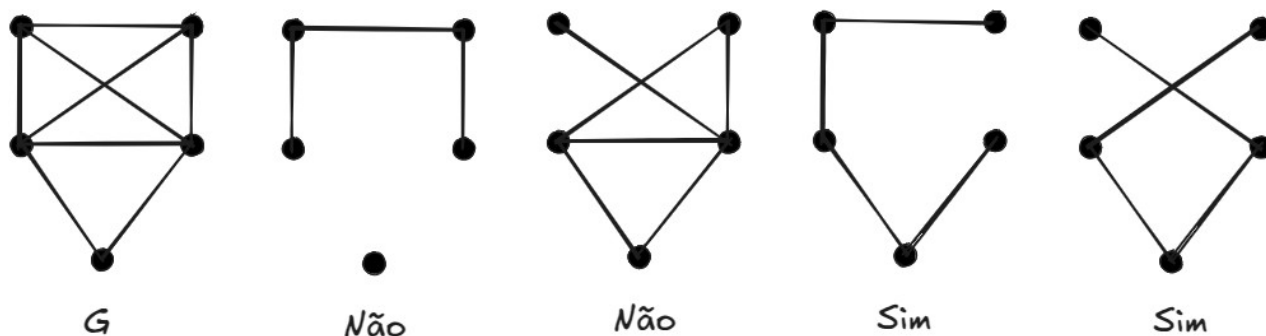
Prova por contradição:

1. Suponha uma árvore de n vértices com $m < n - 1$ arestas.
2. Isso implica em um grafo desconexo. Portanto, não pode ser uma árvore.
3. Suponha uma árvore de n vértices com $m > n - 1$ arestas.
4. Isso implica em um grafo que contém ao menos 1 ciclo C . Portanto, também não pode ser uma árvore.

Logo, toda árvore de n vértices deve ter $m = n - 1$ arestas.

Def: Árvore geradora (spanning tree)

Seja $G = (V, E)$ um grafo. Dizemos que $T = (V, E_T)$ é uma árvore geradora de G se T é um subgrafo de G que é uma árvore (ou seja tem que conectar todos os vértices)



O problema da árvore geradora mínima (MST)

Dentre todas as árvores geradoras de G , obter aquela de menor peso.

Def: Dado $G = (V, E, w)$, onde $w: E \rightarrow \mathbb{R}^+$ (peso da aresta e), obter a árvore geradora T que minimiza o peso:

$$w(T) = \sum_{e \in T} w(e) \quad (\text{soma dos pesos das arestas que compõem a árvore})$$

Por exemplo, deseja-se conectar os bairros da cidade com fibra ótica. Qual é a interligação que minimiza o custo?

Estratégia gulosa: abordagem iterativa em que a cada passo devemos escolher a aresta de menor custo que seja segura (uma aresta é segura se, ao ser adicionada em T , T continua sendo uma árvore)

A seguir apresentamos uma função genérica para o problema da árvore geradora mínima.

```

Generic_MST(G, w) {
    T =  $\emptyset$ 
    while T não for uma árvore geradora {
        encontre uma aresta segura (u, v)
        T = T  $\cup$  {(u, v)}
    }
    return T
}

```

A pergunta que surge é: como podemos determinar se uma aresta é segura? Veremos a seguir dois algoritmos que utilizam critérios diferentes para definir o que são arestas seguras: os algoritmos de Kruskal e Prim.

O algoritmo de Kruskal

Objetivo: a cada passo escolha a aresta (u, v) de menor custo que não forme um ciclo em $T + (u, v)$

Ideia: iniciar adicionando cada vértice de G como raiz de uma árvore e a cada passo adicionar a aresta de menor custo com extremidades em árvores distintas.

Para isso, iremos utilizar 3 primitivas básicas:

1. Make_Set(v): cria uma árvore contendo um único vértice v (raiz)

```

Make_Set(v) {
    v.p = v           # pai do vértice v é ele mesmo (raiz)
    v.rank = 0        # altura da árvore (nível)
}

```

2. Find_Set(v): retorna qual é a árvore que o vértice v pertence (com path compression)

```

Find_Set(v) {
    if v  $\neq$  v.p       # se não é raiz
        v.p = Find_Set(v.p) # recursão: v se torna o pai
    return v.p         # retorna raiz da árvore de v
}

```

3. Union(u, v): faz a fusão das raízes das árvores de u e de v, criando uma única árvore

```

Union(u, v) {
    Link(Find_Set(u), Find_Set(v))
}

```

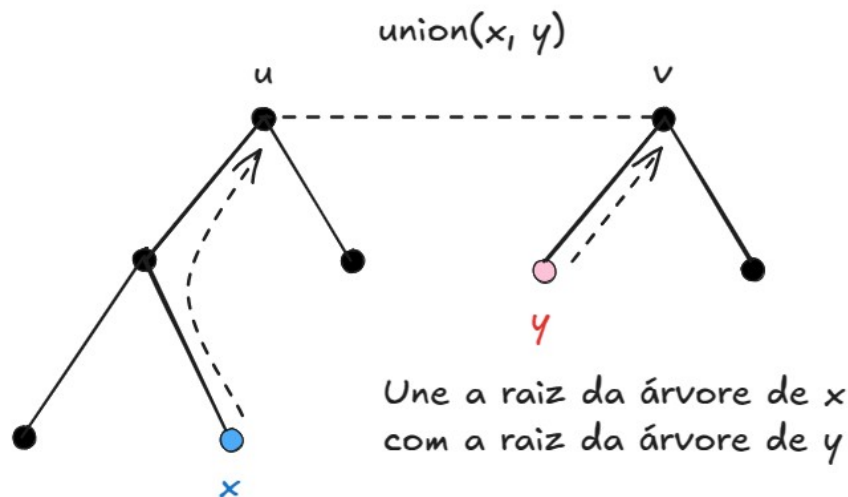
Função auxiliar para fundir as árvores de u e de v

```

Link(u, v) {
    if u.rank > v.rank
        v.p = u
    else {
        u.p = v
        if u.rank == v.rank
            v.rank = v.rank + 1
    }
}

```

A figura a seguir ilustra o processo.



Ao realizar $\text{Union}(x, y)$, primeiro encontramos as raízes das árvores de x e de y , que nesse caso são u e v respectivamente. Então, a função auxiliar Link , realiza a fusão dessas duas árvores, criando uma única. Porém, a pergunta é: quem será pai de quem? Note que nesse caso u será pai de v , pois a altura da árvore de raiz u é maior! Assim, não preciso alterar a altura dela. A maior árvore domina a menor. Somente quando as alturas das duas árvores são iguais é que precisamos incrementar a altura (rank) em uma unidade. A seguir apresentamos o algoritmo de Kruskal para a obtenção de uma árvore geradora mínima.

```

MST_Kruskal( $G, w$ ) {
     $T = \emptyset$ 
    for each  $v \in V$ 
        Make_Set( $v$ )
    Crie uma lista de todas as arestas  $e \in E$ 
    Ordene a lista pelos pesos  $w(e)$  em ordem crescente
    for each  $e = (u, v)$  na lista ordenada {
        if Find_Set( $u$ )  $\neq$  Find_Set( $v$ ) {
             $T = T \cup \{(u, v)\}$ 
            Union( $u, v$ )
        }
    }
}

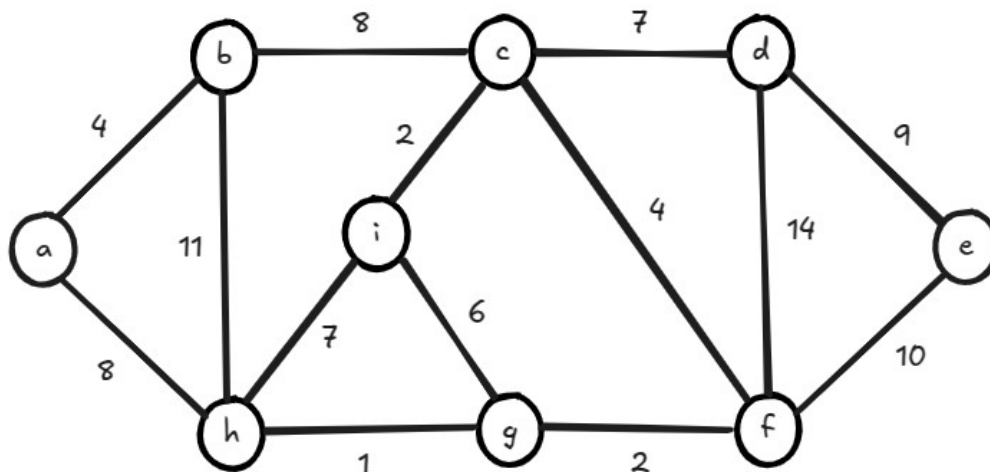
```

Note que trata-se de um algoritmo guloso pois ele sempre tenta incluir a aresta de menor peso a cada iteração. A seguir iremos realizar um trace do algoritmo (simulação passo a passo). Para isso iremos considerar a seguinte notação:

E^- : conjunto das arestas de peso mínimo não seguras ($\text{find_set}(u) = \text{find_set}(v)$)
 E^+ : conjunto das arestas de peso mínimo seguras ($\text{find_set}(u) \neq \text{find_set}(v)$)
 e_k : aresta escolhida no passo k

Ex: Suponha que os vértices representem bairros e as arestas com pesos os custos de interligação desses bairros com fibra ótica. Deseja saber qual é o custo mínimo de interligar todos os bairros.

Para isso, devemos encontrar a MST do grafo G , o que pode ser feito com o algoritmo de Kruskal.



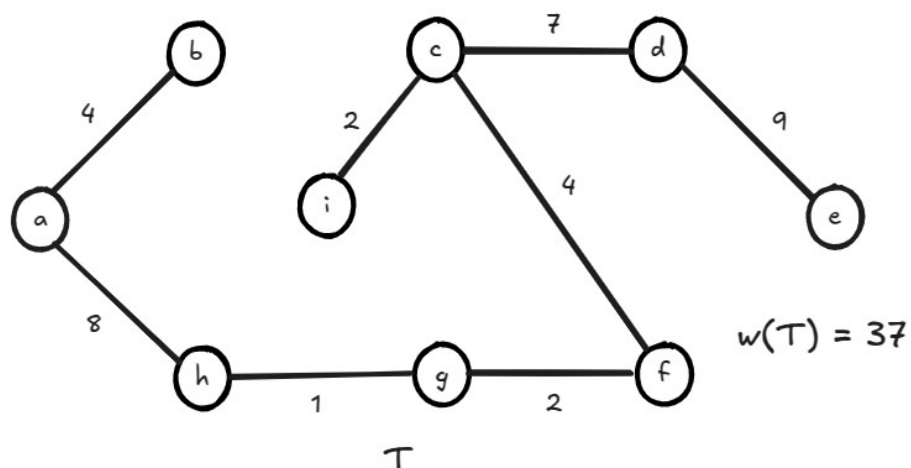
A lista das arestas ordenadas por peso é:

$E_w = [1, 2, 2, 4, 4, 6, 7, 7, 8, 8, 9, 10, 11, 14]$

$E = \{(g, h), (c, i), (f, g), (a, b), (c, f), (g, i), (c, d), (a, h), (b, c), (d, e), (e, f), (b, h), (d, f)\}$

k	E^-	E^+	e_k
1	-	$\{(g,h)\}$	(g,h)
2	-	$\{(c,i), (f,g)\}$	(c,i)
3	-	$\{(g,f)\}$	(g,f)
4	-	$\{(a,b), (c,f)\}$	(a,b)
5	-	$\{(c,f)\}$	(c,f)
6	$\{(g,i)\}$	-	-
7	$\{(h,i)\}$	$\{(c,d)\}$	(c,d)
8	-	$\{(a,h), (b,c)\}$	(a,h)
9	$\{(b,c)\}$	-	-
10	-	$\{(d,e)\}$	(d,e)

A MST resultante do algoritmo de Kruskal é ilustrada a seguir.



A seguir iremos demonstrar a otimalidade do algoritmo de Kruskal, ou seja, que o algoritmo de Kruskal sempre retorna uma MST de G .

Teorema: Toda árvore T gerado pelo algoritmo de Kruskal é uma MST de G

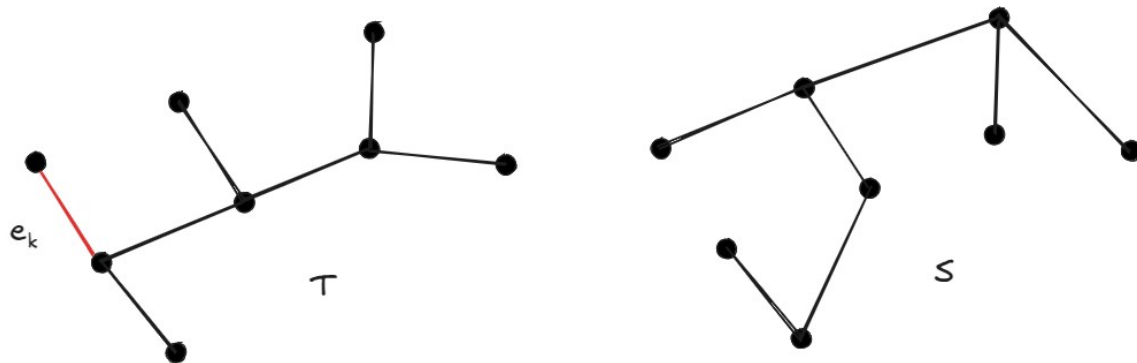
Esse resultado garante que o algoritmo sempre funciona e é ótimo (retorna sempre a solução ótima)

Prova por contradição

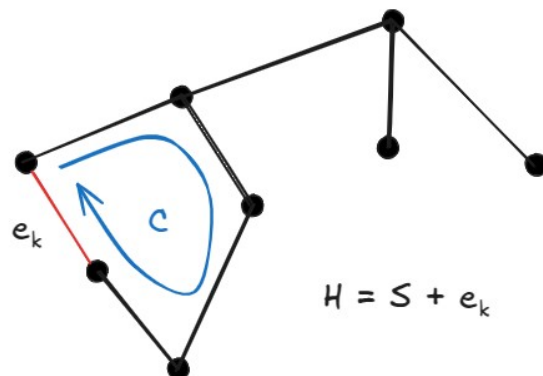
Seja T é a árvore retornada por Kruskal.

1. Suponha que $\exists S \neq T$ tal que $w(S) < w(T)$ (S é uma árvore)

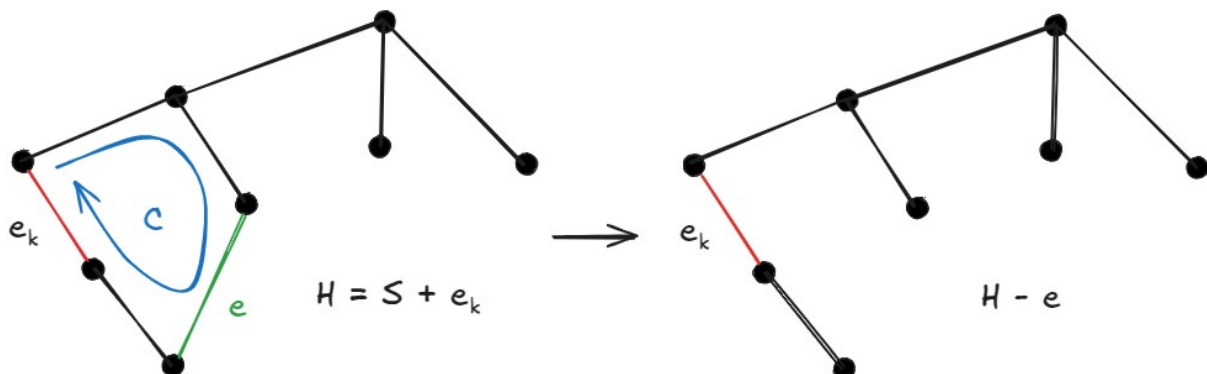
2. Seja $e_k \in T$ a primeira aresta adicionada em T que não está em S (pois árvores são diferentes)



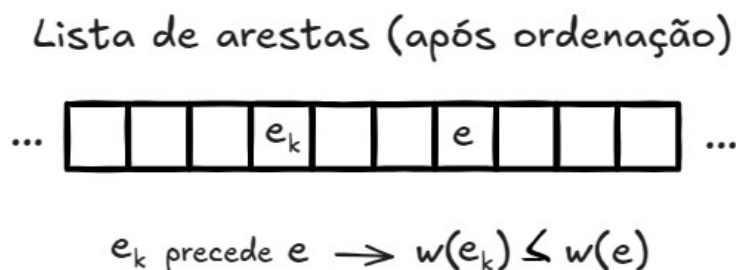
3. Faça $H = (S + e_k)$. Note que H não é mais uma árvore e contém um ciclo



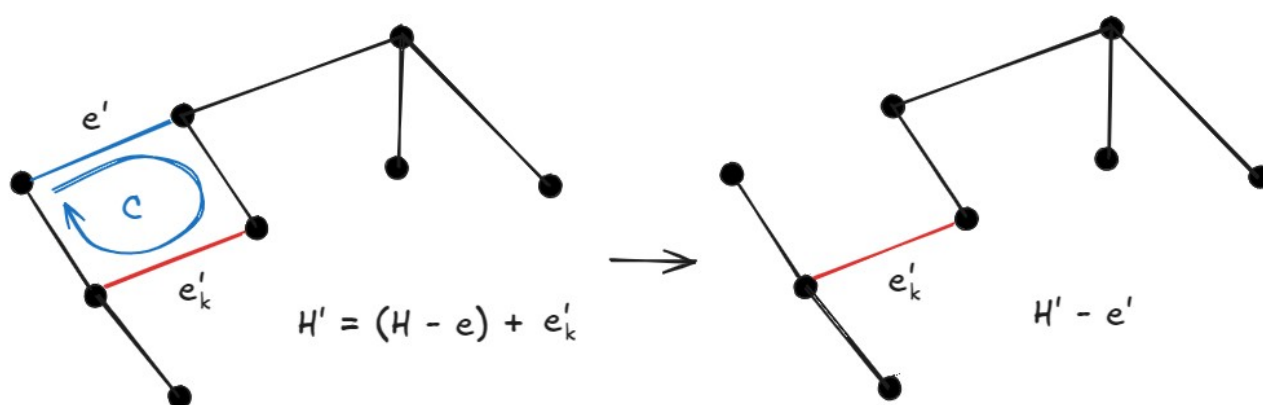
4. Note que no ciclo C , $\exists e \in S$ tal que $e \notin T$ (pois senão C existiria em T). O subgrafo $H - e$ é conexo, possui $n - 1$ arestas e define uma árvore geradora de G .



5. Porém, $w(e_k) \leq w(e)$ e assim $w(H - e) \leq w(S)$ (pois de acordo com Kruskal e_k vem antes de e na lista ordenada de arestas, é garantido pela ordenação)



6. Repetindo o processo usado para gerar $H - e$ a partir de S é possível produzir uma sequência de árvores que se aproximam cada vez mais de T .



Se continuarmos com esse processo, a árvore inicial S será transformada na árvore T , pois:

$$S \rightarrow (H - e) \rightarrow (H' - e') \rightarrow (H'' - e'') \rightarrow \dots \rightarrow T$$

de modo que

$$w(S) \geq w(H - e) \geq w(H' - e') \geq \dots \geq w(T) \quad (\text{contradição a suposição inicial})$$

Portanto, não existe árvore com peso menor que T , mostrando que T tem peso mínimo. (Não há como S ter peso menor que T).

Análise da complexidade

Primeiramente, devemos analisar as complexidades das primitivas básicas.

Pode-se perceber que a função $\text{Make_Set}(v)$ é $O(1)$. Como ela é executada n vezes, temos que no total o custo será $O(n)$.

A ordenação das arestas pode ser realizada com o MergeSort ou QuickSort, que são $O(m \log m)$, onde m denota o número de arestas.

A primitiva $\text{Find_Set}(v)$ deve retornar a raiz da árvore que v pertence. Note que para isso, dependemos da altura da árvore. No caso em que v é um nó folha da árvore binária, $h = \log n$. Portanto, a complexidade da função é $O(\log n)$. Se a árvore não for binária, muda-se apenas a base do logaritmo. Logo, a primitiva $\text{Union}(u, v)$ que faz uso de $\text{Find_Set}(v)$ também é $O(\log n)$.

Note que $\text{Find_Set}(v)$ é executada duas vezes para cada aresta (uma para cada extremidade: u e v). Assim, o custo total é $2m O(\log n)$, o que é $O(m \log n)$.

A primitiva $\text{Union}(u, v)$ é executada somente quando uma aresta é adicionada a árvore. Como uma árvore tem $m = n - 1$ arestas, a complexidade é $O(n \log n)$. Portanto, o custo total do algoritmo de Kruskal é:

$$C = O(n) + O(m \log m) + O(m \log n) + O(n \log n)$$

Podemos observar que o termo dominante é $O(m \log m)$. Como $m < n^2$, temos que:

$$\log m < \log n^2 = 2 \log n = O(\log n)$$

ou seja, a complexidade do algoritmo de Kruskal é $O(m \log n)$.

Algoritmo de Prim

O algoritmo de Prim também utiliza uma abordagem gulosa para a construção da MST.

Ideia: iniciar de uma raiz r e a cada passo adicionar a aresta de menor custo com uma extremidade no conjunto dos vértices visitados e outra extremidade no conjunto dos vértices não visitados.

Definição das variáveis

$\lambda(v)$ ou $v.\text{key}$: menor custo de entrada para o vértice v até o momento

$\pi(v)$ ou $v.\pi$: predecessor do vértice v em T

Q : fila de prioridades (maior prioridade = menor $\lambda(v)$)

Veremos a seguir que esse algoritmo é muito similar ao algoritmo de Dijkstra.

```
MST_Prim(G, w, r) {
    for each v ∈ V {
        λ(v) = ∞
        π(v) = nil
    }
    λ(r) = 0
    # Insere vértices na fila de prioridades Q
    Q = ∅
    for each v ∈ V
        Insert(Q, v)
    while Q ≠ ∅ {
        u = ExtractMin(Q)
        S = S ∪ {u}
        for each v in N(u) {
            if v ∈ Q and w(u,v) < λ(v) { # achou entrada melhor
                λ(v) = w(u,v)
                π(v) = u
                Decrease_Key(Q, v, w(u,v))
            }
        }
    }
}
```

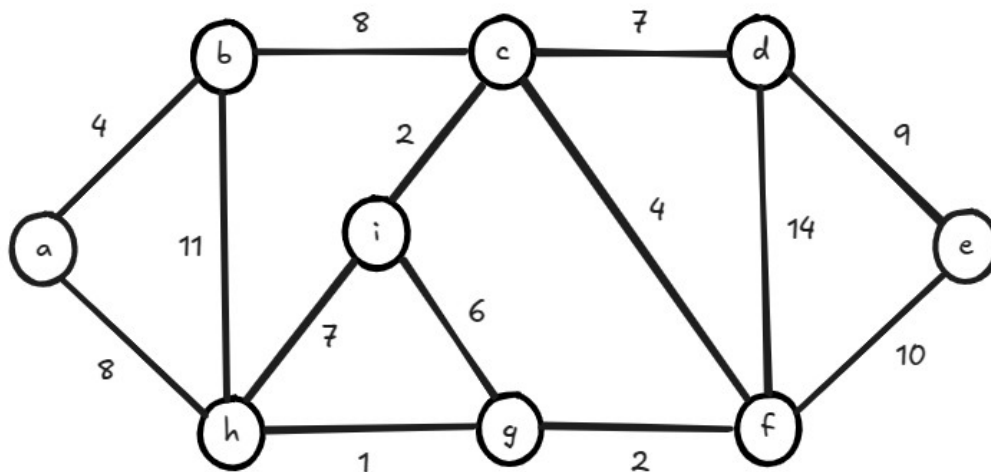
Note que o IF interno no loop FOR é equivalente a:

```

if v ∈ Q {
    λ(v) = min{λ(v), w(u,v)}
    if λ(v) was updated {
        π(v) = u
        Decrease_Key(Q, v, w(u,v))
    }
}

```

A seguir veremos um trace da execução completa do algoritmo de Prim em um simples exemplo.



Fila de prioridades

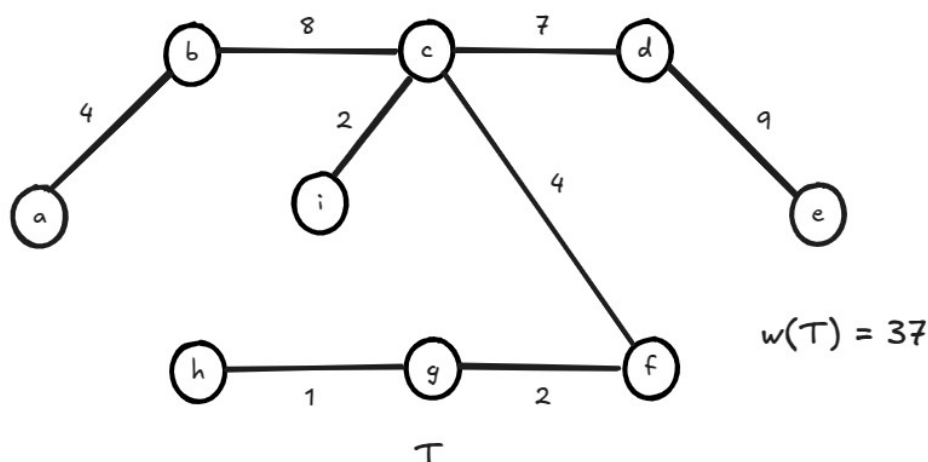
	a	b	c	d	e	f	g	h	i
$\lambda^{(0)}(v)$	0	∞	∞	∞	∞	∞	∞	∞	∞
$\lambda^{(1)}(v)$		4	∞	∞	∞	∞	∞	8	∞
$\lambda^{(2)}(v)$			8	∞	∞	∞	∞	8	∞
$\lambda^{(3)}(v)$				7	∞	4	∞	8	2
$\lambda^{(4)}(v)$				7	∞	4	6	7	
$\lambda^{(5)}(v)$				7	10		2	7	
$\lambda^{(6)}(v)$				7	10			1	
$\lambda^{(7)}(v)$					9				

Ordem de acesso aos vértices

u	$V' = \{v \in N(u) \wedge v \in Q\}$	$\lambda(v), \forall v \in V'$	$\pi(v)$
a	{b, h}	$\lambda(b) = \min\{\lambda(b), w(a,b)\} = \min\{\infty, 4\} = 4$ $\lambda(h) = \min\{\lambda(h), w(a,h)\} = \min\{\infty, 8\} = 8$	$\pi(b) = a$ $\pi(h) = a$
b	{c, h}	$\lambda(c) = \min\{\lambda(c), w(b,c)\} = \min\{\infty, 8\} = 8$ $\lambda(h) = \min\{\lambda(h), w(b,h)\} = \min\{8, 11\} = 8$	$\pi(c) = b$ ---

c	{d, f, i}	$\lambda(d) = \min\{\lambda(d), w(c, d)\} = \min\{\infty, 7\} = 7$ $\lambda(f) = \min\{\lambda(f), w(c, f)\} = \min\{\infty, 4\} = 4$ $\lambda(i) = \min\{\lambda(i), w(c, i)\} = \min\{\infty, 2\} = 2$	$\pi(d) = c$ $\pi(f) = c$ $\pi(i) = c$
i	{h, g}	$\lambda(h) = \min\{\lambda(h), w(i, h)\} = \min\{8, 7\} = 7$ $\lambda(g) = \min\{\lambda(g), w(i, g)\} = \min\{\infty, 6\} = 6$	$\pi(h) = i$ $\pi(g) = i$
f	{d, e, g}	$\lambda(d) = \min\{\lambda(d), w(f, d)\} = \min\{7, 14\} = 7$ $\lambda(e) = \min\{\lambda(e), w(f, e)\} = \min\{\infty, 10\} = 10$ $\lambda(g) = \min\{\lambda(g), w(f, g)\} = \min\{6, 2\} = 2$	--- $\pi(e) = f$ $\pi(g) = f$
g	{h}	$\lambda(h) = \min\{\lambda(h), w(g, h)\} = \min\{7, 1\} = 1$	$\pi(h) = g$
h	$\emptyset$	---	---
d	{e}	$\lambda(e) = \min\{\lambda(e), w(d, e)\} = \min\{10, 9\} = 9$	$\pi(e) = d$
e	$\emptyset$	---	---

Árvore geradora mínima



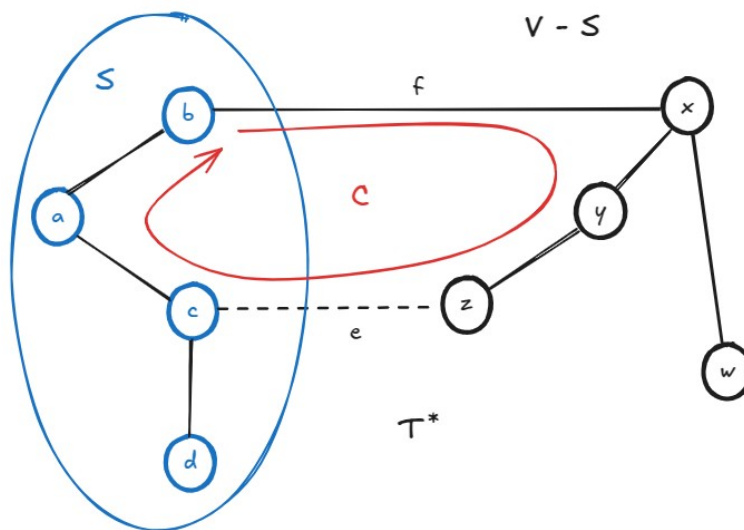
Note que, apesar da MST obtida por Prim ser diferente da MST obtida por Kruskal, ambas possuem o mesmo peso mínimo, $w(T) = 37$. Em geral, a condição para que a MST seja única é que todos os pesos das arestas do grafo $G = (V, E, w)$ sejam distintos, ou seja, não devemos ter arestas com pesos iguais no grafo de entrada.

Mapa de predecessores

v	a	b	c	d	e	f	g	h	i
$\pi(v)$	--	a	b	c	d	c	f	g	c
$\lambda(v)$	0	4	8	7	9	4	2	1	2

A seguir veremos um resultado fundamental para provar a otimalidade do algoritmo de Prim.

Propriedade do corte: Seja $G = (V, E, w)$ um grafo, S um subconjunto qualquer de V e $e \in E$ a aresta de menor custo com exatamente uma extremidade em S . Então, a MST de G contém e .

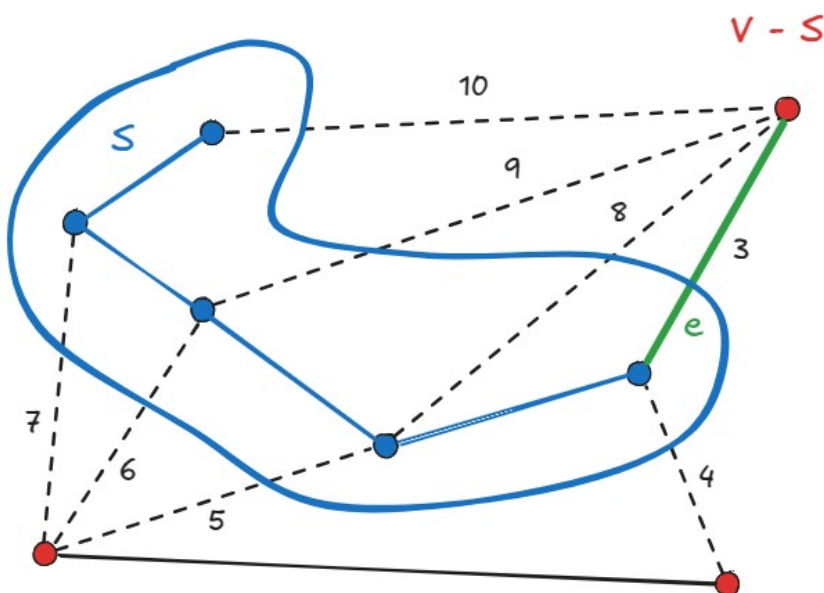


Prova por contradição:

1. Seja T^* uma MST de G .
2. Hipótese: Suponha que $e \notin T^*$ (aresta de menor peso com exatamente uma extremidade em S).
3. Ao adicionar e em T^* cria-se um único ciclo C .
4. Para que $T^* - e$ fosse uma MST, tem que haver alguma outra aresta f com apenas uma extremidade em S (senão T^* seria desconexo).
5. Então $T = T^* + e - f$ também é árvore geradora. Como, $w(e) < w(f)$, segue que T tem peso mínimo.
6. Logo, T^* não é MST de G (contradição), o que invalida a hipótese inicial de que $e \notin T^*$.

Teorema: A árvore T obtida pelo algoritmo de Prim é uma MST de G .

1. Seja S o subconjunto de vértices de G na árvore T (definido pelo algoritmo).
2. O algoritmo de Prim adiciona em T a cada passo a aresta de menor custo com apenas um vértice extremidade em S .
3. Portanto, pela propriedade do corte, toda aresta e adicionada pertence a uma MST de G .



Análise da complexidade

Basicamente, há duas formas de analisar a complexidade do algoritmo de Dijkstra dependendo das estruturas de dados utilizadas.

Caso 1: $G = (V, E)$ é representado por uma matriz de adjacências
Fila de prioridades Q representada por um array simples de n elementos.

- a) Inicialização dos $\lambda(v)$ é $O(n)$
- b) Inserção dos vértices na fila Q é $O(n)$ (é $O(1)$ para cada vértice)
- c) Loop WHILE é executado n vezes (1 vez para cada $v \in Q$)
- d) $u = \text{ExtractMin}(Q)$ é $O(n)$ (equivale a encontrar menor elemento do array)
- e) Atualização do valor de $\lambda(v)$ é $O(1)$, mas executa $k = d(u)$ vezes (loop FOR)
- f) Decrease_Key é $O(1)$ (acesso direto)

Sendo assim a função $T(n)$ que mede a complexidade do algoritmo é:

$$T(n) = O(n) + O(n) + O(n) * O(n) + (O(1) + O(1)) * (d(v_1) + d(v_2) + \dots + d(v_n))$$

Sabendo que a multiplicação de dois termos lineares resulta em quadrático e que de acordo com o Handshaking Lema, a soma dos graus de um grafo é igual a duas vezes o número de arestas, temos:

$$T(n) = O(n) + O(n^2) + O(1) * O(m)$$

Como em todo grafo básico simples $m < n^2$, temos finalmente que o algoritmo é $O(n^2)$.

Caso 2: $G = (V, E)$ representado por uma lista de adjacências
Fila de prioridades Q representada por um heap binário (min-heap)

- a) Inicialização dos $\lambda(v)$ é $O(n)$
- b) Inserção dos vértices na fila Q é $O(n)$, pois todos os pesos são iguais inicialmente (infinitos).
- c) Loop WHILE é executado n vezes (1 vez para cada $v \in Q$)
- d) $u = \text{ExtractMin}(Q)$ é $O(\log n)$ (dequeue), mas dentro de loop (n vezes)
- e) Atualização do valor de $\lambda(v)$ é $O(1)$, mas executa $k = d(u)$ vezes (loop FOR)
- f) Decrease_Key é $O(\log n)$ (pode ter que subir no min-heap até a raiz – altura da árvore)

Sendo assim a função $T(n)$ que mede a complexidade do algoritmo é:

$$T(n) = O(n) + O(n \log n) + (O(1) + O(\log n)) * (d(v_1) + d(v_2) + \dots + d(v_n))$$

De modo similar ao caso anterior, podemos escrever:

$$T(n) = O(n) + O(n \log n) + O(m) + O(m \log n)$$

Como o termo com logaritmo domina os demais: $T(n) = O((n+m) \log n)$

Mas como em grafos conexos $m > n - 1$, chega-se que $T(n)$ é $O(m \log n)$.

Qual das duas implementações é mais eficiente? Depende! Devemos preferir a primeira opção se $m \log n > n^2$, o que equivale a:

$$\frac{m}{n^2} > \frac{1}{\log n} \rightarrow d > \frac{1}{\log n}$$

Em grafos mais densos, o caso 1 é mais eficiente.
Em grafos menos densos, o caso 2 é mais eficiente.

Por exemplo, se $n = 1024$, temos a seguinte regra:

- se $d > 0.1$, opção 1 é melhor
- se $d < 0.1$, opção 2 é melhor

Para que d seja igual a 0.1, um grafo com $n = 1024$ vértices deve ter 104857 arestas.

O algoritmo de compactação de Huffman

Trata-se de um algoritmo guloso muito utilizado para compressão de dados sem perdas.

Possibilita compactação de dados como zip e unzip, bastante conhecidos na computação.

Utilizado na codificação de dados, pois é capaz de gerar códigos binários de tamanhos variáveis baseados na probabilidade de ocorrência dos símbolos.

Na codificação tradicional, todos os símbolos do alfabeto possuem códigos do mesmo tamanho.

Seja $\alpha = (a, b, c, d, e, f)$ o alfabeto de símbolos a serem codificados. Para cada símbolo $\alpha_i \in \alpha$, seja w_i a probabilidade de ocorrência de α_i nos dados. A ideia do código de Huffman é gerar códigos menores para os símbolos mais prováveis e códigos maiores para símbolos menos prováveis, fazendo com que na média, o número de bits utilizado na codificação seja menor.

Suponha que nos nossos dados, as probabilidades w_i de cada símbolo sejam dadas por:

	a	b	c	d	e	f	
w	0.45	0.13	0.12	0.16	0.09	0.05	(soma igual a 1)

Na codificação tradicional, devemos utilizar 3 bits, pois com apenas 2 conseguimos identificar apenas 4 símbolos diferentes. Então, iniciando do símbolo a, temos a seguinte codificação:

	a	b	c	d	e	f
C	000	001	010	011	100	101
L(C)	3	3	3	3	3	3

Note que o comprimento de todos os códigos é fixo e igual a 3. Isso significa que, precisamos de 3 bits por símbolo, pois:

$$l(C) = \sum_{i=1}^n w_i l_i = 0.45 \times 3 + 0.13 \times 3 + 0.12 \times 3 + 0.16 \times 3 + 0.09 \times 3 + 0.05 \times 3 = 3$$

O código de Huffman para esses símbolos pode ser calculado a partir das probabilidades w_i . Veremos mais adiante como fazer isso. Por hora, basta sabermos que os códigos gerados pelo método de Huffman são:

	a	b	c	d	e	f
C	0	101	100	111	1101	1100
L(C)	1	3	3	3	4	4

Note que neste caso, o tamanho médio do código (valor esperado) é dado por:

$$l(C) = \sum_{i=1}^n w_i l_i = 0.45 \times 1 + 0.13 \times 3 + 0.12 \times 3 + 0.16 \times 3 + 0.09 \times 4 + 0.05 \times 4 = 0.45 + 0.39 + 0.36 + 0.48 + 0.36 + 0.20 = 2.1$$

Isso significa que, em média o número de bits necessário para codificar um símbolo é 2.1, o que é cerca de 30% menor que a codificação tradicional!

Algoritmo guloso

A seguir definimos as principais variáveis do algoritmo de Huffman.

1. α : conjunto de n caracteres (alfabeto)
2. $\forall \alpha_i \in \alpha$ seja $w(\alpha_i)$ a probabilidade de ocorrência de cada símbolo
3. Q: Fila de prioridades (quanto menor $w(\alpha_i)$, maior a prioridade)

O algoritmo a seguir apresenta o código de Huffman.

```
Huffman( $\alpha$ ) {
    n = | $\alpha$ |           # number of symbols
    # Insert symbols in Q
    for each  $\alpha_i \in \alpha$ 
        Insert(Q,  $\alpha_i$ )
    for i = 1 to n-1 {
        Create a new node z
        x = ExtractMin(Q)
        y = ExtractMin(Q)
        z.left = x
        z.right = y
        w(z) = w(x) + w(y)
        Insert(Q, z)
    }
    return ExtractMin(Q)    # returns the root of the tree
}
```

=> Porque o algoritmo de Huffman é guloso?

Note que o algoritmo utiliza uma abordagem bottom-up (inicia pelas folhas). Uma vez que o algoritmo inicia removendo da fila de prioridades os 2 símbolos de menor probabilidade, eles serão os nós mais distantes da raiz e por isso terão o maior código binário.

A seguir veremos um exemplo de como gerar o código de Huffman para um conjunto de símbolos.

Exemplo:

	a	b	c	d	e	f	(alfabeto)
w	0.45	0.13	0.12	0.16	0.09	0.05	(probabilidades)

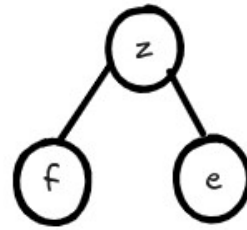
i = 1

$$Q^{(1)} = \{a:0.45, b:0.13, c:0.12, d:0.16, e:0.09, f:0.05\}$$

$$x = f$$

$$y = e$$

$$w(z) = w(x) + w(y) = 0.09 + 0.05 = 0.14$$



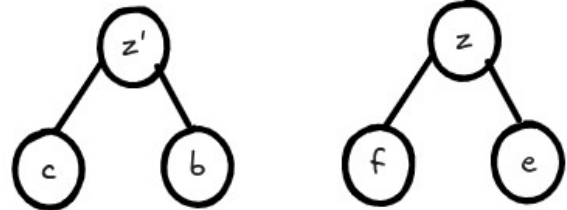
i = 2

$$Q^{(2)} = \{a:0.45, b:0.13, c:0.12, d:0.16, z:0.14\}$$

$$x = c$$

$$y = b$$

$$w(z') = w(x) + w(y) = 0.12 + 0.13 = 0.25$$



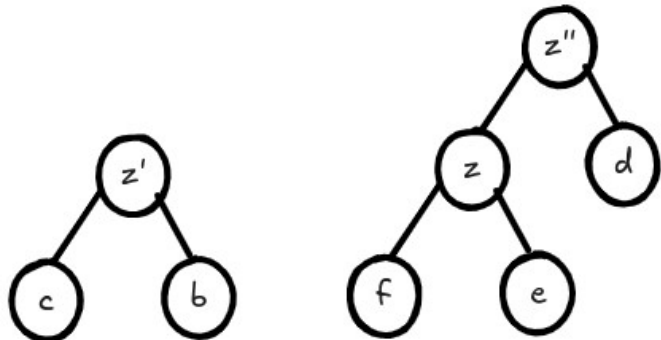
i = 3

$$Q^{(3)} = \{a:0.45, z':0.25, d:0.16, z:0.14\}$$

$$x = z$$

$$y = d$$

$$w(z'') = w(x) + w(y) = 0.14 + 0.16 = 0.25$$



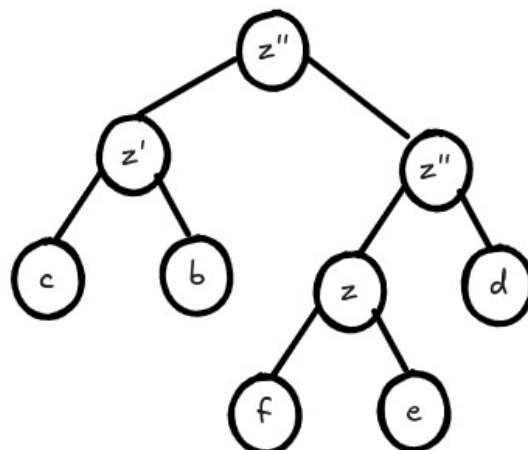
i = 4

$$Q^{(4)} = \{a:0.45, z':0.25, z'':0.3\}$$

$$x = z'$$

$$y = z''$$

$$w(z''') = w(x) + w(y) = 0.25 + 0.3 = 0.55$$



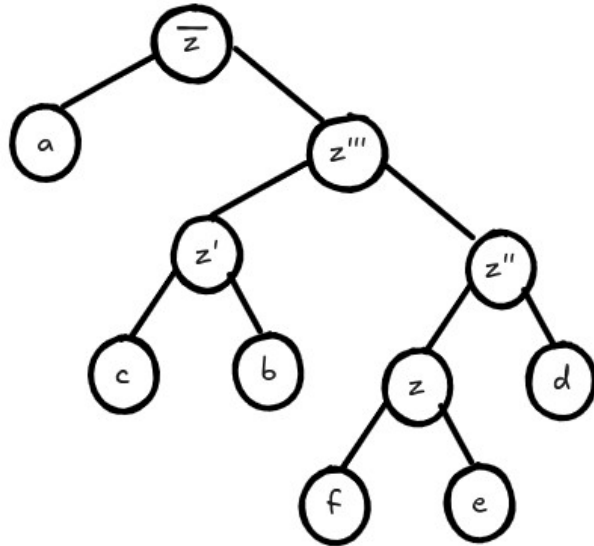
i = 5

$Q^{(5)} = \{a:0.45, z''':0.55\}$

$x = a$

$y = z'''$

$w(\bar{z}) = w(x) + w(y) = 0.45 + 0.55 = 1.0$



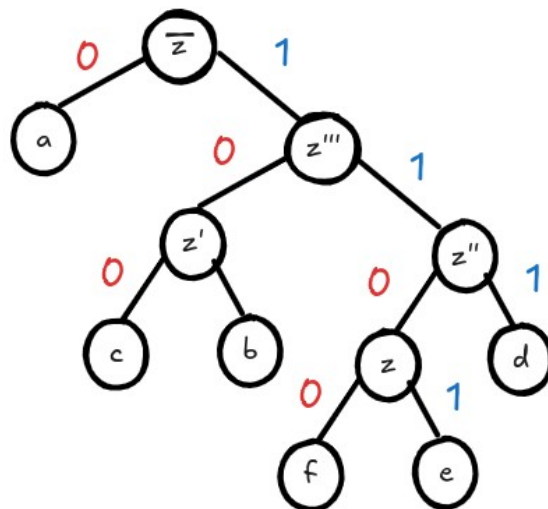
A árvore binária está completa. Como gerar os códigos dos símbolos?

Note que na árvore de Huffman, todos os símbolos são nós folhas da árvore. A ideia consiste em percorrer a árvore iniciando pela raiz. Cada vez que visitarmos um nó esquerda, adicionamos um 0 ao código e cada vez que visitarmos um nó a direita, adicionamos um 1 ao código.

A raiz da árvore deve ser dada

```
printCodes(node, code = '') {  
  if node.left ≠ NIL {  
    code = str(code) + '0'  
    PrintCodes(node.left, code)  
  }  
  if node.right ≠ NIL {  
    code = str(code) + '1'  
    PrintCodes(node.right, code)  
  }  
  # Se é nó folha (símbolo)  
  if node.left == NIL and node.right == NIL  
    print('node: ', code)  
}
```

Esse processo é equivalente a rotular as arestas da árvore de Huffman como segue:



o que nos leva aos seguintes códigos:

a: 0
b: 101
c: 100
d: 111
e: 1101
f: 1100

Note que quanto menor a probabilidade do símbolo, mais para baixo ele estará na árvore e portanto, maior será o tamanho de seu código. Isso faz com que símbolos raros tenham código grande e símbolos recorrentes tenham código pequeno. Em termos práticos, na média, o número de bits utilizado para a representação dos símbolos é menor, representando economia de memória.

Análise da complexidade

Basicamente, há duas formas de analisar o algoritmo de Huffman, dependendo das estruturas de dados utilizadas para implementar a fila de prioridades Q. A complexidade das primitivas Insert e ExtractMin dependem da estrutura de dados empregada para implementar Q.

=> **Caso 1:** Fila de prioridades Q como um array estático (n elementos).

* Neste caso, temos que:

- i) a primitiva Insert(Q, x) é $O(1)$ (inserção no final).
- ii) a primitiva ExtractMin(Q) é $O(n)$ (encontrar a menor probabilidade).

* Sendo assim, a complexidade do algoritmo de Huffman é dada por:

$$T(n) = n O(1) + n [O(n) + O(n) + O(1)] = O(n^2)$$

ou seja, temos uma complexidade quadrática.

=> **Caso 1:** Fila de prioridades Q como um heap binário (n elementos)

* Neste caso, temos que:

- i) a primitiva Insert(Q, x) é $O(\log n)$, devido a altura da árvore
- ii) a primitiva ExtractMin(Q) é $O(\log n)$, devido a altura da árvore.

* Sendo assim, a complexidade do algoritmo de Huffman é dada por:

$$T(n) = n O(\log n) + n [O(\log n) + O(\log n) + O(\log n)] = O(n \log n)$$

ou seja, temos uma complexidade log-linear.

Portanto, a melhor opção para implementar o algoritmo de Huffman é sempre a utilização de uma estrutura de dados do tipo min-heap.

A seguir demonstraremos um teorema que garante a otimalidade do algoritmo de Huffman, isto é, não existe nenhuma outra codificação que gere um tamanho médio de código menor que a codificação de Huffman.

Teorema: O código de Huffman é ótimo em termos de minimizar o tamanho do código. Seja

$$l(C) = \sum_{i=1}^n w_i l(\alpha_i)$$

o tamanho esperado do código C. Então, pode-se mostrar que não existe nenhum código C cujo valor de $l(C)$ é inferior ao valor de $l(CH)$, onde CH denota o código de Huffman.

Prova:

De acordo com Shannon, a quantidade de informação do símbolo α_i é:

$$h(\alpha_i) = \log_2 \frac{1}{w_i} \quad (\text{logaritmo do inverso da probabilidade})$$

A entropia associada a distribuição de probabilidades dos símbolos é dada por:

$$H(w) = \sum_{i=1}^n w_i h(\alpha_i) = \sum_{i=1}^n w_i \log \frac{1}{w_i} = \sum_{i=1}^n w_i \log w_i^{-1} = - \sum_{i=1}^n w_i \log w_i$$

O valor de $H(w)$ define um limite inferior para o tamanho médio do código. Nenhum código C possui $l(C) \geq H(w)$: esse é o resultado do Teorema da Codificação de Shannon.

Iniciamos calculando a diferença entre $H(w)$ e $l(C)$.

Então, desejamos calcular a diferença

$$H(w) - l(C) = \sum_{j=1}^n w_j \log_2 \frac{1}{w_j} - \sum_{j=1}^n w_j l(\alpha_j)$$

Note que podemos reescrever $l(\alpha_j)$ como:

$$l(\alpha_j) = \log_2 2^{l(\alpha_j)}$$

de modo que

$$H(w) - l(C) = \sum_{j=1}^n w_j \log_2 \frac{1}{w_j} - \sum_{j=1}^n w_j \log_2 2^{l(\alpha_j)} = \sum_{j=1}^n w_j \log_2 \frac{1}{w_j} + \sum_{j=1}^n w_j \log_2 2^{-l(\alpha_j)}$$

Colocando em evidência, temos:

$$H(w) - l(C) = \sum_{j=1}^n w_j \log_2 \frac{2^{-l(\alpha_j)}}{w_j}$$

Sabemos que:

$$\log_b a = \frac{\log_x a}{\log_x b}$$

ou seja,

$$\log_2 \frac{2^{-l(\alpha_j)}}{w_j} = \frac{\ln \frac{2^{-l(\alpha_j)}}{w_j}}{\ln 2} = \frac{1}{\ln 2} \ln \frac{2^{-l(\alpha_j)}}{w_j}$$

o que nos leva a:

$$H(w) - l(C) = \frac{1}{\ln 2} \sum_{j=1}^n w_j \ln \frac{2^{-l(\alpha_j)}}{w_j}$$

Mas, sabemos que $\ln x \leq x - 1$, ou seja:

$$H(w) - l(C) \leq \frac{1}{\ln 2} \sum_{j=1}^n w_j \left(\frac{2^{-l(\alpha_j)}}{w_j} - 1 \right)$$

Aplicando a distributiva, temos:

$$H(w) - l(C) \leq \frac{1}{\ln 2} \left(\sum_{j=1}^n 2^{-l(\alpha_j)} - \sum_{j=1}^n w_j \right)$$

Mas sabemos que a soma das probabilidades em uma distribuição é igual a 1:

$$H(w) - l(C) \leq \frac{1}{\ln 2} \left(\sum_{j=1}^n 2^{-l(\alpha_j)} - 1 \right) \quad (*)$$

Veremos a seguir que se C for o código de Huffman, a sequência $l(\alpha_1) + l(\alpha_2) + \dots + l(\alpha_n)$ deve satisfazer:

$$\left(\frac{1}{2}\right)^{l(\alpha_1)} + \left(\frac{1}{2}\right)^{l(\alpha_2)} + \dots + \left(\frac{1}{2}\right)^{l(\alpha_n)} = \sum_{j=1}^n 2^{-l(\alpha_j)} \leq 1$$

pois os tamanhos dos códigos $l(\alpha_j)$ denotam a profundidade dos símbolos α_i em uma árvore binária. Seja $l_{\max} = \max\{l(\alpha_1), l(\alpha_2), \dots, l(\alpha_n)\}$ o maior tamanho de código na árvore gerada pelo algoritmo de Huffman. Então, como temos uma árvore binária, a soma do número de nós dos níveis intermediários é sempre menor que o número de nós da última camada. Supondo uma árvore de n níveis, temos que:

$$\sum_{j=1}^n 2^{n-j} = 2^{n-1} + 2^{n-2} + \dots + 2^0 \leq 2^n$$

Por exemplo, se $n = 5$, as 4 primeiras camadas podem ter no máximo quantos nós?

$$2^0 + 2^1 + 2^2 + 2^3 + 2^4 = 1 + 2 + 4 + 8 + 16 = 31$$

o que é menor que o número máximo de nós na camada mais profunda da árvore, $2^5 = 32$. Por uma simples substituição de variáveis, podemos escrever:

$$\sum_{j=1}^n 2^{l_{\max} - l(\alpha_j)} \leq 2^{l_{\max}}$$

Dividindo tudo por $2^{l_{\max}}$, chega-se em:

$$\sum_{j=1}^n 2^{-l(\alpha_j)} \leq 1$$

Logo, voltando a equação (*), temos finalmente que:

$$H(w) - l(C) \leq 0$$

ou seja, $l(C) \geq H(w)$, o tamanho do código está limitado pela entropia da distribuição de probabilidades dos símbolos.

Portanto, como

$$l(C) = \sum_{i=1}^n w_i l(\alpha_i) \quad \text{e} \quad H(w) = \sum_{i=1}^n w_i h(\alpha_i)$$

a intuição consiste em notar que $l(C) = H(w)$ quando $l(\alpha_i) = h(\alpha_i)$, ou seja, quando o tamanho do código é proporcional a quantidade de informação de α_i , que é justamente a ideia do código de Huffman, pois quanto menor a probabilidade, maior a quantidade de informação e o código daquele símbolo. Lembre-se que no código de Huffman, quanto maior a quantidade de informação $h(\alpha_i)$, mais longe o símbolo α_i está da raiz, e portanto maior o seu tamanho do código $l(\alpha_i)$.

"The real voyage of discovery consists not in seeking new landscapes, but in having new eyes."
-- Marcel Proust

Problemas NP-Completos

Há problemas computacionais que não podem ser resolvidos por algoritmos, nem que dispuséssemos de um período de tempo infinito: são os problemas incomputáveis ou indecidíveis.

Um exemplo clássico, formulado por Alan Turing, considerado o pai da computação, é o problema da parada (The halting problem). Dado um programa P e uma entrada I , devemos decidir se o programa terminará sua execução com essa entrada ou será executado para sempre. Não pode existir um algoritmo que receba P e resolva esse problema de decisão!

Veremos a seguir um breve esboço do principal argumento utilizado na prova formal.

Suponha que exista uma função computável $\text{halts}(f)$ que retorna True se a função f termina e False caso contrário (nunca termina). Seja a função $g()$ a seguir:

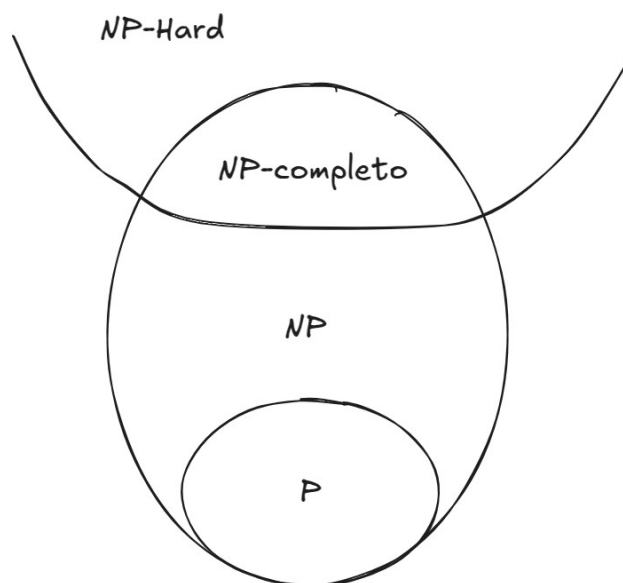
```
function g() {  
    if halts(g)  
        loop_forever  
}
```

Note que se $\text{halts}(g)$ retorna True, significa que ela termina, mas então ela entra em loop infinito, o que gera uma contradição!

Mas se $\text{halts}(g)$ retorna False, então a função g irá terminar (não entra no if), o que também gera uma contradição!

Portanto, a suposição inicial de que existe um função computável $\text{halts}()$ é FALSA!

Dentre os problemas computáveis, podemos organizar a seguinte estrutura de classes: problemas P, NP, NP-Completo e NP-Hard. A figura a seguir ilustra um diagrama.



Def: Dizemos que $p \in P$ se o problema é facilmente solucionável, ou seja, existe um algoritmo A com complexidade $O(n^k)$ (polinomial) que resolve P . Em termos matemáticos, o problema p pode ser resolvido por uma máquina de Turing determinística.

Exemplos são problemas como a ordenação, que possuem algoritmos exatos.

Def: Dizemos que $p \in NP$ se o problema é facilmente verificável, mas não facilmente solucionável. Em outras palavras, não existe algoritmo eficiente que resolva p , mas dada uma solução, é fácil verificar se ela é válida. Em termos matemáticos, p pode ser resolvido por uma máquina de Turing não determinística.

Def: Os problemas $p \in NP$ para os quais $\forall q \in NP$ podem ser reduzidos a p de maneira eficiente (em tempo polinomial) são conhecidos como NP-Completo.

NP-Completo são problemas NP-Hard que estão em NP. Por isso são geralmente muito difíceis de serem resolvidos.

Def: Existem problemas $p \notin NP$ para os quais verificar se uma dada solução é válida sequer é viável: são os problemas NP-Hard (considerados os mais difíceis de todos)

Obs: A diferença é que problemas NP-Completo são problemas de decisão, enquanto que problemas NP-Hard são problemas de otimização.

A pergunta natural que surge é: o que é redução?

A redução de problemas é uma técnica fundamental na teoria da complexidade computacional para comparar a dificuldade de dois problemas. No contexto dos problemas NP-Completo, ela permite demonstrar que um problema é "pelo menos tão difícil quanto" outro, estabelecendo relações hierárquicas entre problemas. Essa abordagem é central para provar que um problema pertence à classe NP-Completo e para entender a estrutura da complexidade computacional.

Uma redução transforma instâncias de um problema A em instâncias de um problema B, de forma que:

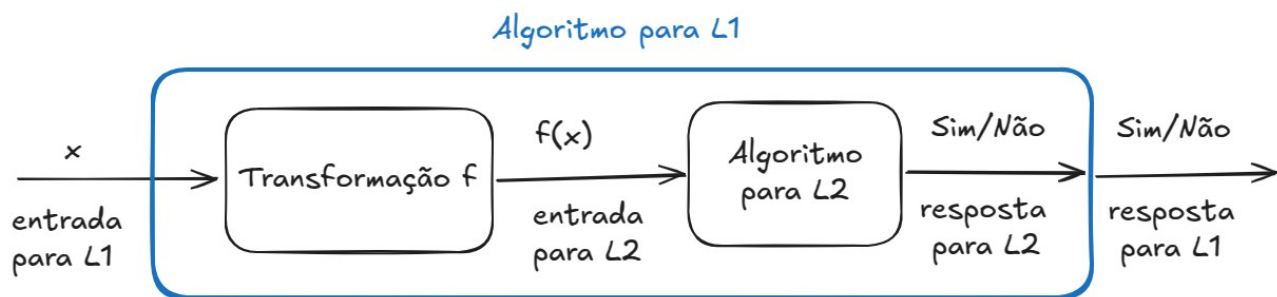
1. A resposta para a instância de B (sim/não, no caso de problemas de decisão) corresponda à resposta para A.
2. A transformação seja feita em tempo polinomial (polynomial-time reduction).

Se tal redução existe, dizemos que A é redutível a B (notação $A \leq_p B$). Isso implica que:

- Se B for resolvido eficientemente (em tempo polinomial), A também pode ser resolvido eficientemente.
- Se A for "difícil" (ex: NP-Completo), então B é pelo menos tão difícil quanto A.

Em outras palavras, sejam L_1 e L_2 dois problemas de decisão e suponha que o algoritmo A_2 resolva L_2 . Em outras palavras, se y é uma entrada para L_2 , então A_2 retornará True ou False, dependendo se $y \in L_2$ ou $y \notin L_2$.

Ideia: encontrar uma transformação de L_1 para L_2 de modo que o algoritmo A_2 possa ser parte de um algoritmo A_1 para resolver L_1 .



=> Pergunta: Como provar que um problema p é NP-Completo?

Parece impossível: Todo problema $q \in NP$ deve ser reduzido a p ?

Ideia: tomar um problema NP-Completo conhecido e reduzi-lo a p . Se a redução for possível em tempo polinomial, prova-se que p é NP-Completo por transitividade da redução, ou seja, se um problema NP-Completo é reduzido a p em tempo polinomial, então todos os problemas em NP são reduzíveis a p em tempo polinomial.

Qual foi o primeiro problema a provado a ser NP-Completo?

SAT (Boolean satisfiability problem)

Consiste em determinar se existe uma interpretação que satisfaz uma dada expressão Booleana arbitrária. Existe alguma forma de atribuir valores lógicos as variáveis Booleanas da expressão lógica de modo que ela seja verdadeira?

Precisamos construir uma Tabela-Verdade.

Por exemplo, se tivermos uma expressão lógica com n variáveis atômicas, precisamos de 2^n linhas na Tabela-Verdade.

Claramente, a complexidade do problema é exponencial: $O(2^n)$

Um dos 7 problemas do milênio: $P = NP$?

Instituto Clay dos EUA oferece um prêmio de 1 milhão de dólares para quem provar que $P = NP$.

Traria implicações inimagináveis na computação: se eventualmente você conseguir reduzir um problema NP-Completo para P , todos os problemas de NP virariam P (criptografia, segurança, etc.)

Grafos Hamiltonianos

Iremos iniciar o estudo de problemas NP-Completo com um problema clássico da computação.

Dado um grafo $G = (V, E)$, decidir se G é Hamiltoniano, ou seja, se é possível sair de um vértice v , visitar todo outro vértice de G exatamente uma vez e retornar ao vértice de origem v .

Para isso precisamos definir o que são grafo Hamiltonianos e como são caracterizados.

Def: Um ciclo Hamiltoniano é um ciclo que engloba todo vértice de G (ciclo não permite repetição)

Def: Um grafo G é Hamiltoniano se e somente se G possui um ciclo Hamiltoniano

Obs: Todo grafo completo de n vértices, denotado por K_n é Hamiltoniano.

Pergunta: Como decidir se um dado grafo G é Hamiltoniano? Problema difícil. (NP-Completo)
Porque? Melhor resultado que se tem até hoje

Teorema (Dirac, 1952): Seja $G = (V, E)$ um grafo básico simples com $|V| = n > 2$. Então,

Se $\forall v \in V \left(d(v) \geq \frac{n}{2} \right)$, então G é Hamiltoniano.

Em outras palavras, o que esse resultado nos diz é que se cada um dos vértices de G se liga pelo menos a metade dos demais, então G é Hamiltoniano. (porém note que se G é Hamiltoniano, essa propriedade não necessariamente precisa ser verdadeira!)

Problema em aberto: ninguém nunca demonstrou a volta. Nem mesmo propôs um critério mais efetivo (do tipo se e somente se)

O teorema anterior, apesar de poderoso, não nos fornece uma caracterização completa de grafos Hamiltonianos. Basicamente, ele diz que todo grafos que satisfaz a propriedade H é Hamiltoniano, mas nem todo grafo Hamiltoniano satisfaz a propriedade H.

Decidir se um grafo $G = (V, E)$ é Hamiltoniano é um problema NP-Completo.

O problema do caixeiro-viajante

Dado um grafo $G = (V, E, w)$ encontrar um ciclo Hamiltoniano de custo mínimo, ou seja, obter o ciclo C que minimiza:

$$w(C) = \sum_{e \in C} w(e)$$

Descrição do problema: A partir de um vértice inicial v_0 , visitar todos os demais uma única vez, voltando para v_0 , caminhando o mínimo possível. É um problema de otimização e não de decisão (NP-Hard).

Pergunta: como obter um ciclo Hamiltoniano de custo mínimo?

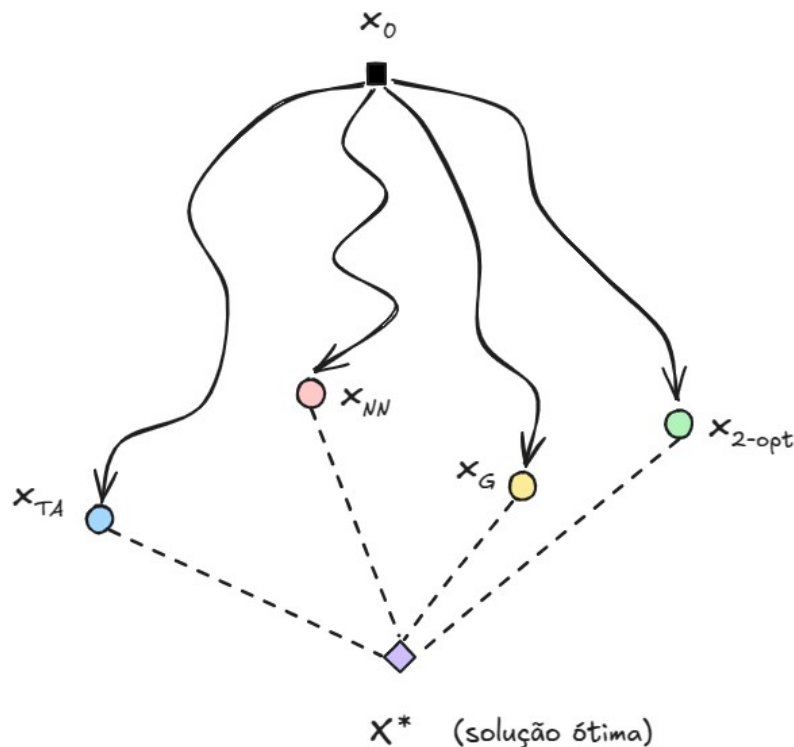
- Não se conhece algoritmo ótimo (não há garantias de que teremos a solução ótima x^*)
- Devemos trabalhar com algoritmos aproximados (sub-ótimos).

Mas como medir a qualidade da solução obtida por um algoritmo aproximado?

Devemos medir quão próximo da solução ótima é a solução aproximada. Definem-se:

$f(x^*)$: custo da solução ótima (melhor possível)
 $f(x_A)$: custo da solução obtida pelo algoritmo A

Dizemos que o algoritmo A fornece uma c-aproximação para o TSP se $f(x_A) \leq c f(x^*)$



TSP métrico

Seja $G = (V, E, w)$ um grafo conexo com $w: E \rightarrow \mathbb{R}^+$, em que w é uma função de distância. Então, a desigualdade triangular é satisfeita, ou seja:

$$\forall i, j, k \quad w(v_i, v_k) \leq w(v_i, v_j) + w(v_j, v_k)$$

Isso significa que tomar atalhos é sempre vantajoso (corta caminhos).

Uma instância do TSP em que os pesos das arestas do grafo de entrada G é uma função de distância é conhecida como TSP métrico.

Algoritmos para o TSP

A seguir apresentamos alguns dos principais algoritmos aproximados para o TSP, iniciando pelo método dos vizinhos mais próximos.

Nearest Neighbor

O algoritmo mais simples para o TSP utiliza uma abordagem gulosa: visitar sempre o vértice mais próximo do vértice atual. Porém, esse algoritmo não fornece boas soluções, em especial quando o número de vértices cresce de maneira arbitrária.

```
Nearest_Neighbor( $G, w, v_0$ ) {
     $H = [v_0]$ 
    while  $|H| < n$  {
         $x = \text{Last}(H)$            # retorna o último vértice de H
```

```

        Defina  $v_i$  como o vizinho mais próximo de  $x$ 
        InsertEnd( $H, v_i$ )    # insere  $v_i$  no final de  $H$ 
    }
    InsertEnd( $H, v_0$ )
}

```

Teorema: A solução obtida pelo algoritmo Nearest Neighbor no TSP métrico satisfaz:

$$f(x_{NN}) \leq \frac{1}{2}(\log_2 n + 1)f(x^*)$$

Note que se o número de vértices do grafo G aumenta, a qualidade da solução cai de maneira logarítmica. Em outras palavras, não é muito bom para grafos com grande quantidade de vértices.

Análise da complexidade

1. Note que o loop WHILE executa n vezes
2. Obter o último elemento da lista H pode ser realizado em tempo constante $O(1)$ se mantivermos um contador para o número de elementos em H .
3. Para definir o vizinho mais próximo de x , é preciso verificar todas as distâncias da linha de x na matriz de adjacências, o que é $O(n)$
4. Inserção no final da lista H também pode ser feito em $O(1)$ se mantivermos um contador para o número de elementos de H

Custo total: $n \times [O(1) + O(n) + O(1)] = O(n^2)$

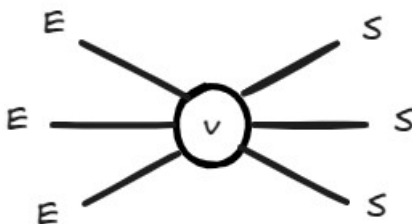
Twice-Around

Antes de apresentarmos o algoritmo Twice-Around, precisamos definir o que é um grafo Euleriano.

Def: Um grafo $G = (V, E)$ é Euleriano se G contém um circuito Euleriano, que é uma trilha fechada que envolve toda aresta de G . Em outras palavras, a partir de um vértice v , é possível passar uma única vez por cada aresta de G e voltar a v .

Teorema: Um grafo $G = (V, E)$ é Euleriano, se e somente se, $\forall v \in V (d(v) \bmod 2 = 0)$, ou seja, se o grau de todos os vértices é par.

Em um grafo Euleriano é possível extrair um circuito Euleriano, ou seja, atravessar toda aresta exatamente uma única vez, voltando ao vértice inicial. A ideia é que para cada aresta de entrada, haverá uma outra aresta de saída, de modo que não é possível ficar preso durante o circuito.



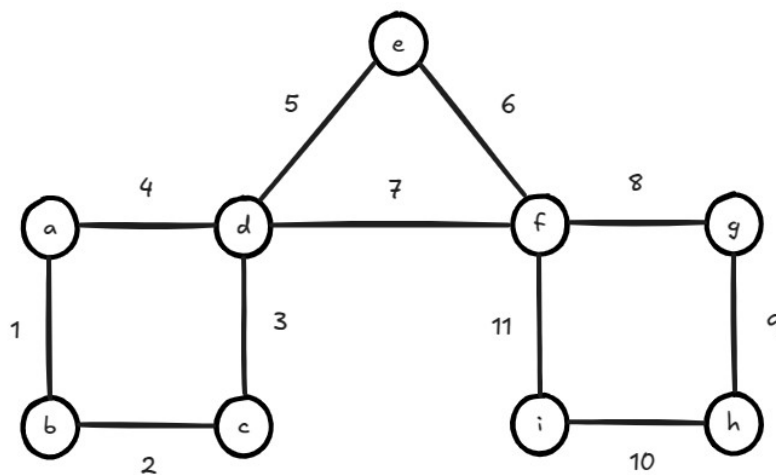
Para entender como podemos extrair um circuito Euleriano a partir de um grafo em que todos os graus são pares, apresentamos o algoritmo de Hierholzer.

```

Eulerian_Circuit(G, v) {
    # Arestas iniciam desmarcadas
    for each e = (u,v) in E
         $\alpha(u,v)$  = False
    S = Stack()           # Pilha S
    Q = Queue()           # Fila Q
    push(S, v)
    while S  $\neq$   $\emptyset$  {
        u = Peek(S)       # verifica quem é o vértice do topo
        dead_end = True
        for each w  $\in$  N(u) {
            if  $\alpha(u,w)$  == False {    # pode atravessar
                push(S, w)
                 $\alpha(u,w)$  = True      # já passou pela aresta
                dead_end = False
                break
            }
        }
        if dead_end {
            u = pop(S)
            InsertLeft(Q, u)    # insere no início da fila
        }
    }
    return Q
}

```

Vejamos um exemplo ilustrativo a seguir.



Iremos iniciar no vértice **a** e adicionar as arestas na ordem em que aparecem na figura:

S = [a]
Q = $\emptyset$

Após inserirmos os vértices b, c, d e a no topo de S, chegamos num beco sem saída:

$S = [a, b, c, d, a]$ // desempilha a e adiciona em Q
 $Q = \emptyset$

Devemos desempilhar a e adicionar a esquerda em Q:

$S = [a, b, c, d]$
 $Q = [a]$

Seguindo a ordem, iremos adicionar e, f e d no topo de S, até chegarmos em outro beco sem saída:

$S = [a, b, c, d, e, f, d]$
 $Q = [a]$

Devemos desempilhar d e adicionar a esquerda em Q
 $S = [a, b, c, d, e, f]$
 $Q = [d, a]$

Seguindo a ordem, adicionamos os vértices g, h, i e f, novamente atingindo um beco sem saída:

$S = [a, b, c, d, e, f, g, h, i, f]$
 $Q = [d, a]$

Como todas as arestas já foram marcadas, só nos resta desempilhar os vértices de S em Q:

$S = \emptyset$
 $Q = [a, b, c, d, e, f, g, h, i, f, d, a]$

Note que no final do algoritmo a pilha encontra-se vazia e a fila contém o circuito Euleriano.

Análise da complexidade

1. Para marcar inicialmente as arestas o custo é $O(m)$
2. O loop WHILE executa uma vez para cada vértice, ou seja n vezes.
3. Verificamos quem é o vértice do topo da pilha n vezes: Peek é $O(1)$ x $n = O(n)$
4. Como cada aresta é visitada uma única vez, o FOR executa $d(u)$ vezes, em que $d(u)$ é o grau de u.
5. Como sabemos, pelo Handshaking Lema, temos que:

$$\sum_{i=1}^n d(v_i) = 2m$$

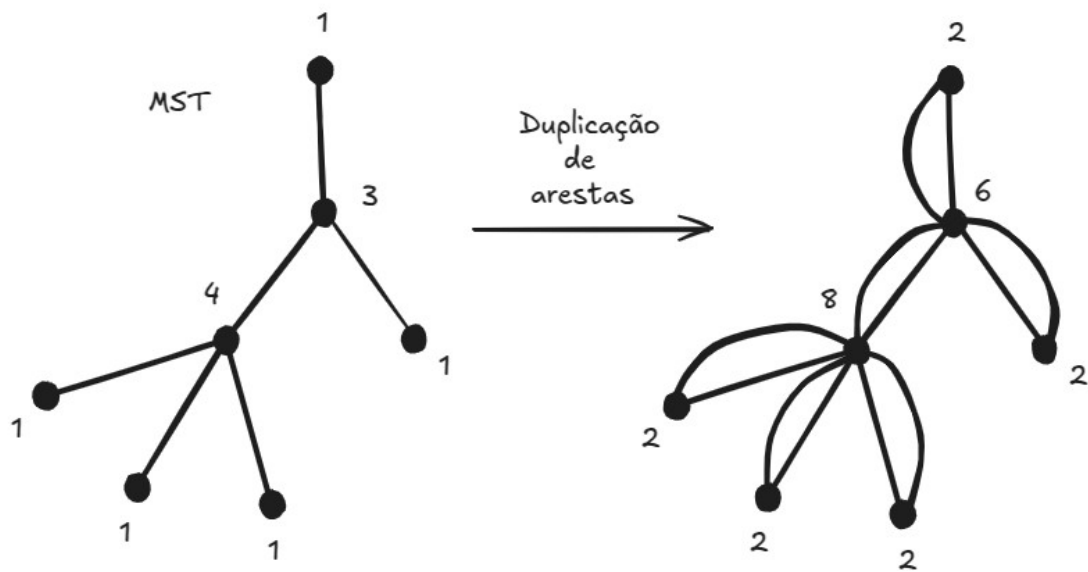
ou seja, o somatório dos graus dos vértices de um grafo G é igual a duas vezes o número de arestas.

6. Portanto, o custo total do algoritmo é dado por:

$$O(m) + O(n) + O(2m) = O(m)$$

uma vez que em grafos conexos $m > n - 1$, ou seja, m domina n.

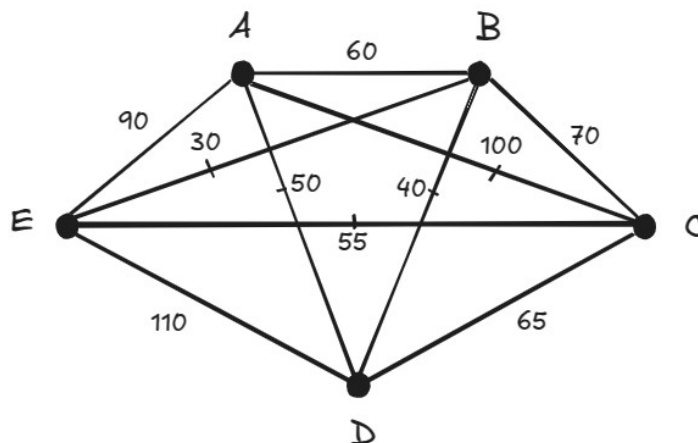
Teorema: Seja $G = (V, E)$ um grafo conexo. Se T é uma MST de G , então se definirmos um supergrafo T' duplicando toda aresta de T , T' será Euleriano.



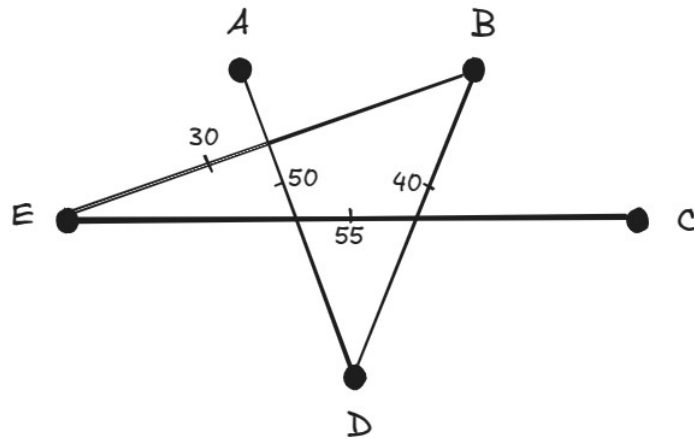
É fácil notar que ao duplicar todas as arestas da MST, estamos multiplicando os graus por 2. Assim, os nós folhas, passarão a ter grau 2 e todos os demais nós internos serão múltiplos de 2. A seguir apresentamos o algoritmo `Twice_Around`, para a solução aproximada do TSP métrico.

```
Twice_Around( $G, w, v_0$ ) {
     $H = \emptyset$ 
     $T = \text{MST}(G)$                                 # Extrai MST de G
    for each  $e \in T$                                 # Duplica as arestas da MST
         $T = T \cup \{e\}$ 
     $L = \text{Eulerian\_Circuit}(T)$                     # Obtém circuito Euleriano
    while  $L \neq \emptyset$  {
         $l = \text{RemoveFirst}(L)$ 
        if  $l \notin H$                                 # Remove as repetições
             $H = H \cup \{l\}$ 
    }
    return  $H$ 
}
```

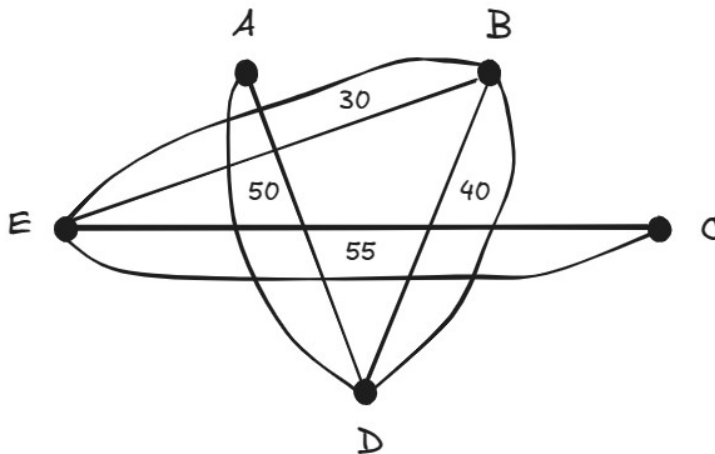
Veremos a seguir um exemplo ilustrativo da aplicação do algoritmo `Twice-Around`.



1º passo: Extrair MST de G



2º passo: Duplicar arestas da MST

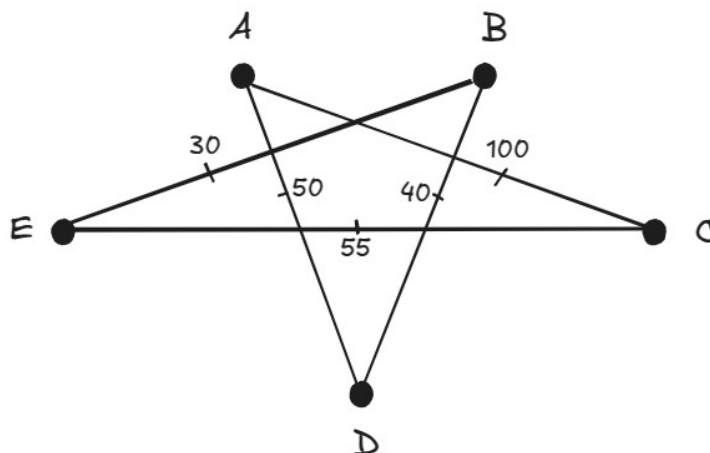


3º passo: Extrair um circuito Euleriano L

$L = [A, D, B, E, C, E, B, D, A]$

4º passo: Eliminar as repetições (equivale a tomar atalhos)

$H = [A, D, B, E, C, A]$



Note que o custo do ciclo Hamiltoniano obtido é $w(C) = 50 + 40 + 30 + 55 + 100 = 275$

Análise da complexidade

1. Obter a MST de G com Kruskal é $O(m \log n)$. Com Prim é $O(n^2)$ (estruturas estáticas) ou $O(m \log n)$ (estruturas dinâmicas).
2. A duplicação das arestas da MST pode ser feita em $O(m)$
3. A extração do circuito Euleriano L é pelo algoritmo de Hierholzer é $O(m)$
4. Por fim, o WHILE é executado uma vez para cada aresta duplicada: $2m$ iterações, o que é $O(m)$
5. Portanto, o custo final é: $O(m \log n) + O(m) + O(m) + O(m) = O(m \log n)$

Ou seja, o ponto crítico (gargalo) no algoritmo Twice-Around é o cálculo da MST.

Veremos a seguir um resultado muito importante sobre a qualidade da solução obtida pelo algoritmo Twice-Around.

Teorema: A solução obtida pelo algoritmo Twice-Around no TSP métrico satisfaz:

$$f(x_{TA}) \leq 2f(x^*)$$

ou seja, a solução do TA é no máximo 2 vezes pior que a solução ótima (não importa o número de vértices de G). Melhor que Nearest Neighbor.

É fácil verificar que se x^* é a solução ótima, então se subtrairmos uma aresta de x^* , obtemos uma árvore T, ou seja, $T = x^* - e$. Também sabemos que essa árvore T tem peso maior ou igual que a MST do grafo e por isso temos:

$$f(x^*) > w(T) \geq w(T_{MST})$$

o custo de x^* é maior que de T pois o ciclo tem uma aresta a mais

Então temos que:

$$f(x^*) > w(T_{MST})$$

o que implica em

$$2f(x^*) > 2w(T_{MST}) \quad (*) \text{ (é o peso do tour de Euler extraído no passo 2)}$$

Se G é Euclidiano (tomar atalhos é sempre mais vantajoso devido a desigualdade triangular)

$$f(x_{TA}) \leq 2w(T_{MST}) \quad (**)$$

E portanto, combinando (*) e (**)

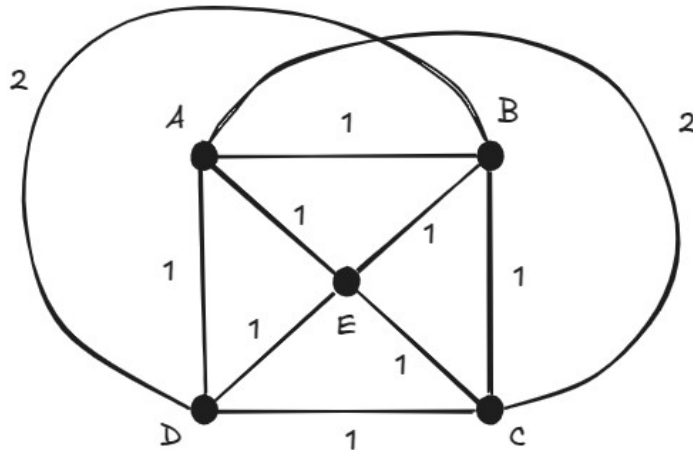
$$2f(x^*) > 2w(T_{MST}) \geq f(x_{TA})$$

O algoritmo de Christofides

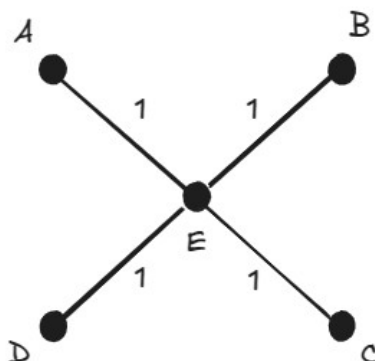
A ideia do algoritmo de Christofides consistem em melhorar ainda mais o algoritmo Twice-Around, modificando a forma com que transformamos a MST de G em um grafo Euleriano. Ao invés de simplesmente duplicar as arestas da árvore, devemos encontrar um emparelhamento perfeito de custo mínimo entre os vértices de grau ímpar na MST. Dessa forma, com menos duplicações de arestas, o custo adicional é menor.

```
Christofides( $G, w, v_0$ ) {  
     $H = \emptyset$   
     $T = \text{MST}(G)$                                 # Extrai MST de  $G$   
     $K = \text{Odd\_Vertices}(T)$                         # Constrói  $K_n$  (ímpares)  
     $M = \text{Minimum Weight\_Matching}(K_u)$           # Emparelhamento mínimo  
     $T = T + M$                                      # Adiciona arestas de  $M$   
     $T = \text{Eulerian\_Circuit}(T)$                   # Obtém circuito Euleriano  
    while  $L \neq \emptyset$  {  
         $l = \text{RemoveFirst}(L)$   
        if  $l \notin H$                                // Remove as repetições  
             $H = H \cup \{l\}$   
    }  
    return  $H$   
}
```

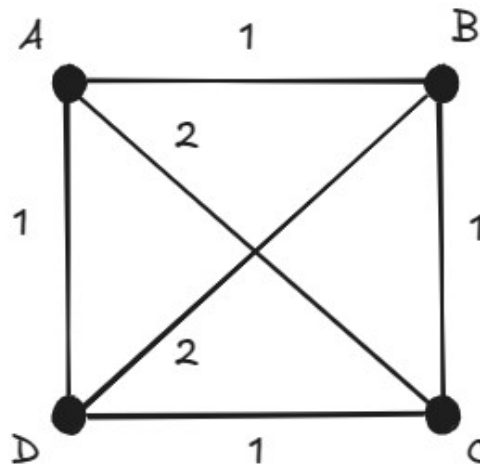
A seguir veremos um exemplo ilustrativo do funcionamento do algoritmo de Christofides. Considere o seguinte grafo G .



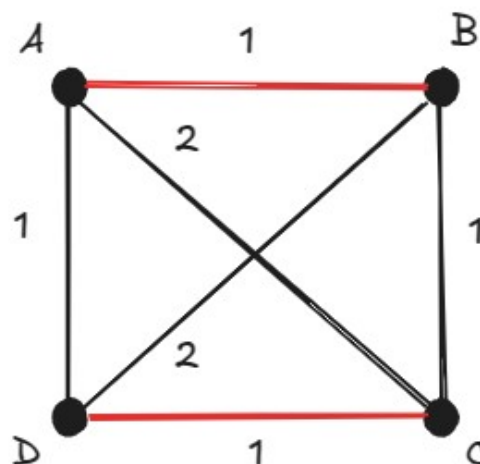
1º passo: Extrair a MST de G



2º passo: criar grafo completo K_n com todos os vértices de grau ímpar e ponderar as arestas com os caminhos mínimos dentre cada par de vértices.



3º passo: encontrar um emparelhamento de custo mínimo entre os vértices de grau ímpar.



Pode-se mostrar que o número de emparelhamentos perfeitos no K_n é dado por:

$$\prod_{i \text{ ímpar}, i < n} i$$

No caso de $n = 4$, temos $1 \times 3 = 3$

No caso de $n = 6$, temos $1 \times 3 \times 5 = 15$

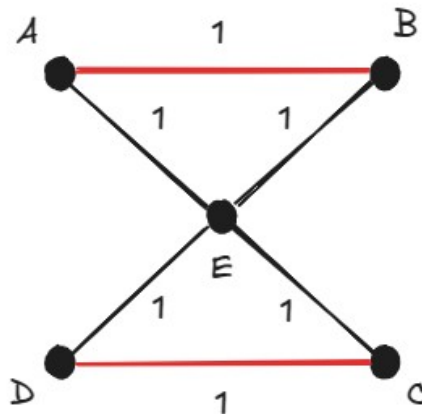
No caso de $n = 8$, temos $1 \times 3 \times 5 \times 7 = 105$

No caso de $n = 10$, temos $1 \times 3 \times 5 \times 7 \times 9 = 945$

Note que o número de possíveis emparelhamentos perfeitos cresce exponencialmente, o que inviabiliza a busca exaustiva pelo emparelhamento mínimo quando o número de vértices cresce. Existe um algoritmo com complexidade $O(n^2 m)$ que obtém o emparelhamento M de custo mínimo em um grafo arbitrário (não precisa ser bipartido). É o algoritmo de Edmonds, também conhecido como *Blossom algorithm*. Para os leitores interessados, indicamos o link a seguir:

https://en.wikipedia.org/wiki/Blossom_algorithm

4º passo: Adicione as arestas de M à MST



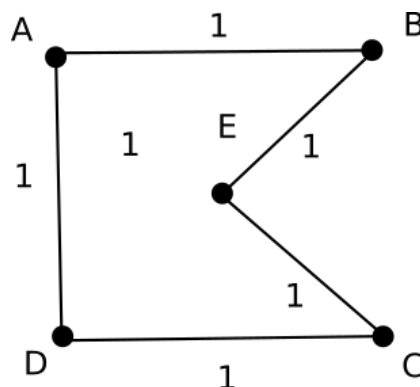
Agora, todos os vértices possuem grau par, então é possível extrair um circuito Euleriano.

5º passo: Encontre um circuito Euleriano L

$L = [A, B, E, C, D, E, A]$

6º passo: Eliminar as repetições (equivalente a tomar atalhos)

$L = [A, B, E, C, D, A]$



Análise da complexidade

1. Obter a MST de G com Kruskal é $O(m \log n)$. Com Prim é $O(n^2)$ (matriz de adjacências + array) ou $O(m \log n)$ (listas de adjacências + min-heap).

2. Obter o emparelhamento de custo mínimo M entre os vértices de grau ímpar: seja z o número de vértices ímpares. O grafo completo formado pelos vértices ímpares, denotado por K_z , terá m' arestas, onde:

$$m' = \frac{z(z-1)}{2}$$

3. Para ponderar os pesos do grafo K_z , será preciso encontrar caminhos mínimos entre cada par de vértices. Podemos executar z vezes o algoritmo de Dijkstra no grafo original, o que nos leva a uma complexidade $O(zm \log n)$

4. Para a construção do emparelhamento M de mínimo custo, o algoritmo de Edmonds possui complexidade $O(z^2 m')$

5. A extração do circuito Euleriano L é lo algoritmo de Hierholzer é $O(m)$

6. Por fim, o WHILE é executado uma vez para cada aresta duplicada: $2m$ iterações, o que é $O(m)$

7. Portanto, o custo final é:

$$O(m \log n) + O(z m \log n) + O(k^2 m') + O(m) \rightarrow O(z m \log n) + O(z^2 m')$$

Ou seja, é quadrático no número de vértices ímpares! Quando o número de vértices ímpares é grande, ou seja, $z \rightarrow n$, o custo pode ser bastante elevado em comparação com o algoritmo Twice-Around.

A seguir, iremos demonstrar um resultado fundamental sobre o algoritmo de Christofides.

Teorema: O algoritmo de Christofides fornece uma $3/2$ -aproximação para o TSP métrico.

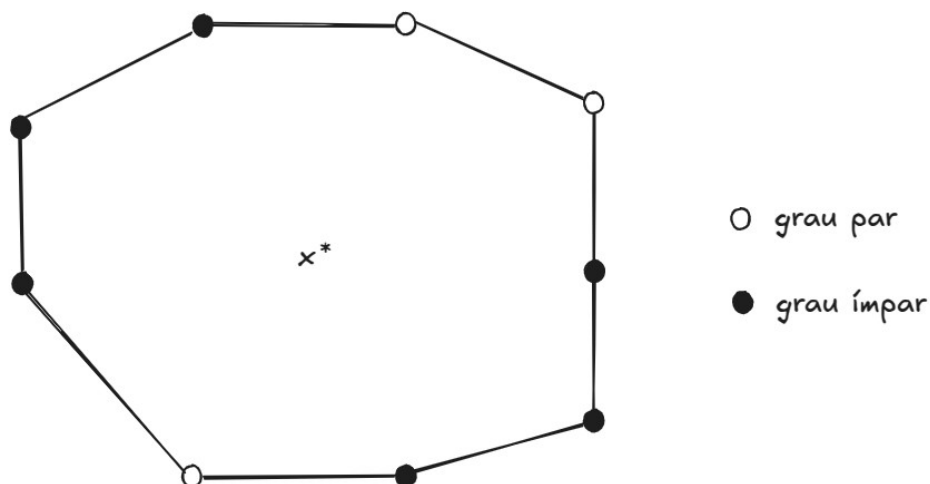
Prova: Desejamos mostrar que:

$$f(x_c) \leq \frac{3}{2} f(x^*)$$

onde x^* é a solução ótima. Sabemos que o custo da MST T é menor que x^* , ou seja:

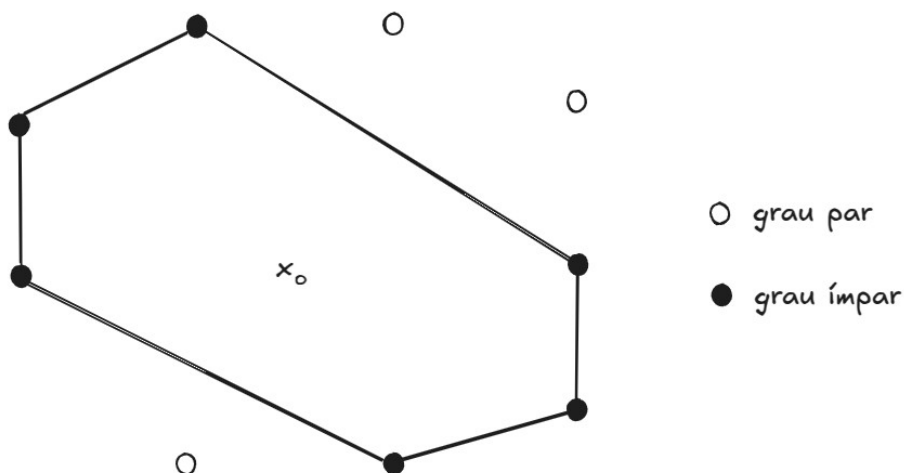
$$w(T_{MST}) < f(x^*)$$

pois $x^* - e$ é uma árvore com peso maior ou igual ao peso da MST de G . Basta agora provar que o custo do emparelhamento M entre os vértices ímpares é menor ou igual a $\frac{1}{2} f(x^*)$. Lembre-se que aqui, ao invés de duplicar todas as arestas da MST, adicionamos menos arestas a MST. Iniciamos a discussão com o ciclo Hamiltoniano de custo mínimo, cujo custo é $f(x^*)$.



onde os vértices pretos denotam os vértices de grau ímpar (subconjunto O de odd degree) e os vértices brancos denotam os vértices de grau par. Existe um ciclo Hamiltoniano de custo mínimo

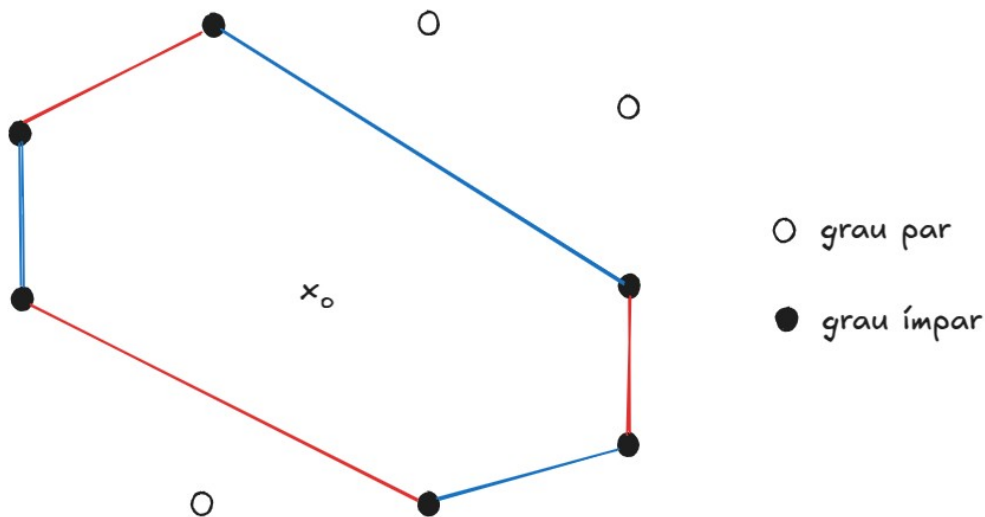
que passa apenas pelos vértices do subconjunto O , denotado por x_o , conforme ilustra a figura a seguir:



O custo do ciclo x_o é menor que o custo da solução ótima, pois ele passa por menos vértices:

$$f(x_o) \leq f(x^*)$$

Porém, note que podemos particionar o ciclo x_o em arestas alternadas de duas cores: vermelhas e azuis. Note que como o número de vértices ímpares é sempre par, o número de arestas no ciclo x_o será sempre par, e assim, temos 2 emparelhamentos distintos com o mesmo número de arestas: M' (azuis) e M'' (vermelhas).



Como o peso total dos 2 emparelhamentos é igual ao custo do ciclo x_o , temos:

$$w(M') + w(M'') = f(x_o) \leq f(x^*)$$

Se os pesos de M' e M'' forem iguais:

$$w(M') \leq \frac{f(x^*)}{2}$$

Se os pesos de M' e M'' forem diferentes, seja $\bar{M} = \min\{M', M''\}$. Então, temos:

$$w(\overline{M}) \leq \frac{f(x^*)}{2}$$

Assim, como no algoritmo de Christofides encontramos o emparelhamento M de custo mínimo entre os vértices de grau ímpar, $M = \overline{M}$ e $w(M) \leq \frac{f(x^*)}{2}$. Portanto o custo da solução final satisfaz:

$$f(x_c) = w(T_{MST}) + w(M)$$

Como $w(T_{MST}) < f(x^*)$ e $w(M) \leq \frac{f(x^*)}{2}$, temos finalmente:

$$f(x_c) \leq f(x^*) + \frac{f(x^*)}{2} = \frac{3}{2} f(x^*)$$

O algoritmo 2-otimal

Ideia: partir de uma solução inicial arbitrária e tentar melhorá-la iterativamente a partir de buscas na vizinhança local, através de operações de religações.

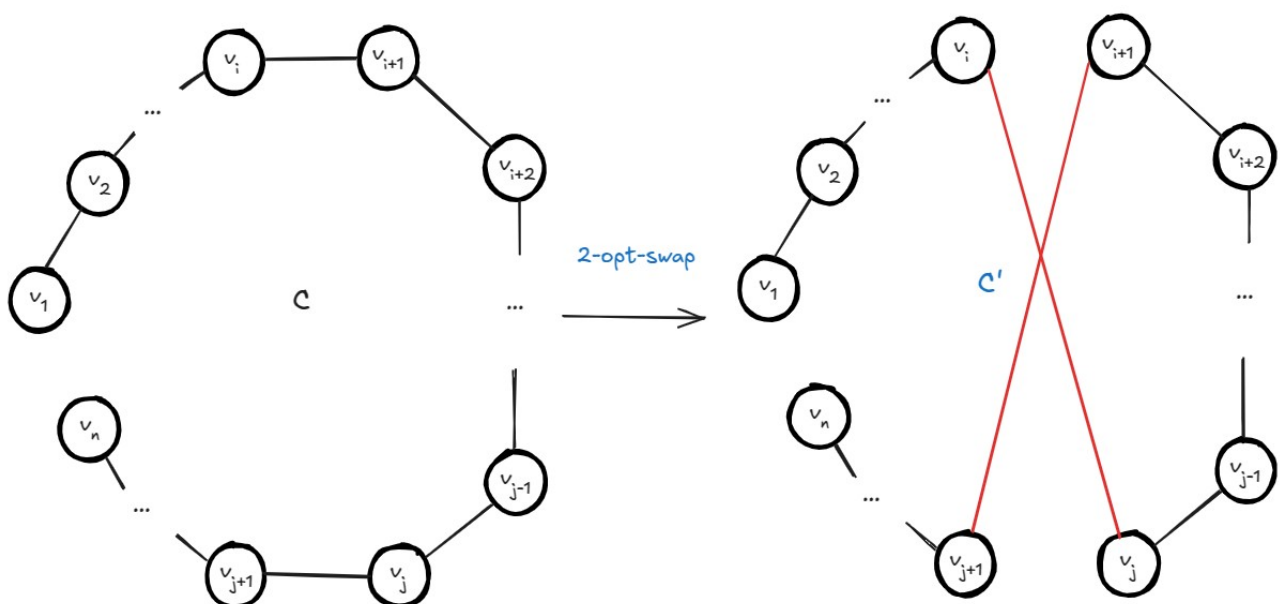
Pseudocódigo

Sem perda de generalidade, seja $C = v_1 v_2 v_3 \dots v_{n-1} v_n$ a solução inicial. Definiremos a operação Two_Opt_Swap(C) conforme a seguir:

Two-Opt-Swap(C, i, j) {

Seja C_{ij} p ciclo obtido pela religação de v_i e v_j como segue:

$$C' = v_1 v_2 \dots v_i v_j v_{j-1} v_{j-2} \dots v_{i+1} v_{j+1} \dots v_n v_1$$



return C'

}

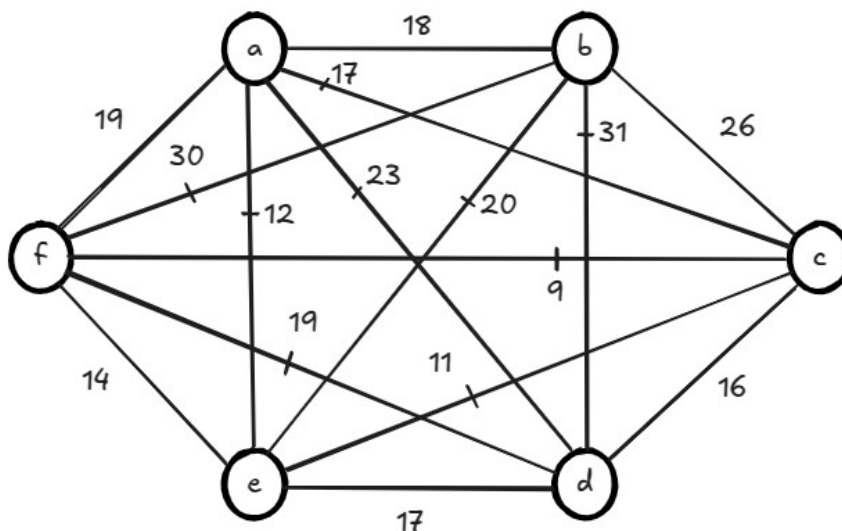
Essa operação é a base para a busca local e define como perturbamos a solução corrente na busca por uma solução melhor que esteja nas proximidades.

Note que em essência, o que a operação Two-Opt-Swap faz é primeiramente desconectar v_i de v_{i+1} e desconectar v_j de v_{j+1} e em seguida conectar com uma aresta o par de vértices v_i e v_j , além de conectar com outra aresta o par de vértices v_{i+1} e v_{j+1} . A vantagem desta operação é que ela é eficiente, uma vez que pode ser realizada com complexidade constante $O(1)$.

A seguir é apresentado o algoritmo 2-Optimal, também conhecido como 2-Opt, que é um método iterativo para melhorar uma solução inicial dada como entrada.

```
Two-Opt(C, n) {
  best = C
  improved = True
  while improved {
    improved = False
    for i = 1 to n-2 {
      for j = i+2 to n {
        C = Two-Opt-Swap(C, i, j)
        if w(C) < w(best) {
          best = C
          improved = True
        }
      }
    }
  }
  C = best
}
```

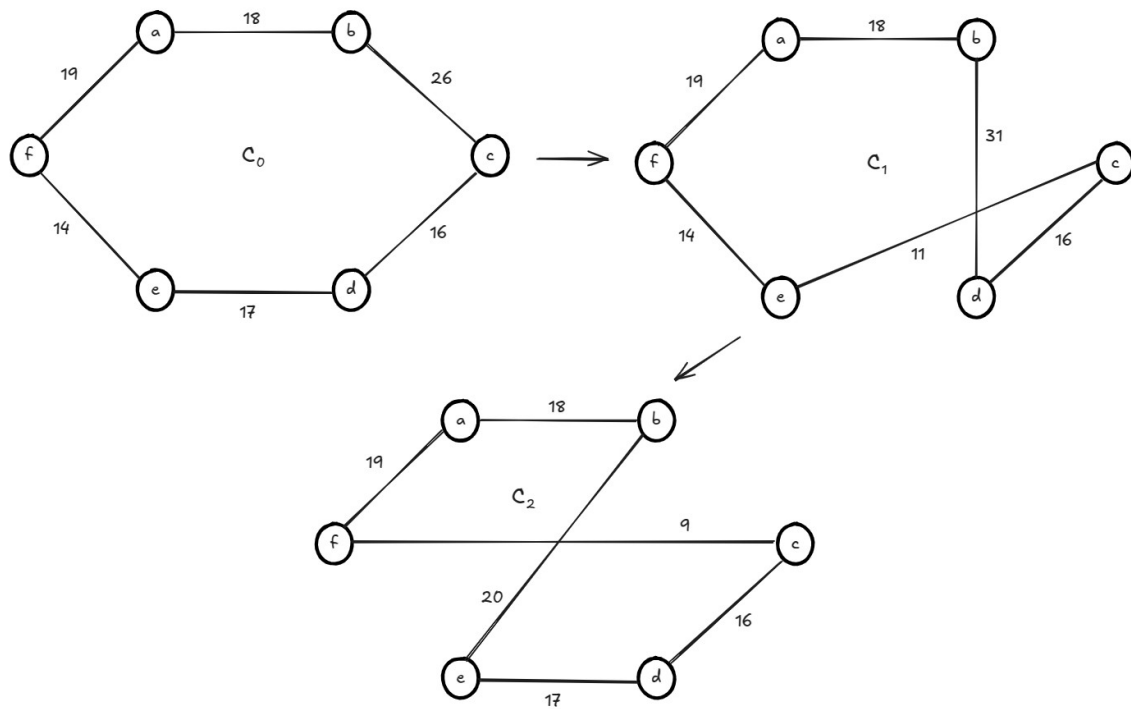
A seguir mostramos um exemplo ilustrativo da aplicação do algoritmo 2-Opt



Consideramos como solução inicial o ciclo $C = a b c d e f a$ que possui custo $w(C) = 110$.

C									W
i	j	1	2	3	4	5	6	a	
1	3	a	c	b	d	e	f	a	110
1	4	a	d	c	b	e	f	a	124
1	5	a	e	d	c	b	f	a	118
1	6	a	f	e	d	c	b	a	120
1	6	a	f	e	d	c	b	a	110
2	4	a	b	d	c	e	f	a	109 *
1	3	a	d	b	c	e	f	a	124
1	4	a	c	d	b	e	f	a	117
1	5	a	e	c	d	b	f	a	119
1	6	a	f	e	c	d	b	a	109
2	4	a	b	c	d	e	f	a	110
2	5	a	b	e	c	d	f	a	103 *
1	3	a	e	b	c	d	f	a	112
1	4	a	c	e	b	d	f	a	117
1	5	a	d	c	e	b	f	a	119
1	6	a	f	d	c	e	b	a	103
2	4	a	b	c	e	d	f	a	110
2	5	a	b	d	c	e	f	a	109
2	6	a	b	f	d	c	e	a	104
3	5	a	b	e	d	c	f	a	99 *
1	3	a	e	b	d	c	f	a	107
1	4	a	d	e	b	c	f	a	114
1	5	a	c	d	e	b	f	a	119
1	6	a	f	c	d	e	b	a	99
2	4	a	b	d	e	c	f	a	105
2	5	a	b	c	d	e	f	a	110
2	6	a	b	f	c	d	e	a	102
3	5	a	b	e	c	d	f	a	103
3	6	a	b	e	f	c	d	a	100
4	6	a	b	e	d	f	c	a	100

Após toda uma varredura ao redor da solução com custo $w(C) = 99$, não foi possível encontrar nenhuma solução melhor. PARE!



Análise da complexidade

É possível realizar a operação $\text{Two_Opt_Swap}(C, i, j)$ com complexidade $O(1)$. Sendo assim, em uma rodada do algoritmo 2-Opt, temos o seguinte número de operações:

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 1 = (n-2) + (n-3) + (n-4) + \dots + (n-(n-1)) = (n+n+n+\dots+n) - (2+3+4+\dots+(n-1))$$

Note que o primeiro somatório é justamente $n(n-2)$. Já o segundo somatório é igual a:

$$\left(\sum_{i=1}^{n-1} i \right) - 1 = \frac{n(n-1)}{2} - 1 = \frac{n^2 - n - 2}{2}$$

Logo, temos:

$$T(n) = n^2 - 2n - \left(\frac{n^2}{2} - \frac{n}{2} - 1 \right) = \frac{n^2}{2} - \frac{3n}{2} + 1 = \frac{n^2 - 3n + 2}{2}$$

o que é $O(n^2)$. Portanto, supondo que o loop WHILE mais externo execute K vezes, teremos uma complexidade $O(Kn^2)$. Como $K \ll n$ (K é uma constante muito menor que n), podemos dizer que a complexidade do algoritmo 2-Opt é $O(n^2)$.

Teorema: A solução obtida pelo algoritmo 2-Opt em uma instância do TSP métrico satisfaz:

$$f(x_{2-opt}) \leq (\log n) f(x^*)$$

Iremos omitir a prova deste resultado, mas de qualquer forma, note que a qualidade da solução depende do número de vértices do grafo. Quanto mais vértices o grafo possui, menos garantias de que estamos obtendo soluções próximas à solução ótima.

Uma estratégia comum consiste em utilizar os algoritmos Nearest Neighbor, Twice-Around e Christofides para gerar soluções iniciais para o 2-Opt, na tentativa de melhorá-las ao menos um pouco.

Existem algoritmos exatos para o TSP: um deles é o método da força bruta, que testa todas as combinações possíveis de ciclos de comprimento n e possui complexidade $O(n!)$, sendo inviável para a maioria dos problemas. O algoritmo de Held-Karp utiliza uma abordagem baseada em programação dinâmica para resolver o TSP e possui complexidade $O(2^n n^2)$ (exponencial), o que ainda é inviável para grafos com um grande número de vértices. Por essa razão, em geral, utiliza-se os algoritmos aproximados na solução de problemas reais.

Coloração de vértices

Ideia: dado um grafo $G = (V, E)$, particionar o conjunto de vértices V em conjuntos independentes.

Def: Seja $G = (V, E)$ um grafo e $P_1, P_2, \dots, P_n$ uma partição de V . Dizemos que P_i é um subconjunto independente se quaisquer 2 vértices de P_i não compartilham arestas (não são vizinhos).

Objetivo: particionar o conjunto V no menor número possível de subconjuntos independentes

Def: Uma k -coloração de G atribui uma de K cores a cada vértice de G , de modo que a vértices adjacentes sempre são atribuídas cores/rótulos diferentes.

P_1 : subconjunto dos vértices de cor 1

P_2 : subconjunto dos vértices de cor 2

...

P_k : subconjunto dos vértices de cor k

Pergunta: dado um grafo $G = (V, E)$, $\exists$ uma k -coloração para G (com $k > 2$)? Trata-se de um problema NP-Completo

Em outras palavras queremos responder se dado G é possível encontrar 2, 3, 4, etc. conjuntos independentes.

Def: O número mínimo k para o qual existe uma coloração de G é o número cromático de G , denotado por $\chi(G)$ (é um atributo do grafo).

Propriedades: nos ajudam a limitar os valores de $\chi(G)$ em problemas reais: limites inferiores e superiores

a) Se $|V| = n$, então $\chi(G) \leq n$

i) Se G é o K_n , então $\chi(G) = n$

ii) Se G contém K_m como subgrafo, então $\chi(G) \geq m$

Teorema: G é bipartido $\Leftrightarrow \chi(G) = 2$

Teorema: Seja $G = (V, E)$ e $\Delta(G) = \max\{d(v) : v \in V\}$ (grau máximo). Então, $\chi(G) \leq \Delta(G) + 1$
Prova:

1. Inicie colorindo os vértices em uma sequência arbitrária.

2. Um vértice deve sempre receber uma cor livre, ou seja, diferente da cor dos vizinhos.
3. Seja o grau máximo de um vértice v em G , $\Delta(G)$.
4. Pelo princípio da casa dos pombos, sempre irá existir uma cor livre no conjunto $\{1, 2, 3, \dots, \Delta(G)+1\}$.

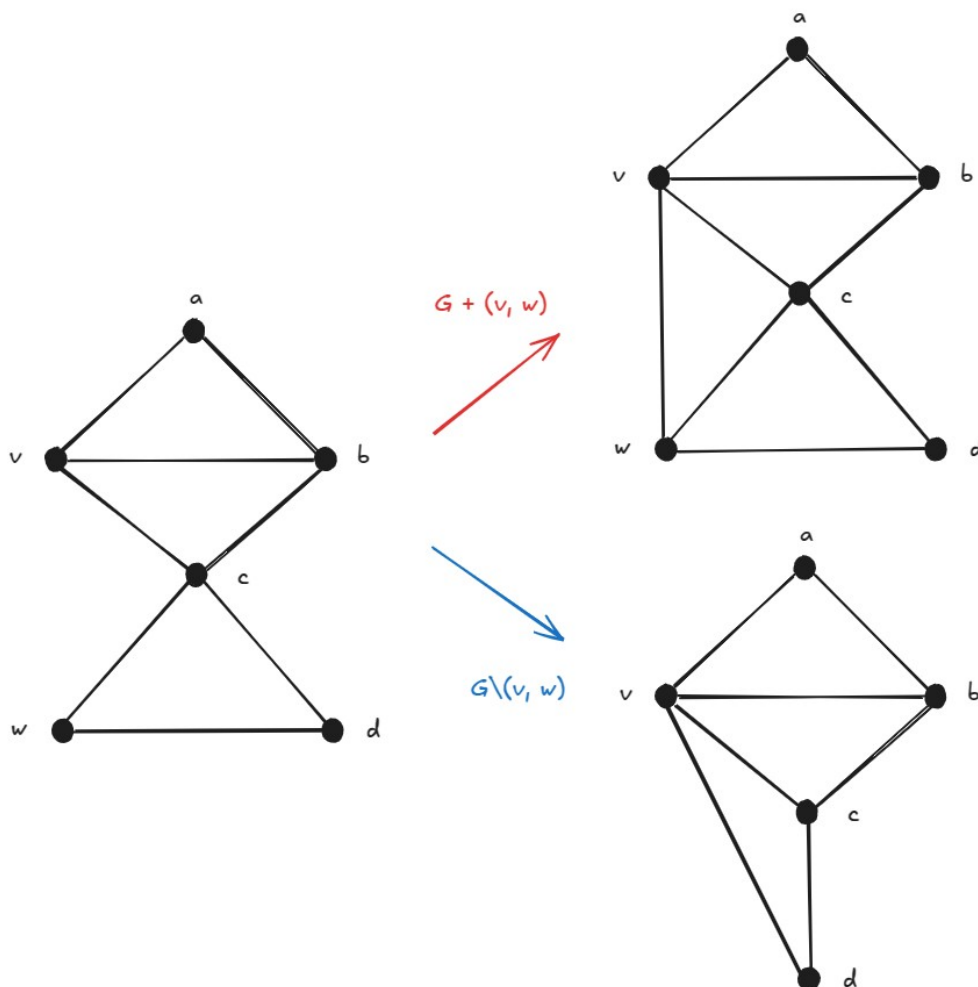
Teorema: Seja $G = (V, E)$ um grafo conexo com $\Delta(G) \geq 3$. Se G não é completo, $\chi(G) \leq \Delta(G)$.

A seguir veremos como podemos definir um algoritmo exato para encontrar o número cromático de um grafo G .

Seja $G = (V, E)$ o grafo de entrada. Se $G = K_n$ então $\chi(G) = n$. Caso contrário, $\exists v, w \in V$ não adjacentes. A ideia é realizar duas alterações estruturais em G :

1. $\alpha_{v,w}(G) = G + (v, w)$ - adição da aresta (v, w) em G
2. $\beta_{v,w}(G) = G \setminus (v, w)$ - condensação dos vértices v e w

onde $G \setminus (v, w)$ é o grafo resultante da fusão dos vértices v e w e posterior eliminação de loops e arestas paralelas que venham a surgir. A figura a seguir ilustra um exemplo.



O método exato de coloração de vértices inicia verificando se $G = K_n$. Em caso positivo, $\chi(G) = n$. Caso contrário, determina-se $\alpha_{v,w}(G)$ e $\beta_{v,w}(G)$. O número cromático $\chi(G)$

será o mínimo entre os números cromáticos de $\alpha_{v,w}(G)$ e $\beta_{v,w}(G)$, que devem ser calculados recursivamente. Através da recursão, chegaremos em um grafo completo, onde $\chi(G)$ é facilmente computado (caso base). O resultado a seguir fundamenta o algoritmo exato em questão.

Teorema: Seja $G = (V, E)$ um grafo não completo e $v, w \in V$ um par de vértices não adjacentes. Então, o número cromático de G satisfaz:

$$\chi(G) = \min\{\chi(\alpha_{v,w}(G)), \chi(\beta_{v,w}(G))\}$$

Prova:

1. Suponha uma coloração ótima C de G .
2. Se os vértices v, w possuem cores distintas em C , então a aresta (v, w) pode ser adicionada em G sem alterar C . Neste caso, $\chi(G) = \chi(\alpha_{v,w}(G))$.
3. Se os vértices v, w possuem cores iguais a $c_i \in C$, então, ao fundirmos v, w em um único vértice z e atribuímos a cor c_i a z mantendo as demais cores, temos $\chi(G) = \chi(\beta_{v,w}(G))$.
4. Portanto, $\chi(G)$ deve ser o menor valor entre $\chi(\alpha_{v,w}(G))$ e $\chi(\beta_{v,w}(G))$.

Note que $\chi(G)$ é igual ao número de vértices do menor grafo completo obtido através das operações $\alpha_{v,w}(G)$ e $\beta_{v,w}(G)$.

O método a seguir define um algoritmo exato para o problema da coloração de vértices.

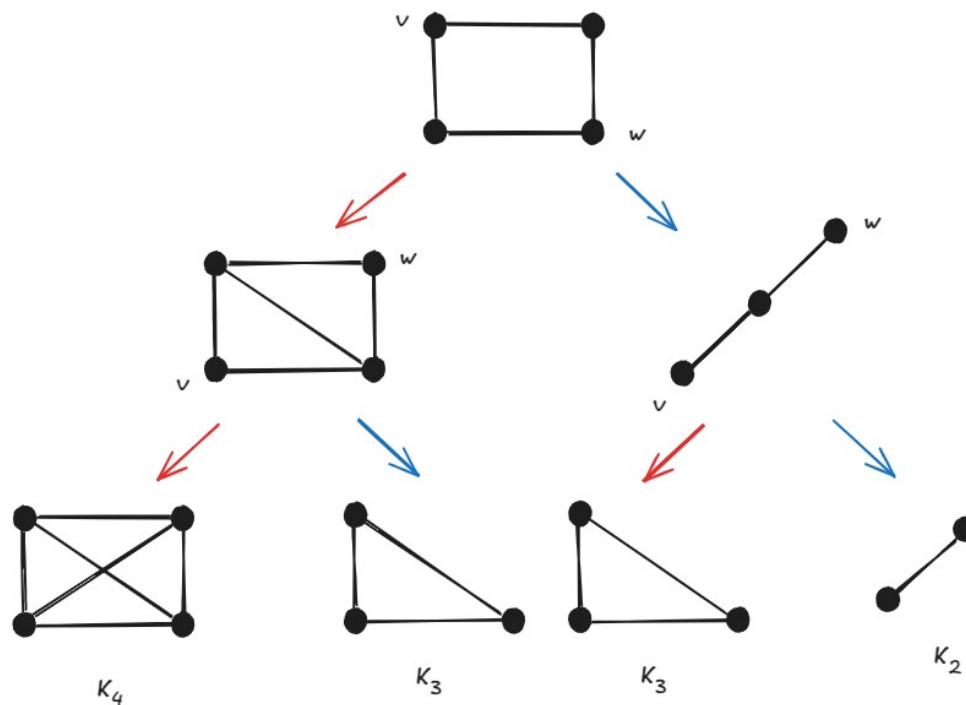
```

Chromatic_Number(G) {
    n = |V|
    if G == K_n
        χ = min{χ, n}
    else {
        Encontre um par de vértices não adjacentes v, w em G
        Chromatic_Number(αv,w(G))
        Chromatic_Number(βv,w(G))
    }
}

```

$\chi = n$
 Chromatic_Number(G)

O algoritmo acima dispara um processo recursivo que constrói uma árvore binária em que cada nó corresponde a um grafo: se G é um nó corrente, seu filho a esquerda é $\alpha_{v,w}(G)$ e seu filho a direita é $\beta_{v,w}(G)$. As folhas da árvore, que indicam a condição de parada da recursão, serão sempre grafos completos. A figura a seguir ilustra um exemplo simples.



Portanto, o número cromático de G é $\chi(G)=2$.

Análise da complexidade

Note que o número máximo de grafos a serem examinados é igual ao número de nós total na árvore binária. Como em cada nível ligamos/fundimos apenas 2 vértices, a profundidade da árvore é proporcional ao número de arestas no grafo complementar de G , denotado por m' .

$$2^0 + 2^1 + 2^2 + \dots + 2^{m'} = \sum_{i=0}^{m'} 2^i$$

Note que $2^{i+1} = 2 \cdot 2^i = 2^i + 2^i$, o que nos leva a $2^i = 2^{i+1} - 2^i$. Dessa forma temos:

$$\sum_{i=0}^{m'} (2^{i+1} - 2^i) = 2^{m'+1} - 2^0 = 2^{m'+1} - 1$$

ou seja, a complexidade é exponencial, $O(2^{m'})$. Portanto, esse algoritmo, torna-se inviável para grafos com muito vértices. A seguir iremos discutir um algoritmo aproximado que utiliza uma estratégia gulosa para encontrar o número cromático de G .

Do ponto de vista da classificação dos problemas em computação, encontrar o número cromático $\chi(G)$ de grafos arbitrários é um problema NP-Hard. Alguns fatos relevantes:

1. Não se conhece algoritmo polinomial que colore um grafo arbitrário com exatamente $\chi(G)$ cores.
2. Existe um algoritmo polinomial para coloração de grafos que colore qualquer grafo G com no máximo $O\left(\frac{n}{\log n}\right)\chi(G)$ cores (um pouco acima do valor mínimo).

Abordagem gulosa para coloração de vértices

Suponha que os vértices estejam ordenados em ordem decrescente dos graus, ou seja, temos $v_1, v_2, \dots, v_n$ e seus respectivos graus $\Delta = d_1 \geq d_2 \geq \dots \geq d_n$

Teorema: $\chi(G) = \max_{1 \leq i \leq n} \min\{d_i + 1, i\}$

Prova:

1. No instante em que colorimos v_i (i-ésimo vértice da sequência), note que:

- a) No máximo $i - 1$ cores foram utilizadas em seus vizinhos, pois antes dele foram exatamente $i - 1$ vértices.
- b) Mas também, no máximo d_i cores são utilizadas pelos vizinhos, uma vez que o grau de v_i é justamente d_i

Esse resultado fundamenta o algoritmo de Welsh & Powell, utilizado para colorir um grafo e aproximar o verdadeiro valor do número cromático de G , e que será apresentado a seguir.

```
Welsh_Powell(G) {
    Ordene os vértices em ordem decrescente de graus
    for each  $v_i \in V$ 
         $C_i = [1, 2, \dots, i]$     # Lista das cores do vértice  $v_i$ 
    for  $i = 1$  to  $n$  {
         $color = C_i[1]$           # Escolhe a primeira cor da lista
         $v_i.c = color$           # Colore  $v_i$  com a cor escolhida
        for each  $v_j$  in  $N(v_i)$ 
            # Remove a cor da listas dos vizinhos
             $C_j = C_j - color$ 
        }
         $\chi = 0$ 
        for each  $v_i \in V$  {
            if  $v_i.c > \chi$ 
                 $\chi = v_i.c$ 
        }
    }
    return  $\chi$ 
}
```

Análise da complexidade

1. Note que a ordenação dos vértices pode ser realizada em $O(n \log n)$

2. A criação das listas de cores envolve trabalhar com n listas de tamanhos variáveis. Para o i -ésimo vértice, a lista de cores tem tamanho i , de modo que o número de operações é dado por:

$$1 + 2 + 3 + 4 + \dots + n = \sum_{i=1}^n i$$

3. Mas é possível reduzir esse custo, sabendo que o número cromático em um grafo que não é o K_n é limitado superiormente pelo maior grau. Seja K o maior grau de um vértice em G , então basta que as listas de cores contenham K cores distintas e não n , ou seja, o número de operações gastas é:

$$1+2+3+\dots+K+K+K \leq nK$$

Portanto, a etapa de criação das listas de cores tem complexidade $O(nK)$

4. O loop principal do algoritmo percorre todo o vértice v de G uma vez e para cada vértice, acessamos os seus $d(v)$ vizinhos. Se utilizarmos um array estático, podemos apagar a cor $color$ da lista C_i em $O(1)$. Logo, temos que a complexidade desse loop é:

$$d(v_1)+d(v_2)+\dots+d(v_n)=2m \text{ , ou seja, } O(m) \text{ .}$$

5. Por fim, o ultimo loop serve apenas para encontrar o maior rótulo utilizado como cor, o que é equivalente a encontrar o máximo do vetor. Assim, o custo é $O(n)$.

Portanto, o custo total é: $O(n \log n)+O(nK)+O(m)+O(n)$. Note que no pior caso, $m=O(n^2)$ o que nos leva a conclusão de que a complexidade do algoritmo Welsh and Powell é $O(n^2)$.

A seguir apresentaremos alguns exemplos de problemas que podem ser resolvido através da coloração de vértices.

O Problema da Alocação de Frequências

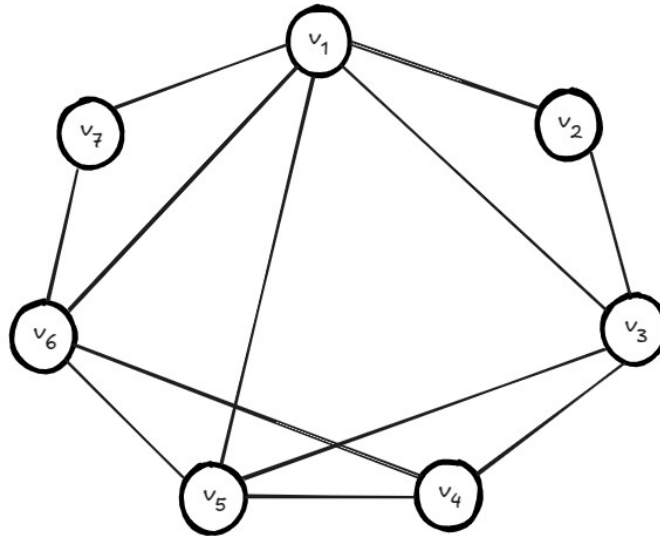
Considere 7 antenas de transmissão de sinais de telefonia. Sabe-se que devido a fatores como localização geográfica e tipo de serviço oferecido, as antenas possuem regiões de influência de modo que tem-se a seguinte matriz de interferências:

	v_1	v_2	v_3	v_4	v_5	v_6	v_7
v_1		x	x		x	x	x
v_2	x		x				
v_3	x	x		x	x		
v_4			x		x	x	
v_5	x		x	x		x	
v_6	x			x	x		x
v_7	x					x	

Essa matriz indica quando duas antenas interferem uma na outra com um x. Pergunta-se:

- Qual o menor número de frequências necessárias para que a comunicação se dê de forma correta?
- As antenas devem operar em qual configuração de frequência?

1º passo: Devemos construir o grafo de incompatibilidade. Neste grafo, dois vértices devem ser ligados por uma aresta se existe interferência entre as duas antenas.

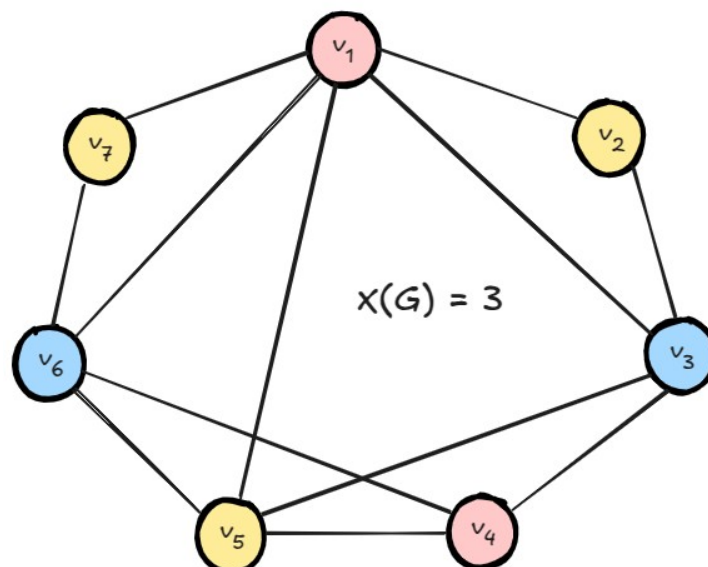


2º passo: Ordenar os vértices em ordem decrescente de graus: $v_1, v_3, v_5, v_6, v_4, v_2, v_7$

3º passo: Definir as listas de cores para cada vértice

4º passo: Colorir os vértices iterativamente removendo as cores usadas das listas dos vizinhos.

	$C_1=\{1\}$	$C_3=\{1,2\}$	$C_5=\{1,2,3\}$	$C_6=\{1,2,3,4\}$	$C_4=\{1,2,3,4,5\}$	$C_2=\{1,2,3,4,5\}$	$C_7=\{1,2,3,4,5\}$
$v_1 \leftarrow 1$	x						
$v_3 \leftarrow 2$		x					
$v_5 \leftarrow 3$			x				
$v_6 \leftarrow 2$				x			
$v_4 \leftarrow 1$					x		
$v_2 \leftarrow 3$						x	
$v_7 \leftarrow 3$							x



A aproximação para o número cromático obtida pelo algoritmo guloso é 3, o que, neste caso, corresponde ao verdadeiro valor de $\chi(G)$. As partições obtidas são:

$$P_1 = \{v_1, v_4\}$$

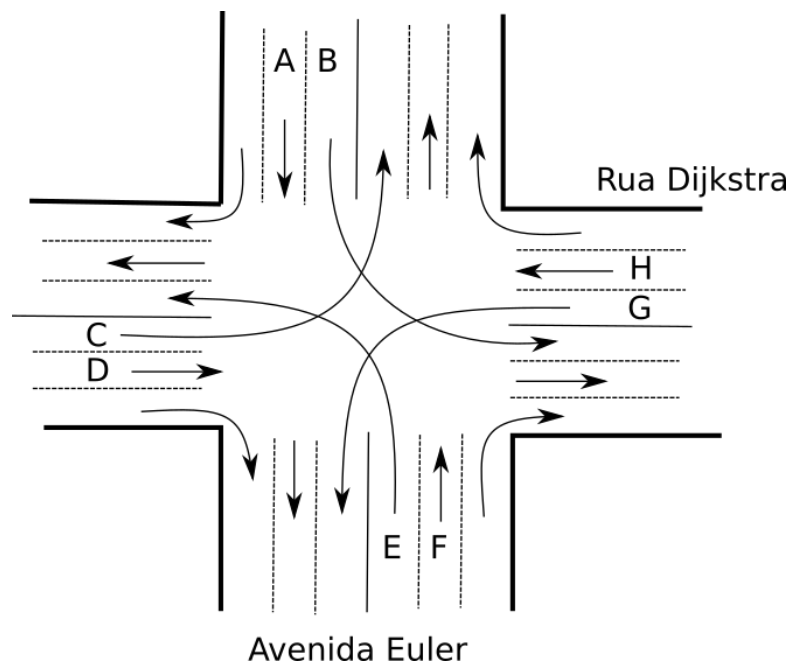
$$P_2 = \{v_3, v_6\}$$

$$P_3 = \{v_2, v_5, v_7\}$$

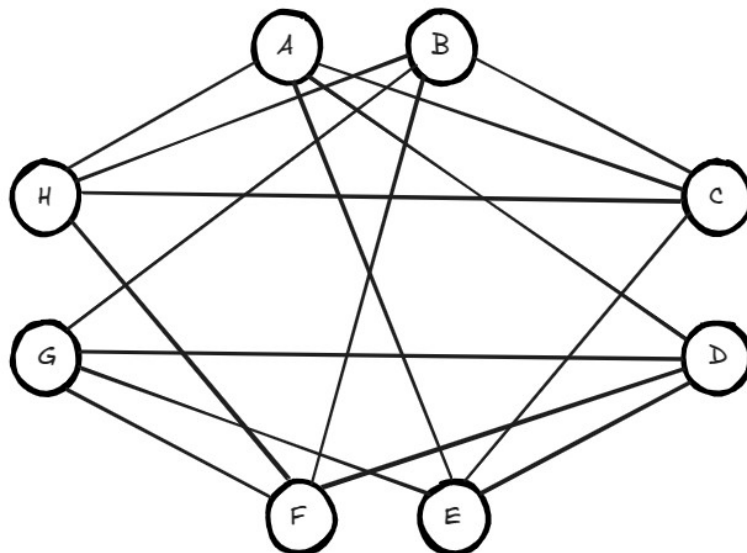
O problema do semáforo

O objetivo deste problema consiste em sincronizar os semáforos de um cruzamento utilizando coloração de vértices.

Dado o cruzamento de ruas a seguir, desenvolver um semáforo com o menor número de tempos possíveis. Qual deve ser o modo de funcionamento do mesmo.



1º passo: Consiste em gerar o grafo de incompatibilidade. Para isso cada uma das faixas de A a H deve ser um vértice e a cada possível cruzamento de faixas deve existir uma aresta (ou seja, deve existir uma aresta sempre que for possível ocorrer uma batida). O grafo resultante é ilustrado pela figura a seguir.



2º passo: Consiste na aplicação do algoritmo guloso (Welsh & Powell) para colorir o grafo acima utilizando o menor número de cores possíveis. Como o grau máximo é 4 e o grafo não é completo, temos garantia de que o número cromático é menor ou igual a 4. A execução desse passo é deixada como exercício para o leitor.

O problema do aeroporto

Uma nova empresa aérea irá começar a operar com 7 aeronaves seguindo a programação de vôos (de A a G) definida pela tabela abaixo, sendo que todos os vôos partem de São Paulo e visitam cada uma das cidades listadas nas rotas na sequência em que elas aparecem:

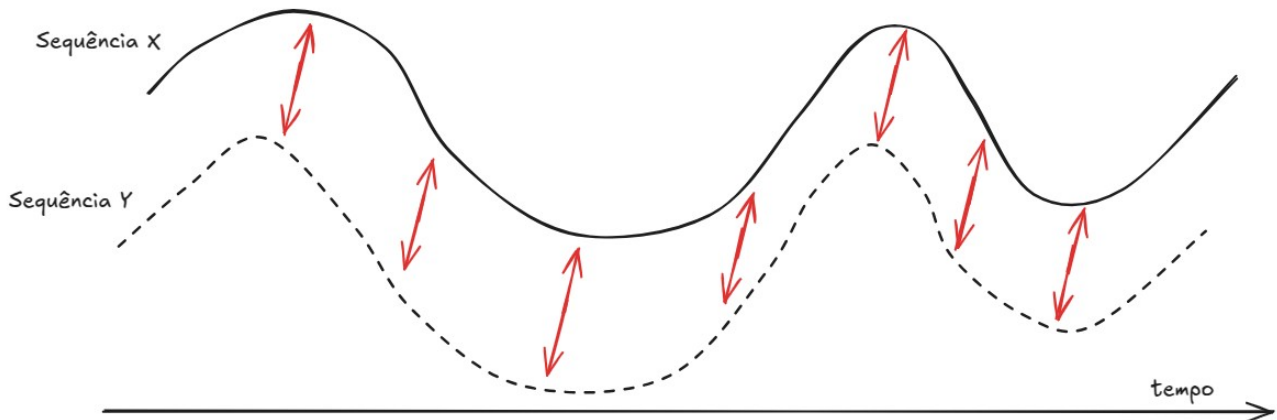
Vôo	Rota
A	Florianópolis – Rio de Janeiro – Natal – Fortaleza
B	Curitiba - Campinas – Ribeirão Preto – Fortaleza
C	Belo Horizonte – Natal – Fortaleza – Manaus
D	Belo Horizonte – São José do Rio Preto – Rio de Janeiro
E	Belo Horizonte – Recife – Natal
F	Brasília – Ribeirão Preto – Fortaleza
G	Brasília - Presidente Prudente – Campinas

Devido ao número limitado de aeronaves, o diretor da companhia não quer mais de um vôo por dia visitando uma determinada cidade, ou seja, se dois vôos passam pela cidade X eles devem obrigatoriamente não estar alocados para o mesmo dia. Sendo assim, modelando o problema com um grafo, e utilizando coloração de vértices, determine o número mínimo de dias necessários para que a empresa opere de acordo com a sua política de funcionamento.

"Solutions are not found by pointing fingers; they are reached by extending hands."
-- Aysha Taryam

Apêndice A: Programação dinâmica e o algoritmo DTW

Dynamic time warping (DTW), ou sincronização temporal dinâmica, é um algoritmo bem conhecido para encontrar um alinhamento ideal entre duas sequências dadas (dependentes do tempo) sob certas condições. Intuitivamente, as sequências são distorcidas de maneira não linear para combinar umas com as outras. Originalmente, o DTW foi usado para comparar diferentes padrões de fala no reconhecimento automático. Em campos como mineração de dados e recuperação de informação, o DTW foi aplicado com sucesso para lidar automaticamente com deformações de tempo e diferentes velocidades associadas a séries temporais. A figura a seguir ilustra o processo de alinhamento entre duas sequências temporais.



O objetivo do algoritmo DTW é comparar duas sequências:

$X = (x_1, x_2, \dots, x_n)$ de tamanho n e

$Y = (y_1, y_2, \dots, y_m)$ de tamanho m

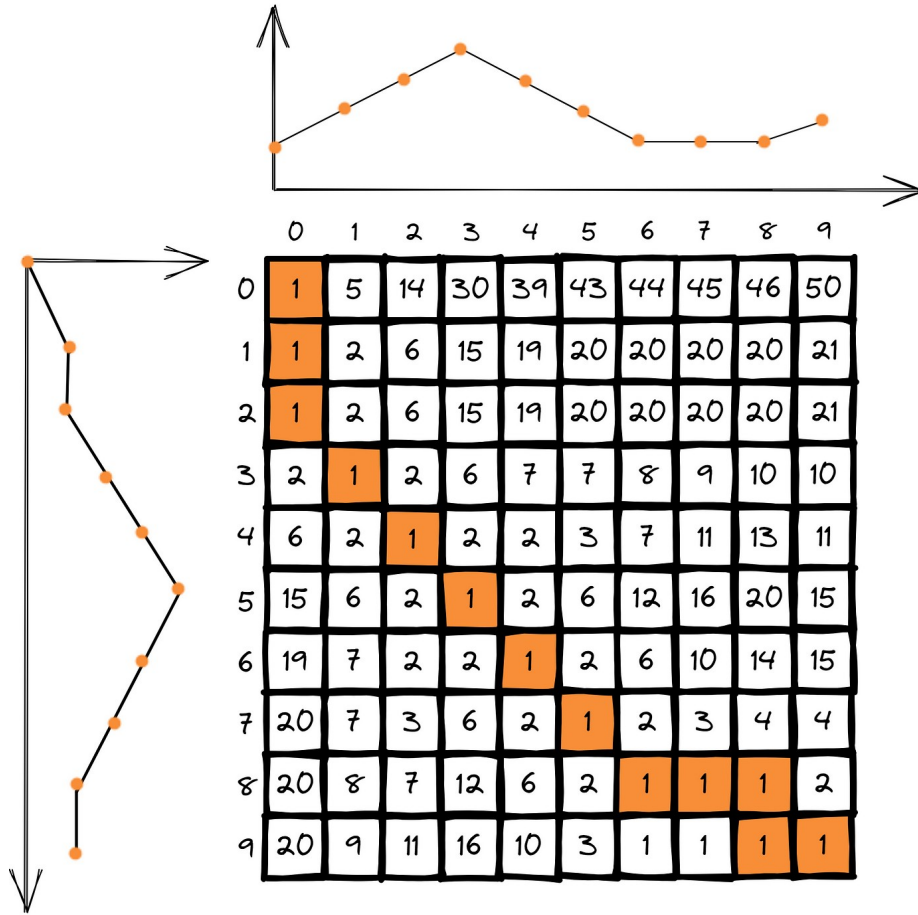
Essas sequências podem ser sinais discretos, séries temporais ou mesmo sequências de atributos amostradas em pontos equidistantes do tempo. A ideia é calcular o match ideal entre as duas sequências a partir da construção de uma matriz de atributos F de dimensões n por m (matriz), tal que para todo $i = 1 \dots n$ e todo $j = 1 \dots m$, $(x_i, y_j) \in F$.

Para comparar diferentes atributos $(x, y) \in F$ é preciso uma função de custo local, ou medida de distância local $c: F \times F \rightarrow \mathbb{R}^+$. Uma escolha comum para essa função é a distância de Manhattan ou norma $L1$:

$$c(x, y) = |x - y|$$

Essa função deve retornar um valor baixo se x e y são similares, e retornar um valor elevado se x e y são bem distintos. Ao avaliar essa função de custo para todos os pares possíveis, obtemos uma matriz de custos $C \in \mathbb{R}^{n \times m}$ tal que $C(i, j) = c(x_i, y_j)$.

Então, o objetivo consiste em encontrar um alinhamento entre X e Y com custo total mínimo. Intuitivamente, tal alinhamento ótimo corre ao longo de um “vale” de baixo custo dentro da matriz de custos C . A figura a seguir é a visualização de uma matriz de custos. A ideia do algoritmo DTW consiste em gerar uma matriz de custos acumulados e encontrar um caminho pelos vales dessa matriz. Esse caminho é o que representa o menor caminho *warp* entre as duas sequências.



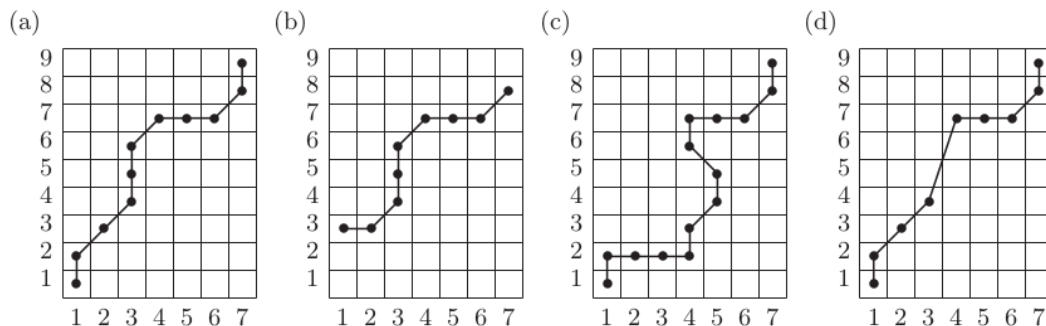
Definição: um caminho warp (warp path) é uma sequência $p=(p_1, p_2, \dots, p_L)$ com:

$$p_l=(n_l, m_l) \in [1..n] \times [1..m] \quad \text{para } l \in [1..L]$$

satisfazendo as seguintes condições:

- i) Condição de borda: $p_1=(1,1)$ e $p_L=(n,m)$ (inicia no canto superior (ou inferior) esquerdo e termina no canto inferior (ou superior) direito)
- ii) Condição de monotonicidade: $n_1 < n_2 < \dots < n_L$ e $m_1 < m_2 < \dots < m_L$ (caminho não pode ter desvios para esquerda e para baixo)
- iii) Condição de tamanho do passo: $p_{l+1} - p_l \in \{(1,0), (0,1), (1,1)\}$ para $l \in [1..L-1]$ (o tamanho do passo é sempre igual a 1, mover uma posição para cima (ou baixo), direita (ou esquerda) ou diagonal superior (inferior))

A figura a seguir ilustra um caminho warp válido (a) e outros três caminhos warp inválidos.



O caminho *warp* (b) não satisfaz a condição de borda. O caminho *warp* (c) não satisfaz a condição de monotonicidade. Já o caminho *warp* (c) não satisfaz a condição de tamanho do passo.

O custo total $c_p(X, Y)$ de um caminho *warp* p entre X e Y é definido em termos dos custos locais como:

$$c_p(X, Y) = \sum_{l=1}^L c(x_{i_l}, y_{j_l})$$

Além disso, o caminho *warp* ótimo entre X e Y é um caminho *warp* p^* com mínimo custo total dentre todos os caminhos *warp*. A distância DTW(X, Y) entre X e Y é definida como o custo do caminho ótimo p^* :

$$DTW(X, Y) = c_{p^*}(X, Y) = \min \{ c_p(X, Y) \mid p \text{ é um caminho warp} \}$$

Para determinar o caminho *warp* ótimo p^* , podemos testar todos os possíveis caminhos *warp* entre X e Y, mas esse procedimento levaria a um algoritmo com complexidade exponencial nos valores de n e m, o que seria inviável. Para entender a razão, vamos definir o que representa o número de Delannoy.

O número de Delannoy é um conceito matemático da combinatória que conta o número de caminhos em uma grade retangular de coordenadas inteiras, permitindo movimentos específicos. Ele tem aplicações em áreas como teoria dos números, algoritmos de alinhamento de sequências (como o DTW) e até mesmo em física teórica.

Def: O número de Delannoy pode ser definido por uma relação de recorrência:

$$D(m, n) = D(m-1, n) + D(m, n-1) + D(m-1, n-1)$$

com condições iniciais:

$$\begin{aligned} D(0, 0) &= 1, \\ D(k, 0) &= 1 \text{ para qualquer } k \geq 0, \\ D(0, k) &= 1 \text{ para qualquer } k \geq 0. \end{aligned}$$

Também existe uma fórmula fechada usando coeficientes binomiais, dada por:

$$D(m, n) = \sum_{k=0}^{\min(m, n)} \binom{m}{k} \binom{n}{k} 2^k$$

Pode-se mostrar que o número de Delannoy, cresce exponencialmente, proporcional a 2^{m+n} , o que faz a busca via força bruta por todos os caminhos *warp* um problema computacionalmente inviável.

Apresentamos a seguir um algoritmo baseado em programação dinâmica que possui complexidade $O(nm)$. Iremos definir as sequências a seguir:

$$\begin{aligned} X(1:i) &= (x_1, x_2, \dots, x_i) & \text{para } i = 1 \dots n \\ Y(1:j) &= (y_1, y_2, \dots, y_j) & \text{para } j = 1 \dots m \end{aligned}$$

e também o elemento (n, m) da matriz de custo acumulados como:

$$D(n, m) = DTW(X(1:i), Y(1:j))$$

Os valores de $D(n, m)$ definem uma matriz D de dimensões $n \times m$, chamada de matriz de custos acumulados, de modo que $D(n, m) = \text{DTW}(X, Y)$, ou seja, ao definir o último elemento dessa matriz, teremos o valor da distância DTW entre X e Y . O resultado a seguir mostra que podemos calcular os elementos da matriz de custos acumulados de maneira direta, propagando o menor custo.

Teorema: A matriz de custos acumulados D satisfaz as seguintes identidades:

$$D(i, 1) = \sum_{k=1}^i c(x_k, y_1) \quad \text{para } i = 1 \dots n \text{ (na primeira coluna é a soma dos anteriores)}$$

$$D(1, j) = \sum_{k=1}^j c(x_1, y_k) \quad \text{para } j = 1 \dots m \text{ (na primeira linha é a soma dos anteriores)}$$

$$D(i, j) = \min\{D(i-1, j-1), D(i-1, j), D(i, j-1)\} + c(x_i, y_j) \quad \text{para } i = 2 \dots n \text{ e } j = 2 \dots m$$

de modo que $\text{DTW}(X, Y) = D(n, m)$ pode ser computada em $O(nm)$.

Para que a recursão seja válida em todas as células da matriz, podemos definir bordas (linha e coluna zero) fazendo:

$$\begin{aligned} D(i, 0) &= \infty & \text{para } i = 1 \dots n \text{ (coluna zero preenchida com infinitos)} \\ D(0, j) &= \infty & \text{para } j = 1 \dots m \text{ (coluna zero preenchida com infinitos)} \\ D(0, 0) &= 0 \end{aligned}$$

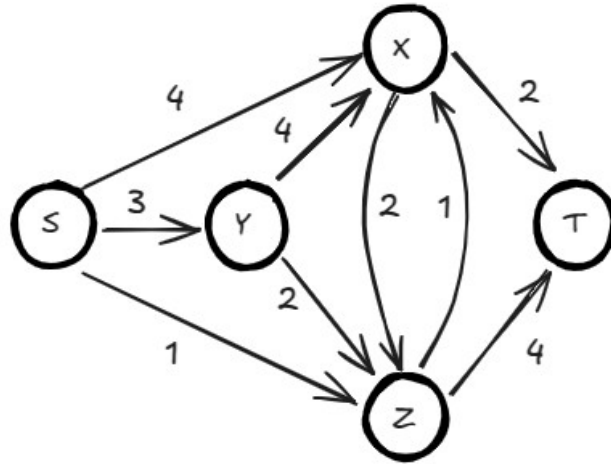
O algoritmo a seguir mostra como podemos calcular a distância DTW entre duas sequências.

```
DTW_Distance(X: array[1..n], Y: array[1..m]) {
    DTW = array[0..n, 0..m]
    # Inicia com custos infinitos
    for i = 0 to n {
        for j = 0 to m
            DTW[i, j] = ∞
    }
    # Ponto de partida inicia com zero
    DTW[0, 0] = 0
    # Loop principal
    for i = 1 to n {
        for j = 1 to m {
            cost = d(X[i], Y[j])
            DTW[i, j] = cost + min(DTW[i-1, j], DTW[i, j-1], DTW[i-1, j-1])
        }
    }
    return DTW[n, m]
}
```

É fácil notar que a complexidade do algoritmo em questão é $O(nm)$.

Apêndice B: Algoritmos gulosos e o problema do fluxo máximo

Motivação: maximizar o fluxo que pode ser produzido pela fonte s e consumido pelo terminal t .



Consideraremos grafos direcionados $G = (V, E, c)$ onde $c: E \rightarrow \mathbb{N}^+$ é a capacidade de uma aresta. Capacidade é a quantidade máxima de informação que pode ser transmitida por uma aresta. Iremos classificar os vértices de G como:

$s \in V$: source (gerador de fluxo: não entra nada, apenas sai)

$t \in V$: terminal ou sink (absorve fluxo: não sai nada, apenas entra)

$\forall v \in V - \{s, t\}$: nós internos (intermediários)

Em nossos exemplos iremos considerar apenas um único source e um único terminal

Def: Um fluxo s - t é uma função $f: E \rightarrow \mathbb{N}^+$ (associa um inteiro a cada aresta) que satisfaz:

i) $\forall e \in E, f(e) \leq c(e)$ (restrição de capacidade)

ii) $v(f) = \sum_{e \in O(s)} f(e) = \sum_{e \in I(t)} f(e)$ (fluxo gerado na fonte é igual ao fluxo consumido no terminal)

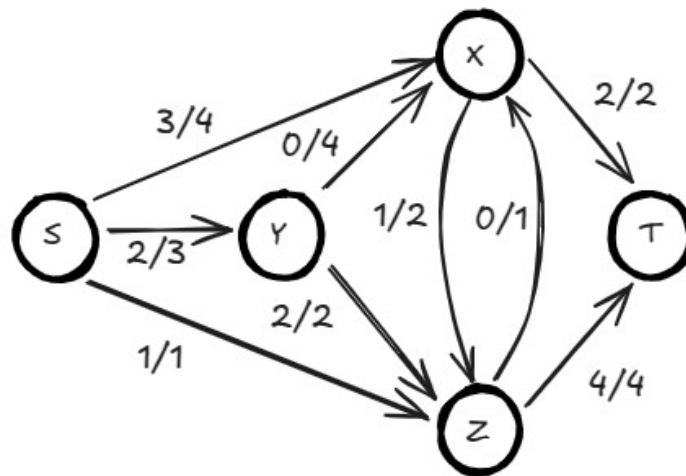
iii) $\forall v \in V - \{s, t\}$ (nó interno)

$$\sum_{e \in I(v)} f(e) = \sum_{e \in O(v)} f(e) \quad (\text{conservação do fluxo})$$

iv) Limite superior: para o valor de fluxo máximo que pode circular em G

$$v(f) = \sum_{e \in O(s)} c(e)$$

Como exemplo, verifique que um fluxo válido para o grafo anterior é representado pela figura a seguir.



Basta verificar as propriedades:

i) OK, pode-se ver facilmente que $\forall e \in E, f(e) \leq c(e)$

ii) $\sum_{e \in O(s)} f(e) = 1 + 2 + 3 = 6 = 2 + 4 = \sum_{e \in I(t)} f(e)$ (OK)

iii) Para $\{x, y, z\}$ temos

$$\sum_{e \in I(x)} f(e) = 3 + 0 + 0 = 1 + 2 = \sum_{e \in O(x)} f(e) \quad (\text{OK})$$

$$\sum_{e \in I(y)} f(e) = 2 = 0 + 2 = \sum_{e \in O(y)} f(e) \quad (\text{OK})$$

$$\sum_{e \in I(z)} f(e) = 1 + 1 + 2 = 0 + 4 = \sum_{e \in O(z)} f(e) \quad (\text{OK})$$

Portanto, temos de fato um fluxo válido.

Notação:

$$f^{\text{in}}(v) = \sum_{e \in I(v)} f(e) \qquad f^{\text{out}}(v) = \sum_{e \in O(v)} f(e)$$

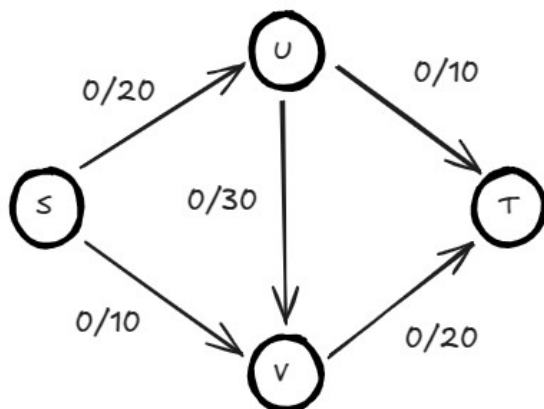
O Problema do Fluxo Máximo

Dado $G = (V, E, c)$ qual o máximo valor de $v(f)$ que pode chegar em t ?

Ideia geral

1. Condição inicial: $f(e) = 0, \forall e \in E$
2. Iteração: encontrar um caminho s - t e transmitir fluxo
3. Condição de parada: Todo caminho s - t encontra-se saturado

Exemplo:



Caso 1. $P = s \rightarrow u \rightarrow v \rightarrow t$
 $v(f) = 20$

Caso 2. $P_1 = s \rightarrow u \rightarrow t$ $v(f) = 10$
 $P_2 = s \rightarrow v \rightarrow t$ $v(f) = 10 + 10 = 20$
 $P_3 = s \rightarrow u \rightarrow v \rightarrow t$ $v(f) = 20 + 10 = 30$

Note que em cada um dos casos, o valor do fluxo obtido é diferente. Não é bom que o valor do fluxo dependa da escolha dos caminhos, pois senão o algoritmo iria chegar a resultados diferentes para um mesmo problema. O ideal é que o fluxo máximo seja obtido independente da escolha dos caminhos. Para isso iremos definir o grafo residual.

Def: Grafo Residual $G_f = (V_f, E_f)$ gerado a partir de $G = (V, E, c)$

- i) $V_f = V$ (conjunto de vértices é o mesmo)
- ii) $E_f \neq E$ pois $\forall e \in E$ pode gerar até 2 arestas em E_f
 - a) Forward-Edge (FE): na mesma direção da aresta e
 $\forall e \in E$ tal que $f(e) < c(e)$, $\exists$ em E_f uma aresta e' com capacidade residual $c(e') = c(e) - f(e)$ (exatamente o que falta para saturar)
 - b) Backward-Edge (BE): na direção contrária a aresta e
 $\forall e \in E$ tal que $f(e) > 0$, $\exists$ em E_f uma aresta e'' com capacidade residual $c(e'') = f(e)$ (exatamente o que já passa)

OBS: Note que no início não existem Backward-Edges pois os fluxos iniciais são nulos

RESUMO

1. $\forall e \in E$ com $f(e) = 0$ gera apenas Forward-Edge e'
2. $\forall e \in E$ com $f(e) = c(e)$ gera apenas Backward-Edge e''
3. $\forall e \in E$ com $0 < f(e) < c(e)$ gera 2 arestas: e' (FE) e e'' (BE)



Caminhos aumentados

Para melhorar um fluxo inicial com valor f , devemos buscar por caminhos P_{st} no grafo residual $G_f = (V_f, E_f)$. A primitiva Augment que realiza essa operação de melhorar um fluxo f é dada por:

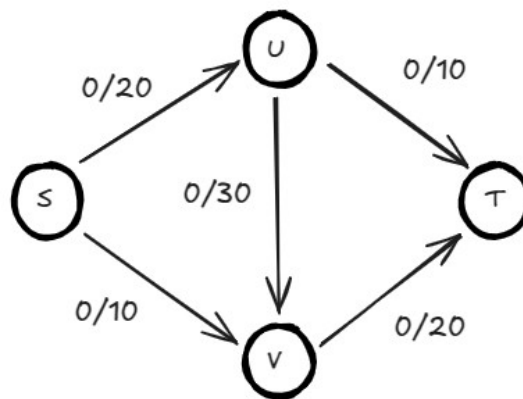
melhora fluxo f utilizando o caminho P em G_f

```

Augment( $f$ ,  $P$ ) {
   $b = \text{gargalo}(P)$  # é o máximo que podemos passar por  $P$ 
  for each  $e = (u, v)$  in  $P$  {
    if  $e$  está a favor do fluxo  $s$ - $t$  em  $G$  (F.E.)
       $f(e) = f(e) + b$ 
    else (B.E.)
       $f(e) = f(e) - b$ 
  }
  return  $f$ 
}

```

Exemplo:



Passo 1: Grafo residual é o próprio G

$f = 0$

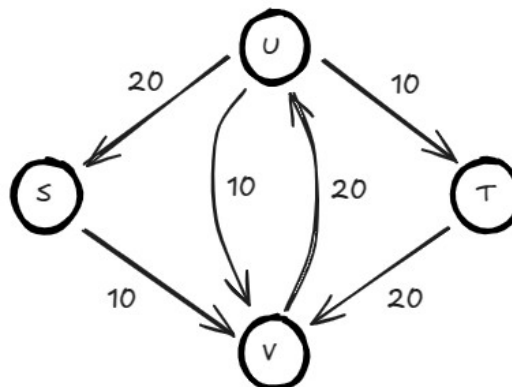
$P = \text{s} \rightarrow \text{u} \rightarrow \text{v} \rightarrow \text{t}$

$b = 20$

$f(\text{s} \rightarrow \text{u}) = 0 + 20 = 20$

$f(\text{u} \rightarrow \text{v}) = 0 + 20 = 20$

$f(\text{v} \rightarrow \text{t}) = 0 + 20 = 20$



Passo 2:

$f = 20$

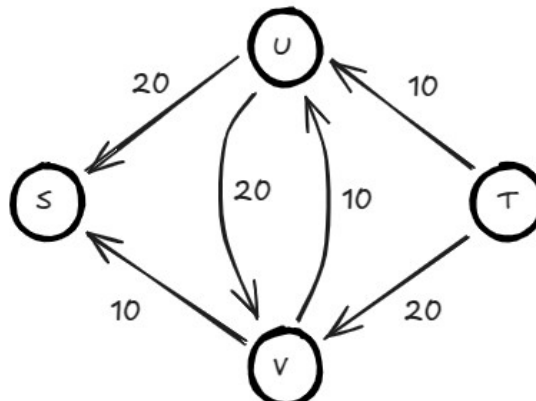
$P = \text{s} \rightarrow \text{v} \rightarrow \text{u} \rightarrow \text{t}$

$b = 10$

$f(\text{s} \rightarrow \text{v}) = 10 + 10 = 20$

Contrária a aresta (u, v)

$f(\text{v} \rightarrow \text{u}) = f(\text{u} \rightarrow \text{v}) - 10 = 20 - 10 = 10$



Passo 3:

$f = 30$

$\nexists P_{st}$ em $G_f \rightarrow \text{PARE}$

=> O fluxo máximo vale 30

Teorema: O resultado da operação $f' = \text{Augment}(f, P)$ é um fluxo válido.

i) f' não excede a capacidade (no caso de F.E.)

$$f'(e) = f(e) + b \leq f(e) + (c(e) - f(e))$$

pois para qualquer aresta F.E. do caminho $c(e) - f(e)$ será maior ou igual que b

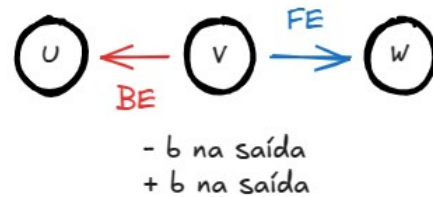
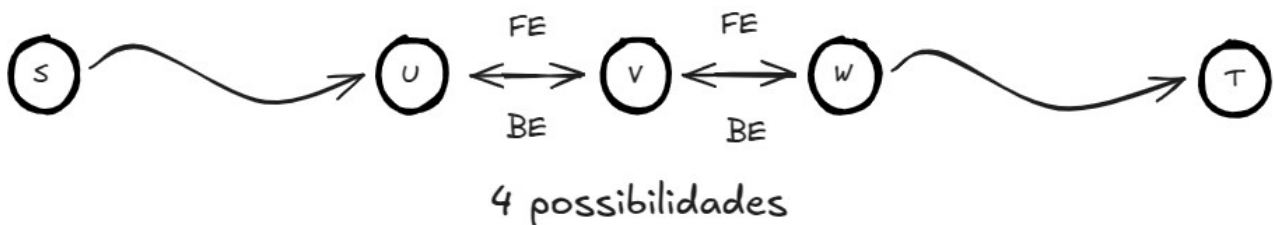
ii) f' nunca é inferior a zero (no caso de B.E.)

$$f(e) \geq f'(e) = f(e) - b \geq f(e) - (f(e) - 0) = 0$$

pois para qualquer aresta B.E. do caminho $f(e)$ será maior ou igual que b

iii) conservação (tudo que entra em v sai de v)

$$f^{\text{in}}(v) = f^{\text{out}}(v) \quad \text{para todo nó interno } v$$



mostrando que em qualquer um dos casos, a conservação do fluxo é preservada!

Portanto, a operação $\text{Augment}(f, P)$ preserva fluxos e podemos usá-la para melhorar um dado fluxo. Veremos a seguir uma sequência lógica de passos para obter o fluxo máximo em um grafo: o algoritmo de Ford-Fulkerson, um algoritmo guloso pois tenta passar o máximo de fluxo possível em cada caminho aumentado.

```

Max_Flow( $G, s, t, c$ ) {
   $f = 0$ 
  for each  $e \in E$ 
     $f(e) = 0$ 
  while  $\exists P_{st}$  in  $G_f$  {

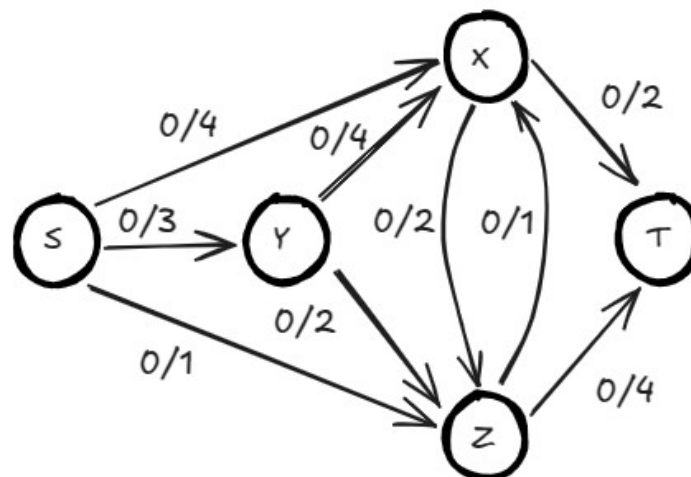
```

```

    Seja  $P_{st}$  um caminho  $P_{st}$  no grafo residual
     $f' = \text{Augment}(f, P_{st})$ 
    Atualize o grafo residual  $G_f$ 
  }
}

```

Exemplo: Utilizando ao algoritmo de Ford-Fulkerson, encontre o fluxo máximo



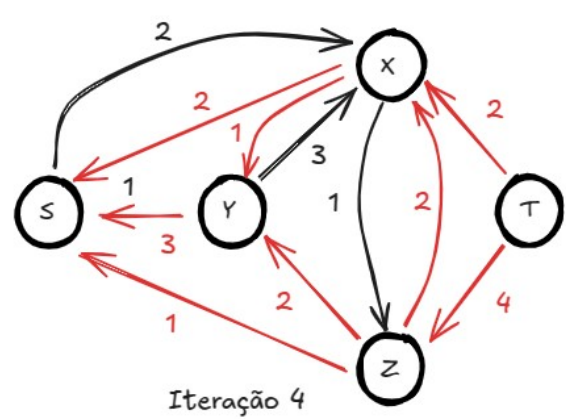
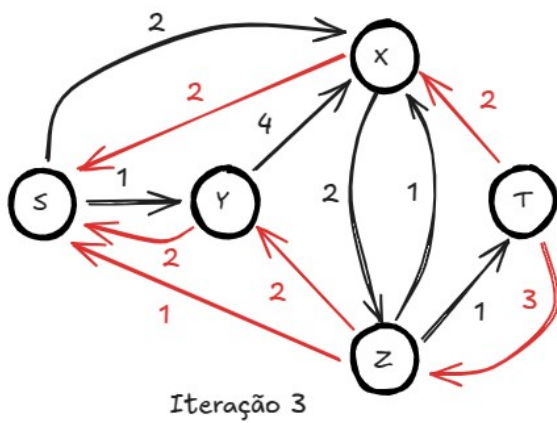
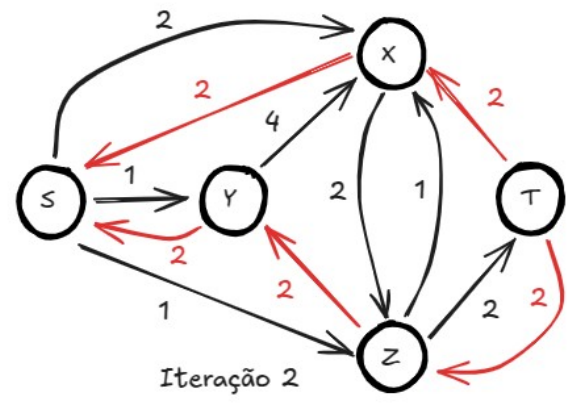
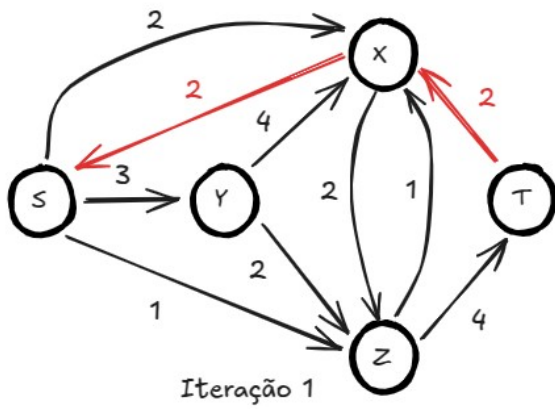
Trace do algoritmo

i	P_{st}	b	$f(e), \forall e \in P_{st}$	f
1	sxt	2	$f(sx) = 0 + 2 = 2$ $f(xt) = 0 + 2 = 2$	$0 + 2 = 2$
2	syzt	2	$f(sy) = 0 + 2 = 2$ $f(yz) = 0 + 2 = 2$ $f(zt) = 0 + 2 = 2$	$2 + 2 = 4$
3	sz	1	$f(sz) = 0 + 1 = 1$ $f(zt) = 2 + 1 = 3$	$4 + 1 = 5$
4	syxzt	1	$f(sy) = 2 + 1 = 3$ $f(yx) = 0 + 1 = 1$ $f(xz) = 0 + 1 = 1$ $f(zt) = 3 + 1 = 4$	$5 + 1 = 6$

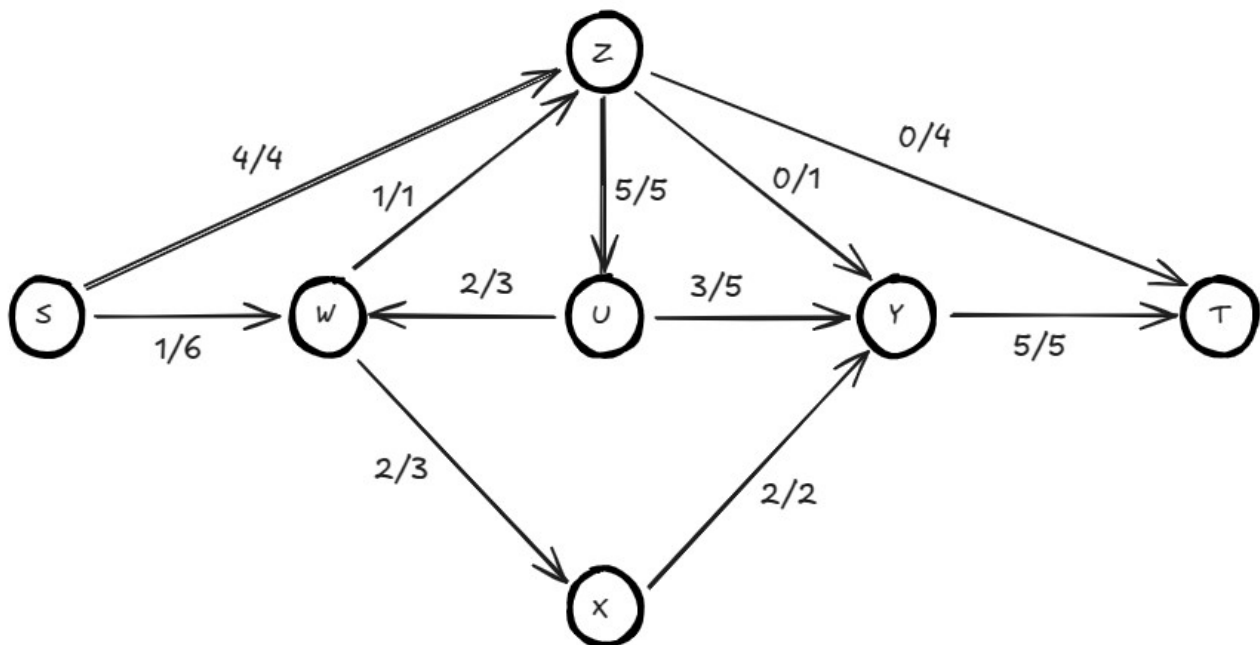
Portanto, o fluxo máximo vale 6.

A seguir encontram-se os grafos residuais após cada um dos passos do algoritmo.

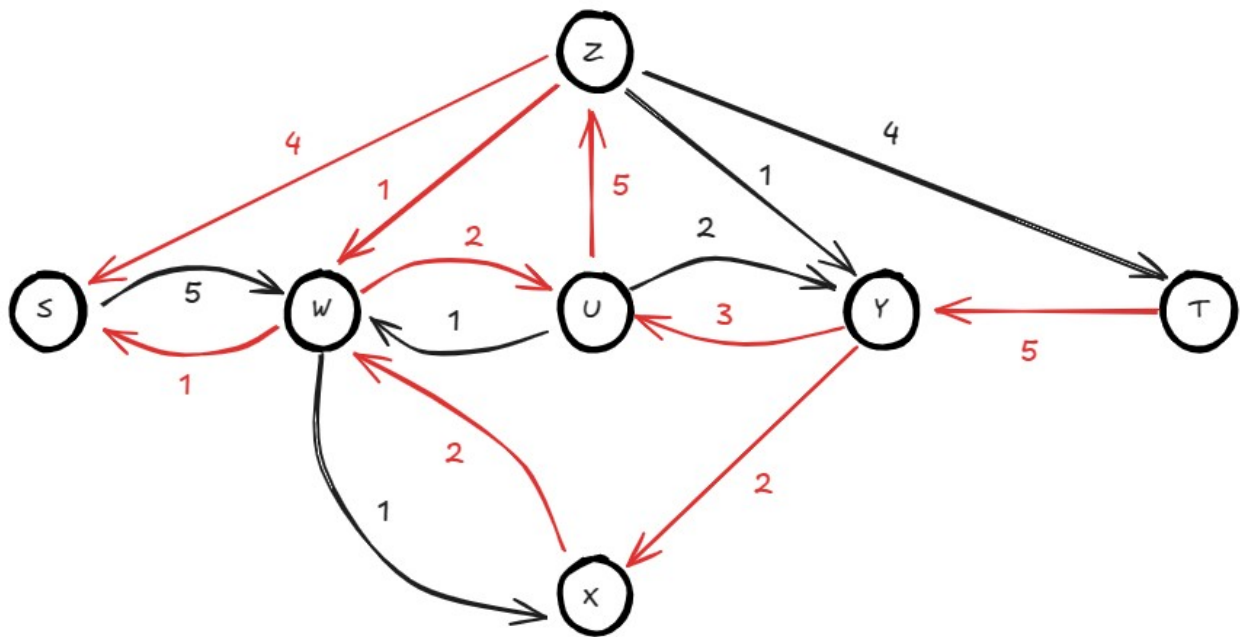
Note que a cada iteração, após passarmos um fluxo pelo caminho st , o grafo residual é modificado para refletir as novas capacidades restantes.



Exercício: Dado o grafo G a seguir, responda: o fluxo dado é máximo? Justifique sua resposta.



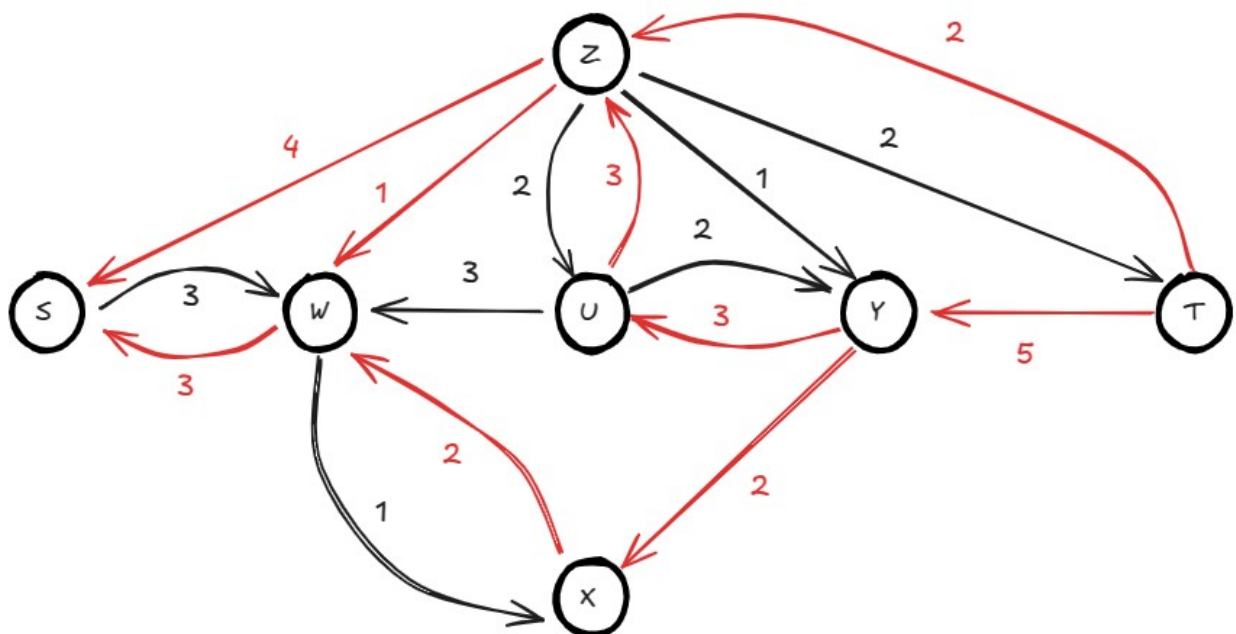
Para verificar se o fluxo dado é máximo, devemos observar o grafo residual. Sendo assim, o grafo residual de G fica:



Note que $\exists P_{st} = swuzt$ no grafo residual, portanto o fluxo não pode ser máximo. Como o gargalo do caminho é 2, podemos aumentar o fluxo nessas arestas de duas unidades, ou seja:

$f(sw) = 1 + 2 = 3$, pois a aresta (s,w) está alinhada ao fluxo em G
 $f(wu) = 2 - 2 = 0$, pois a aresta (w,u) está ao contrário do fluxo em G
 $f(uz) = 5 - 2 = 3$, pois a aresta (u,z) está ao contrário do fluxo em G
 $f(zt) = 0 + 2 = 2$, pois a aresta (z,t) está alinhada ao fluxo em G

De modo que o valor total do fluxo agora é 7. O grafo residual é atualizado para:



Note que $\nexists P_{st}$ no grafo residual. Portanto, podemos concluir agora que o fluxo é máximo.

Fluxos e cortes

Aplicações: segmentação de imagens, classificação supervisionada, emparelhamentos

Motivação: cortes especificam limites superiores para $v(f)$ (valor do fluxo máximo). A relação entre fluxo e cortes nos permite resolver um problema NP-Hard (corte mínimo em grafos) em tempo polinomial.

Ideia: Ao particionar G em 2 componentes A e B , a capacidade das arestas cortadas impõe um limite superior para o fluxo que pode sair de A e chegar em B .

Def: Um corte s - t em G é qualquer partição de V em A e B tal que $s \in A$ e $t \in B$

Def: A capacidade de um corte é definida como:

$$c(A, B) = \sum_{e \in O(A)} c(e) \quad (\text{soma das capacidades das arestas que saem de } A \text{ e chegam em } B)$$

Min-cut/Max-flow: resultado que mostra a equivalência entre 2 problemas de otimização duais.

=> Encontrar o fluxo máximo corresponde a encontrar o corte de mínima capacidade (2 problemas que podem ser resolvidos com um único algoritmo)

Para provar o teorema Min-cut/Max-flow, iremos primeiramente demonstrar uma série de resultados preliminares importantes que serão utilizados depois.

Teorema: Seja f um fluxo s - t e (A, B) um corte s - t . Então:

$$v(f) = f^{\text{out}}(A) - f^{\text{in}}(A) \quad (\text{ou seja, o valor do fluxo depende diretamente do corte: tudo que sai de } A \text{ menos tudo que entra em } A)$$

$$v(f) = \sum_{e \in O(s)} f(e) = f^{\text{out}}(s)$$

Sabe-se que $f^{\text{in}}(s) = 0$ então podemos escrever $v(f) = f^{\text{out}}(s) - f^{\text{in}}(s)$

Mas também sabemos que $\forall v \in V - \{s, t\}$ (nó intermediário), pela conservação do fluxo vale:

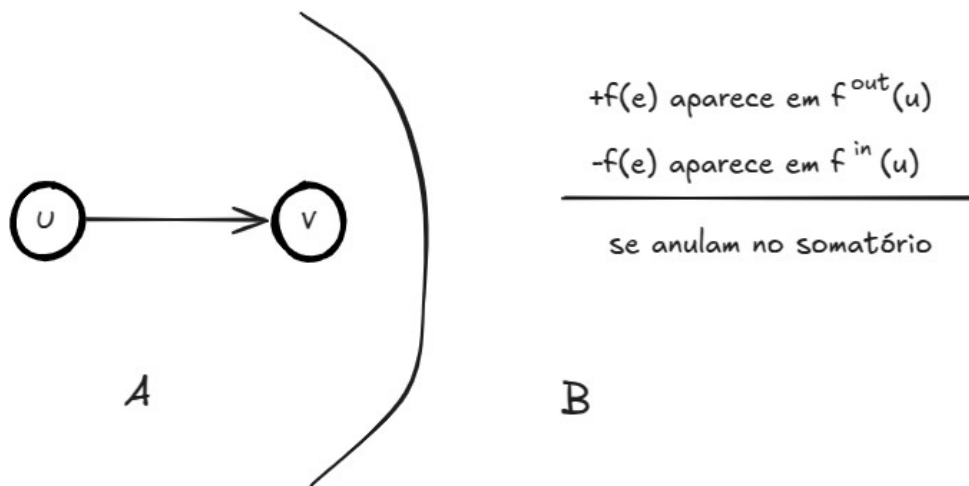
$$f^{\text{in}}(v) = f^{\text{out}}(v) \Leftrightarrow f^{\text{out}}(v) - f^{\text{in}}(v) = 0$$

Desse modo, podemos expressar $v(f)$ como:

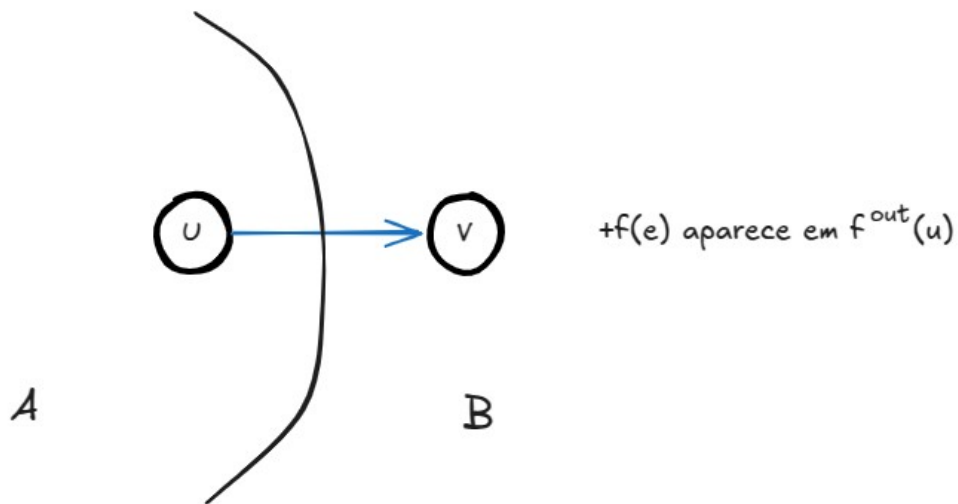
$$v(f) = \sum_{v \in A} (f^{\text{out}}(v) - f^{\text{in}}(v)) \quad (\text{todos os termos desse somatório são nulos, exceto para } s)$$

Porém, existem apenas 4 tipos de arestas em G :

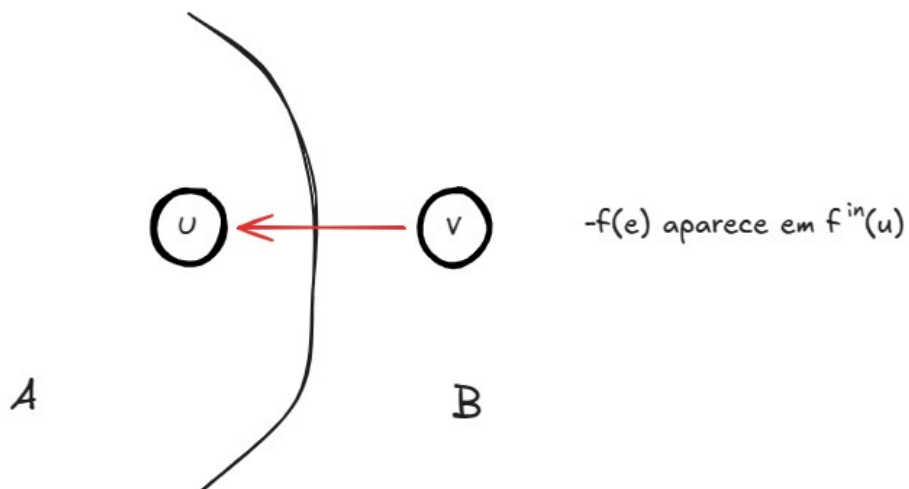
i) $e = \langle u, v \rangle$ com $u, v \in A$



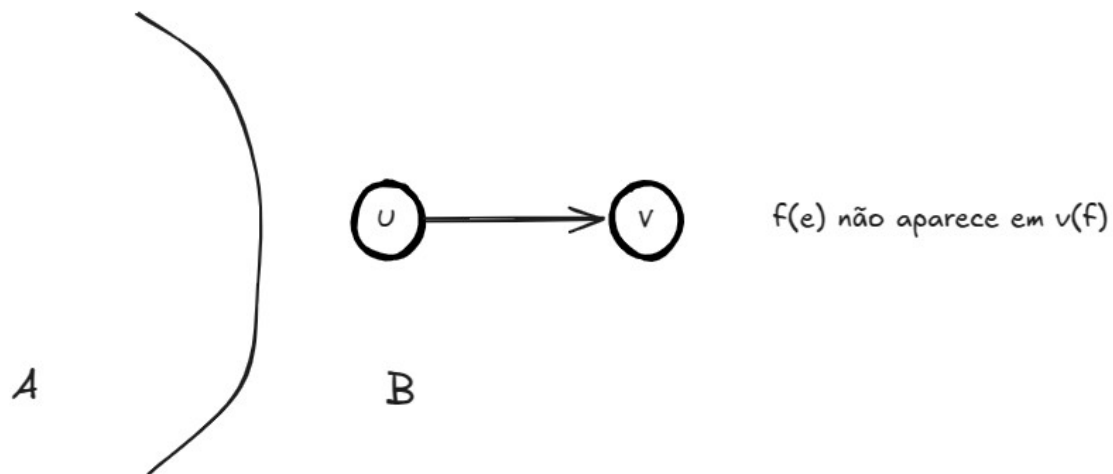
ii) $e = \langle u, v \rangle$ com $u \in A$ e $v \in B$



iii) $e = \langle v, u \rangle$ com $u \in A$ e $v \in B$



iv) $e = \langle u, v \rangle$ com $u, v \in B$



Portanto, $v(f)$ pode ser expresso por:

$$v(f) = \sum_{e \in O(A)} f(e) - \sum_{e \in I(A)} f(e) = f^{\text{out}}(A) - f^{\text{in}}(A)$$

OBS: Note que $f^{\text{out}}(A) - f^{\text{in}}(A) = f^{\text{in}}(B) - f^{\text{out}}(B)$

Teorema: Seja um fluxo s-t qualquer e (A, B) um corte s-t. Então, $v(f) \leq c(A, B)$

$$v(f) = f^{\text{out}}(A) - f^{\text{in}}(A) \leq f^{\text{out}}(A) = \sum_{e \in O(A)} f(e) \leq \sum_{e \in O(A)} c(e) = c(A, B)$$

OBS:

a) Se encontrarmos um fluxo s-t máximo f^* , não existe corte $c(A, B)$ com capacidade menor que $v(f^*)$.

b) Se encontrarmos um corte s-t (A, B) com capacidade mínima c^* , não existe fluxo s-t com valor maior que c^*

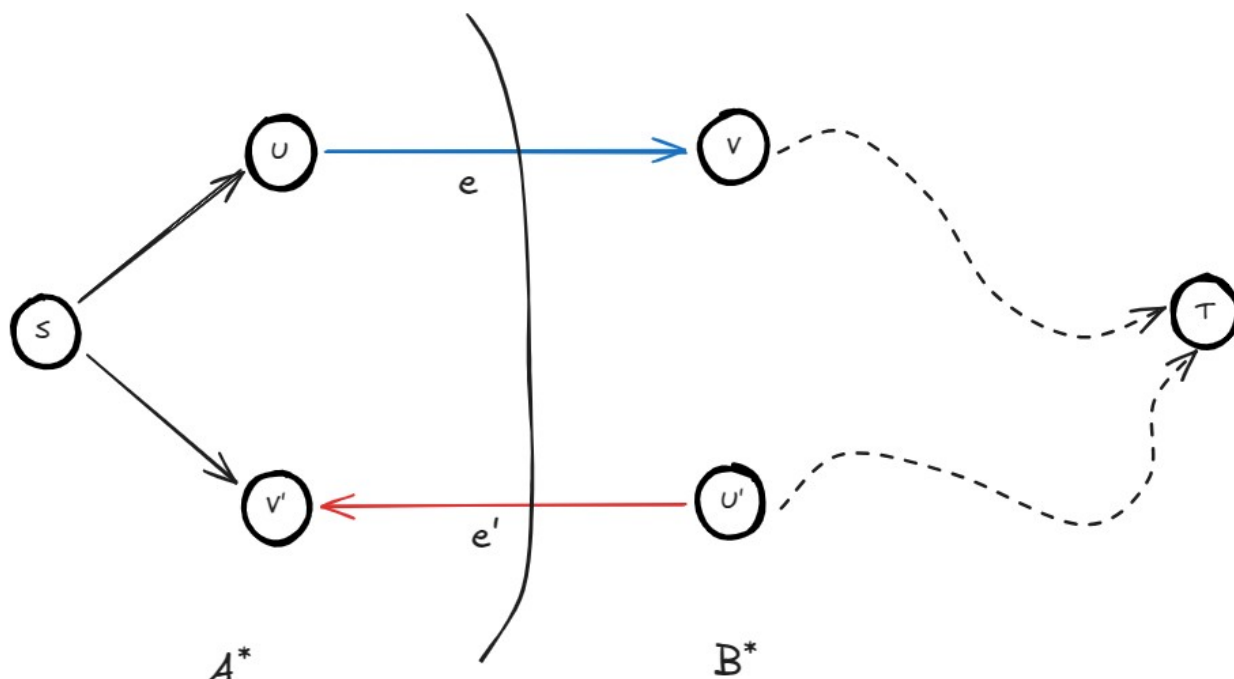
Teorema (Min-cut/Max-flow): Seja f^* o fluxo s-t tal que $\nexists P_{st}$ em G_f (fluxo gerado pelo algoritmo Ford-Fulkerson). Então, existe um corte (A^*, B^*) para qual o valor $v(f^*) = c(A^*, B^*)$. Consequentemente, f^* tem máximo valor de fluxo e o corte (A^*, B^*) tem a mínima capacidade dentre todos os possíveis. (encontrar fluxo máximo nos dá corte mínimo, o que era NP-hard).

Prova:

Seja $G_f = (V_f, E_f)$ o grafo residual de G no momento em que não existe mais caminho P_{st} . Podemos então dividir o conjunto V em 2 partições:

$$A^* = \{v \in V_f / \exists P_{sv}\} \quad (\text{atingíveis a partir de } s)$$

$$B^* = V_f - A^* \quad (\text{não atingíveis a partir de } s)$$



Note que:

a) $t \in B^*$ pois $\nexists P_{st}$ em G_f

b) Vamos analisar a aresta $e = \langle u, v \rangle$. Essa aresta certamente está saturada, ou seja, $f(e) = c(e)$, pois caso contrário haveria caminho P_{sv} uma vez que a capacidade residual da aresta F.E. correspondente em G_f seria $c(e) - f(e)$. Assim, podemos generalizar essa observação e escrever o seguinte fato:

$\forall e \in O(A^*)$ tem $f(e) = c(e)$ (todas as arestas que saem de A são saturadas)

c) Análise da aresta $e' = \langle u', v' \rangle$. Essa aresta certamente não carrega fluxo algum ($f(e)$ nulo), pois caso contrário seria gerado uma aresta $e'' = \langle v', u' \rangle$ em G_f (B.E.) com capacidade residual $f(e')$, o que resultaria num caminho $P_{su'}$. Assim, podemos generalizar essa observação para:

$\forall e \in I(A^*)$ tem $f(e) = 0$ (todas as arestas que entram em A tem fluxo nulo)

Em resumo, se (A^*, B^*) é um corte, então:

$\forall e \in O(A^*)$, $f(e) = c(e)$

$\forall e \in I(A^*)$, $f(e) = 0$

Calculando o valor do fluxo temos:

$$v(f) = f^{\text{out}}(A^*) - f^{\text{in}}(A^*) = \sum_{e \in O(A^*)} f(e) - \sum_{e \in I(A^*)} f(e) = \sum_{e \in O(A^*)} c(e) = c(A^*, B^*) \quad (\text{min. capac. corte})$$

OBS: A saída do Ford-Fulkerson, $v(f)$, é de fato o valor de $c(A^*, B^*)$. Os conjuntos A^* e B^* são obtidos por uma simples busca em largura a partir de s em G_f .

Bibliografia

CORMEN, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. Introduction to Algorithms. 4ª edição, The MIT Press, 2022.

ROUGHGARDEN, T., Algorithms Illuminated: omnibus edition, Soundslikeyourself publishing, 2022.

KLEINBERG, J.; TARDOS, E. Algorithmic Design, Addison Wesley, 2005.

LEVITIN, A. Introduction to the Design and Analysis of Algorithms, 3ª edição, Pearson, 2012.

SKIENA, S. S. The Algorithm Design Manual, 2ª edição, Springer, 2008.

SRIDHAR, S., Design and Analysis of Algorithms, Oxford University Press, 2014.

R. SEDGEWICK, K. WAYNE. Algorithms, 4th. ed., Addison-Wesley, 2011.

BHASIN, H., Algorithms: Design and Analysis, Oxford University Press, 2015.

S. DASGUPTA, C.H. PAPADIMITRIOU, U.V. VAZIRANI. Algorithms, McGraw-Hill, 2007.

SZWARCFITER, J. L. Teoria Computacional de Grafos: Os algoritmos, Elsevier, 2018.

ZIVIANI N. Projeto de algoritmos: com implementações em Java e C++. 2. ed. São Paulo: Cengage Learning, 2011.

NICOLETTI, M. C.; HRUSCHKA, E. R. Fundamentos da Teoria dos Grafos para Computação, 2ª ed., Série Apontamentos, EdUFSCar, 2009.

CLARK, J., DEREK, A. H. A First Look at Graph Theory, World Scientific, 1998.

Geeks for Geeks – A computer science portal for geeks. <https://www.geeksforgeeks.org/>

Sobre o autor

Alexandre L. M. Levada concluiu o curso de graduação em Bacharelado em Ciência da Computação no Departamento de Estatística, Matemática e Computação (DEMAC) da Universidade Estadual Paulista "Júlio de Mesquita Filho" (UNESP), em Rio Claro, no ano de 2002. Concluiu o mestrado em Ciência da Computação, na Universidade Federal de São Carlos (UFSCar), em fevereiro de 2006. Concluiu o doutorado em Física Computacional no Instituto de Física de São Carlos (IFSC) da USP em 2010 sob a orientação do Prof. Dr. Nelson Delfino d'Ávila Mascarenhas e co-orientação do Prof. Dr. Alberto Tannús. Atualmente é Professor do Departamento de Computação na Universidade Federal de São Carlos. Atua nas áreas de processamento de imagens e sinais e reconhecimento de padrões. Possui experiência em ministrar diversas disciplinas para cursos de Ciência e Engenharia de Computação, como Lógica Matemática, Teoria dos Grafos, Matemática Discreta, Introdução à Computação, Programação e Algoritmos 1 e 2, Programação Científica, Algoritmos e Estruturas de Dados 1 e 2, Projeto e Análise de Algoritmos, Otimização Matemática, Processamento Digital de Imagens e Reconhecimento de Padrões.

"You are a piece of the puzzle of someone else's life. You may never know where you fit, but others will fill the holes in their lives with pieces of you."

-- Bonnie Arbon